PERANCANGAN DAN IMPLEMENTASI SECURITY OPERATIONS CENTER BERBASIS OPEN SOURCE UNTUK DETEKSI DAN MITIGASI ANCAMAN SIBER

TUGAS AKHIR

Oleh:

FERDI SYAMADRI

2201081019



PROGRAM STUDI TEKNIK KOMPUTER

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI PADANG

2025

PERANCANGAN DAN IMPLEMENTASI SECURITY OPERATIONS CENTER BERBASIS OPEN SOURCE UNTUK DETEKSI DAN MITIGASI ANCAMAN SIBER

TUGAS AKHIR

Diajukan sebagai salah satu syarat untuk mendapatkan gelar Ahli Madya pada Jurusan Teknologi Informasi Program Studi D3 Teknik Komputer

FERDI SYAMADRI

2201081019

PROGRAM STUDI TEKNIK KOMPUTER

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI PADANG

2025

PERANCANGAN DAN IMPLEMENTASI SECURITY OPERATIONS CENTER BERBASIS OPEN SOURCE UNTUK DETEKSI DAN MITIGASI ANCAMAN SIBER

TUGAS AKHIR

Diajukan sebagai salah satu syarat untuk mendapatkan gelar Ahli Madya pada Jurusan Teknologi Informasi Program Studi D3 Teknik Komputer

Oleh:

FERDI SYAMADRI

2201081019

Tugas Akhir Ini Telah Diperiksa Dan Disetujui Oleh:

Pembimbing I	Pembimbing II
Alde Alanda, S.Kom, M.T	Andre Febrian Kasmar, S.T., M.T
NIP. 19880825 201504 1 002	NIP. 19880220 201903 1 009

Disahkan Oleh:

Ketua Jurusan Teknologi Informasi	Ketua Program Studi Teknik Komputer
Ronal Hadi, S.T., M.Kom	Andre Febrian Kasmar, S.T., M.T
NIP. 19760129 200212 1 001	NIP. 19880220 201903 1 009

PERANCANGAN DAN IMPLEMENTASI SECURITY OPERATIONS CENTER BERBASIS OPEN SOURCE UNTUK DETEKSI DAN MITIGASI ANCAMAN SIBER

Oleh:

FERDI SYAMADRI

2201081019

Tugas akhir ini telah diuji dan dipertahankan didepan tim penguji sidang Tugas Akhir Diploma III Politeknik Negri Padang Pada Hari Rabu Tanggal 27 Agustus 2025

Tim Penguji

Ketua	Sekretaris
Alde Alanda, S.Kom, M.T	Hidra Amnur, S.E., S.Kom, M.Kom
NIP. 19880825 201504 1 002	NIP. 19820415 201212 1 002

Disahkan Oleh:

Anggota	Anggota
Humaira, S.T, M.T	Rostam Ahmad Efendi, S.Pd., S.H., M.MSI
NIP. 19810319 200604 2 002	NIP. 19720305 202321 1 003

ABSTRAK

Perkembangan teknologi informasi telah mendorong digitalisasi proses bisnis, namun juga meningkatkan kompleksitas ancaman siber yang dapat menimbulkan kerugian finansial, operasional, dan reputasi. Keterbatasan biaya dan sumber daya membuat banyak organisasi menunda implementasi *Security Operations Center* (SOC) konvensional, padahal kebutuhan deteksi dan respons insiden yang cepat semakin mendesak. Penelitian ini dilakukan untuk merancang dan mengimplementasikan SOC berbasis *open source* sebagai solusi efisien dan terjangkau. Objek penelitian melibatkan integrasi Wazuh sebagai SIEM, Suricata sebagai IDS, Zabbix untuk monitoring utilisasi, Grafana untuk visualisasi, TheHive untuk manajemen kasus, Cortex untuk *threat intelligence enrichment*, Zammad untuk manajemen tiket, serta n8n untuk orkestrasi otomasi. Penelitian diselesaikan melalui pengujian berbasis skenario, mencakup simulasi serangan dan kondisi kegagalan layanan, untuk menilai kemampuan sistem dalam deteksi, notifikasi, dan otomasi pembuatan kasus maupun tiket. Hasil pengujian menunjukkan bahwa SOC berbasis *open source* mampu mendeteksi *security alert* (seperti *network scanning* dan *forced browsing*) serta *utilization alert* (seperti layanan Apache2 berhenti, dan host direstart). Seluruh alert berhasil diteruskan otomatis ke TheHive, Telegram, dan Zammad melalui n8n, sehingga membuktikan efektivitas SOC berbasis *open source* dalam mempercepat deteksi dan respons insiden.

Kata kunci: Security Operations Center, **Open Source**, **SIEM**, **IDS**, **Automasi**

ABSTRACT

The development of information technology has driven business digitalization, yet it has also increased the complexity of cyber threats that may cause financial, operational, and reputational losses. Limited budgets and resources often discourage organizations from implementing a conventional Security Operations Center (SOC), even though rapid detection and response are urgently needed. This research was conducted to design and implement an open-source-based SOC as an efficient and affordable solution. The study integrates Wazuh as a SIEM, Suricata as an IDS, Zabbix for utilization monitoring, Grafana for visualization, TheHive for case management, Cortex for threat intelligence enrichment, Zammad for ticket management, and n8n for workflow orchestration. The research was completed through scenario-based testing, including simulated attacks and service failure conditions, to evaluate the system's ability in detection, notification, and automated case or ticket creation. The results show that the open-source SOC is capable of detecting security alerts (such as network scanning and forced browsing) as well as utilization alerts (such as Apache2 service downtime and host restart). All alerts were automatically forwarded to TheHive, Telegram, and Zammad through n8n, proving the effectiveness of an open-source SOC in accelerating incident detection and response.

Keywords: Security Operations Center, **Open Source**, **SIEM**, **IDS**, **Automation**

KATA PENGANTAR

Dengan menyebut nama Allah SWT yang Maha Pengasih lagi Maha Penyayang, penulis panjatkan puji syukur atas limpahan rahmat dan karunia-Nya sehingga laporan Tugas Akhir berjudul "Perancangan dan Implementasi Security Operations Center Berbasis Open Source untuk Deteksi dan Mitigasi Ancaman Siber" dapat diselesaikan.

Laporan ini disusun sebagai salah satu syarat memperoleh gelar Ahli Madya Teknik (A.Md.T) pada Program Studi Teknik Komputer, Jurusan Teknologi Informasi, Politeknik Negeri Padang. Penulis menyadari bahwa penyusunan laporan ini tidak terlepas dari dukungan, arahan, dan bantuan berbagai pihak. Untuk itu penulis menyampaikan terima kasih yang sebesar-besarnya kepada semua pihak yang telah membantu, khususnya keluarga, dosen, dan pembimbing.

1. Orang tua dan keluarga tercinta atas doa, dukungan, dan motivasi yang tiada henti.
2. Bapak Ronal Hadi, S.T., M.Kom., selaku Ketua Jurusan Teknologi Informasi, Politeknik Negeri Padang.
3. Bapak Andre Febrian Kasmar, S.T., M.T., selaku Ketua Program Studi Teknik Komputer, Jurusan Teknologi Informasi, Politeknik Negeri Padang.
4. Bapak Alde Alanda, S.Kom, M.T selaku dosen pembimbing yang telah memberikan arahan, bimbingan, dan masukan yang sangat berharga selama penyusunan Tugas Akhir ini.

Akhir kata, penulis berharap semoga Allah SWT membalas segala kebaikan semua pihak yang telah membantu. Penulis juga berharap laporan Tugas Akhir ini dapat memberikan manfaat dan kontribusi positif bagi pengembangan ilmu pengetahuan, khususnya di bidang Teknologi Informasi dan keamanan siber.

Padang, Agustus 2025

Ferdi Syamadri

DAFTAR ISI

HALAMAN JUDUL i
HALAMAN PEROLEHAN GELAR ii
HALAMAN PENGESAHAN iii
HALAMAN PENGUJI iv
ABSTRAK v
ABSTRACT vi
KATA PENGANTAR vii
DAFTAR ISI viii
DAFTAR TABEL xi
DAFTAR GAMBAR xii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 3
1.3 Tujuan 3
1.4 Manfaat 4
1.5 Batasan Masalah 4
1.6 Metodologi 6
BAB II LANDASAN TEORI 8
2.1 Teori Tentang Sistem 8
2.1.1 Keamanan Siber (Cybersecurity) 8
2.1.2 Security Operations Center (SOC) 8
2.1.3 Security Information and Event Management (SIEM) 8
2.1.4 Intrusion Detection and Prevention System (IDS/IPS) 9
2.1.5 Security Orchestration, Automation, and Response (SOAR) 9
2.1.6 Komponen Sistem SOC Berbasis Open Source 9
2.1.7 Incident Handling & Ticketing System 11
2.1.8 Cyber Threat Intelligence 11
2.1.9 Amazon Elastic Compute Cloud (EC2) 12

BAB III ANALISIS DAN PERANCANGAN 13
3.1 Analisis Sistem yang Sedang Berjalan 13
3.2 Analisa Kebutuhan Sistem 14
3.2.1 Kebutuhan Perangkat Keras 14
3.2.2 Kebutuhan Perangkat Lunak 14
3.2.3 Kebutuhan Jariangan 16
3.3 Rancangan Sistem 21
3.3.1 Rancangan Flowchart 21
3.3.2 Rancangan Topologi 22
3.3.3 Rancangan Arsitektur Sistem 22
BAB IV IMPLEMENTASI DAN PENGUJIAN 27
4.1 Persiapan Lingkungan Implementas 27
4.2 Installasi dan konfigurasi komponen SOC 27
4.2.1 Wazuh Server 27
4.2.2 Suricata 31
4.2.3 Zabbix Server 34
4.2.4 Grafana 38
4.2.5 The Hive 40
4.2.6 Cortex 49
4.2.7 Zammad 59
4.2.8 N8N 66
4.2.9 Wazuh Agent 70
4.2.10 Zabbix Agent 72
4.3 Integrasi Antar Komponen 74
4.3.1 Integrasi Wazuh Dan Suricata 74
4.3.2 Integrasi Zabbix Dan Grafana 77
4.3.3 Automate Alert Wazuh Ke Thehive 82
4.3.4 Utilization Alert & Ticket Automation 85
4.3.5 Security Alert & Ticket Automation 105
4.4 Implementasi Scenario Pengujian 115
4.4.1 Deteksi serangan *"Scanning Attack"* 115

4.4.2 Deteksi Serangan "*Web Content Discovery / Forced Browsing"* 119
4.4.3 Mendeteksi *Host has been restarted* (Restart Client Dummy) 124
4.4.4 Mendeteksi custom service *(Apche2 Service Down)* 127
4.4.5 Pengujian Latensi & Performa End-to-End SOC..... 131
4.4.6 Estimasi AWS Penggunaan AWS ec2 134
BAB V PENUTUP..... 135
5.1 Kesimpulan 135
5.2 Saran..... 135
DAFTAR PUSTAKA..... 137

DAFTAR TABEL

Tabel 3. 1 kebutuhan perangkat keras 14
Tabel 3. 2 kebutuhan perangkat lunak..... 14
Tabel 3. 3 tools utama 15
Tabel 3. 4 depedensi tambahan 15
Tabel 3. 5 elastic ip..... 17
Tabel 3. 6 Inbound security group..... 17
Tabel 3. 7 Outbound security Group 20
Tabel 4. 1 Scanning Attack..... 131
Tabel 4. 2 Invalid Checksum..... 132
Tabel 4. 3 Web Content Discovery Attack 132
Tabel 4. 4 XSS Injection 132
Tabel 4. 5 Sql injection attempt..... 133
Tabel 4. 6 Apache2 Service is down 133
Tabel 4. 7 Client has been restarted 133
Tabel 4. 8 Aws Estimate 134

DAFTAR GAMBAR

Gambar 3. 1 Flowchart sistem 21
Gambar 3. 2 Rancangan topologi 22
Gambar 3. 3 rancangan arsitektur 22
Gambar 3. 4 arsitektur utilization alert 23
Gambar 3. 5 arsitektur security alert 24
Gambar 4. 1 hasil download wazuh 29
Gambar 4. 2 cek status wazuh manager 29
Gambar 4. 3 halaman login wazuh dashboard 30
Gambar 4. 4 Halama utama wazuh dahsboard 31
Gambar 4. 5 configurasi homenet & external net 32
Gambar 4. 6 cek interface 33
Gambar 4. 7 set static global 33
Gambar 4. 8 cek status suricata 34
Gambar 4. 9 zabbix installation assistant 35
Gambar 4. 10 cek status Zabbix server 37
Gambar 4. 11 configurasi awal wazuh server 37
Gambar 4. 12 halaman utama grafana 40
Gambar 4. 13 java version 41
Gambar 4. 14 cluster name thehive 42
Gambar 4. 15 listen address thehive 42
Gambar 4. 16 rpc address thehive 43
Gambar 4. 17 seed provider thehive 43
Gambar 4. 18 status cassandra 44
Gambar 4. 19 script allowed thehive 45
Gambar 4. 20 configurasi jvm 46
Gambar 4. 21 status elasticsearch 46
Gambar 4. 22 baseUrl thehive 47
Gambar 4. 23 database config thehive 48
Gambar 4. 24 lokasi storage thehive 48
Gambar 4. 25 status thehive 48
Gambar 4. 26 halaman login thehive 49
Gambar 4. 27 java version 50
Gambar 4. 28 configurasi elasticsearch cortex 51
Gambar 4. 29 group user cortex 52
Gambar 4. 30 cortex catalog 52
Gambar 4. 31 update database cortex 54
Gambar 4. 32 buat akun administrator cortex 54
Gambar 4. 33 organisasi default cortex 55

Gambar 4. 34 buat organisasi cortex 55
Gambar 4. 35 buat user baru cortex 56
Gambar 4. 36 set user passowrd 56
Gambar 4. 37 enable analyzer 57
Gambar 4. 38 set api key analyzer 57
Gambar 4. 39 mengambil api key abuseIpDb 58
Gambar 4. 40 test analysis 58
Gambar 4. 41 hasil analisys cortex 59
Gambar 4. 42 detail analysis cortex 59
Gambar 4. 43 enable elasticsearch 61
Gambar 4. 44 versi elasticsearch 61
Gambar 4. 45 cek local system 62
Gambar 4. 46 cek status nginx 63
Gambar 4. 47 cek file konfigurasi pada sites-enabled 64
Gambar 4. 48 set server name 64
Gambar 4. 49 konfigurasi trusted certificate 64
Gambar 4. 50 satup awal zammad 65
Gambar 4. 51 buat akun administrator zammad 66
Gambar 4. 52 proses docker compose 68
Gambar 4. 53 cek status container 68
Gambar 4. 54 buat akun owner n8n 68
Gambar 4. 55 mengaktifkan lisence key n8n 69
Gambar 4. 56 n8n lisence key 69
Gambar 4. 57 halaman utama n8n 70
Gambar 4. 58 wazuh agent installation assistant 1 71
Gambar 4. 59 wazuh agent installation assistant 2 71
Gambar 4. 60 wazuh agent status 72
Gambar 4. 61 zabbix agent status 74
Gambar 4. 62 set ossec.conf 75
Gambar 4. 63 suricata custom ping rules 75
Gambar 4. 64 suricata alert pada wazuh dahsboard 76
Gambar 4. 65 detail suricata alert 76
Gambar 4. 66 install Zabbix plugin 77
Gambar 4. 67 cari plugin Zabbix pada grafana UI 78
Gambar 4. 68 Enable Grafana plugin 78
Gambar 4. 69 menambahkan datasource zabbix 79
Gambar 4. 70 set data source 79
Gambar 4. 71 test data source 80
Gambar 4. 72 grafana template zabbix dashboard 80
Gambar 4. 73 import dashboard 80

Gambar 4. 74 setup dashboard 81
Gambar 4. 75 Grafana - zabbix dashboard 81
Gambar 4. 76 client-dummy dashboard 82
Gambar 4. 77 custom-w2thive.py 83
Gambar 4. 78 custom-w2thive 84
Gambar 4. 79 alert wazuh ke thehive 85
Gambar 4. 80 node webhook 86
Gambar 4. 81 konfigurasi AlertScriptsPath 86
Gambar 4. 82 n8n_alert.sh 87
Gambar 4. 83 buat media type baru 88
Gambar 4. 84 detail media type 89
Gambar 4. 85 buat user baru 90
Gambar 4. 86 set permission untuk user 90
Gambar 4. 87 set media type user 91
Gambar 4. 88 buat action baru 91
Gambar 4. 89 buat condition baru 91
Gambar 4. 90 buat operation baru 92
Gambar 4. 91 detail operation baru 93
Gambar 4. 92 status trigger action 93
Gambar 4. 93 node logger 94
Gambar 4. 94 node format message 94
Gambar 4. 95 membuat bot telegram 95
Gambar 4. 96 group telegram info 96
Gambar 4. 97 pesan testing 96
Gambar 4. 98 cek group id telegram 97
Gambar 4. 99 tes mengirimkan pesan menggunakan bot 97
Gambar 4. 100 create credential 98
Gambar 4. 101 cari tipe credential 98
Gambar 4. 102 set telegram credential 99
Gambar 4. 103 test credential telegram 99
Gambar 4. 104 menambahkan node telegram 100
Gambar 4. 105 konfigurasi node telegram 100
Gambar 4. 106 membuat akun cutommer zammad 101
Gambar 4. 107 membuat access token zammad 101
Gambar 4. 108 enable agent ticket 102
Gambar 4. 109 access token zammad 102
Gambar 4. 110 membuat credential zammad 103
Gambar 4. 111 menambahkan node zammad 103
Gambar 4. 112 konfigurasi node zammad 104
Gambar 4. 113 node zammad 104

Gambar 4. 114 aktifkan worflow 105
Gambar 4. 115 security automation workflow 106
Gambar 4. 116 node scedhule trigger 106
Gambar 4. 117 node search alert thehive 107
Gambar 4. 118 node if - cek tag 108
Gambar 4. 119 node harsing 108
Gambar 4. 120 node if severity 109
Gambar 4. 121 node update tag 109
Gambar 4. 122 node create case 110
Gambar 4. 123 node case message 111
Gambar 4. 124 node zammad 111
Gambar 4. 125 node telegram 112
Gambar 4. 126 node prepare ip 112
Gambar 4. 127 node analysis ip 113
Gambar 4. 128 node wait 113
Gambar 4. 129 node hasil cortex 114
Gambar 4. 130 node format message 114
Gambar 4. 131 node telegram 115
Gambar 4. 132 node zammad 115
Gambar 4. 133 wazuh alert 117
Gambar 4. 134 thehive alert 117
Gambar 4. 135 execution workflow 118
Gambar 4. 136 case thehive 118
Gambar 4. 137 notifikasi telgram 119
Gambar 4. 138 zammad ticket 119
Gambar 4. 139 wazuh alert 121
Gambar 4. 140 thehive alert 121
Gambar 4. 141 alert detail 122
Gambar 4. 142 false branch execution 122
Gambar 4. 143 cortex job 123
Gambar 4. 144 tpesan telegram 123
Gambar 4. 145 zammad ticket 124
Gambar 4. 146 reboot instance client 124
Gambar 4. 147 grafana dashboard 125
Gambar 4. 148 action log 125
Gambar 4. 149 worflow execution 126
Gambar 4. 150 pesan telegram 126
Gambar 4. 151 zammad ticket 127
Gambar 4. 152 custo items 127
Gambar 4. 153 custom trigger 128

Gambar 4. 154 mematikan apche2 service 128
Gambar 4. 155 grafana dashboard 129
Gambar 4. 156 action log 129
Gambar 4. 157 workflow execution 130
Gambar 4. 158 pesan telegram 130
Gambar 4. 159 zammad ticket 131

BAB I
PENDAHULUAN

1.1 Latar Belakang

Kemajuan infrastruktur teknologi informasi telah mendorong digitalisasi dan efisiensi proses bisnis pada berbagai sektor industri, pendidikan, hingga pemerintahan. Aktivitas digital yang semula dipandang sebagai pendukung kini menjadi tulang punggung operasional organisasi. Seiring dengan itu, muncul tantangan baru berupa ancaman siber yang semakin kompleks. Dampak dari insiden keamanan sangat beragam, mulai dari kehilangan data penting, kerugian finansial, hilangnya integritas sistem, hingga gangguan operasional dan rusaknya reputasi organisasi. Pada tingkat biaya, laporan IBM Cost of a Data Breach 2025 menunjukkan rata-rata biaya kebocoran data global sebesar USD 4,4 juta (penurunan 9% dibanding tahun sebelumnya), menegaskan urgensi penguatan kapabilitas deteksi dan penanggulangan insiden[1].

Hampir setiap bulan, berita kebocoran data atau serangan siber menghiasi media daring. Bukan hanya perusahaan besar yang menjadi korban, organisasi menengah ke bawah yang sebelumnya merasa “tidak menjadi target” pun tidak kebal terhadap serangan. Berbagai insiden telah membuktikan bahwa semua jenis organisasi, baik skala kecil maupun besar, berpotensi terdampak terutama bila pengendalian keamanan lemah atau pemantauan tidak memadai.

Dalam menghadapi tantangan tersebut, konsep Security Operations Center (SOC) hadir sebagai solusi terpusat untuk deteksi, pemantauan, dan respons insiden secara real-time. Kebutuhan akan kecepatan ini selaras dengan dinamika ancaman terkini. CrowdStrike melalui *Global Threat Report 2025* mencatat rata-rata *eCrime breakout time* adalah 48 menit dengan rekor tercepat 51 detik[2], sehingga celah waktu bagi pelaku untuk bergerak kian sempit serta menuntut respons yang lebih cepat dan terorkestrasi.

Namun, pembangunan SOC secara konvensional membutuhkan investasi yang signifikan, baik dari sisi infrastruktur, lisensi perangkat lunak, maupun sumber daya

manusia yang kompeten. Akibatnya, banyak organisasi kecil hingga menengah menunda atau mengabaikan implementasi SOC karena dianggap mahal, kompleks, dan sulit dioperasikan. Padahal, keterbatasan pemantauan membuka celah terhadap ancaman yang dapat terjadi kapan saja. Di sisi lain, laju serangan terus meningkat: Fortinet Global Threat Landscape 2025 melaporkan bahwa volume eksploitasi kerentanan melonjak, sementara rata-rata waktu dari publikasi kerentanan hingga dieksploitasi tetap mendekati 5 sampai dengan 4 hari, bahkan pada kasus tertentu seperti *CVE-2024-21887 (Ivanti),* eksploitasi terjadi enam hari setelah pengungkapan[3]. Kondisi ini menegaskan perlunya deteksi *real-time* dan pengayaan konteks ancaman dalam operasi keamanan.

Sebagai solusi atas kendala biaya dan kompleksitas, pemanfaatan teknologi *open source* menjadi pendekatan yang efisien dan terjangkau. Dengan mengintegrasikan Wazuh (SIEM), Suricata (IDS), Zabbix (monitoring), Grafana (visualisasi), TheHive (manajemen kasus), Cortex (enrichment intelijen ancaman), Zammad (*ticketing*), serta otomasi melalui n8n, organisasi dapat membangun SOC yang ekonomis, fleksibel, serta andal untuk mendeteksi dan merespons ancaman siber secara efektif. Penelitian ini bertujuan untuk merancang dan mengimplementasikan SOC berbasis *open source* yang dapat diterapkan pada organisasi menengah ke bawah maupun menengah ke atas, dengan fokus pada kemudahan penggunaan, efisiensi sumber daya, dan kemampuan adaptasi terhadap kebutuhan keamanan siber modern.

1.2 Rumusan Masalah

Berdararkan latar belakang yang telah diuraikan di atas, maka dapat dirumuskan beberapa permasalahan sebagai berikut:

1. Bagaimana merancang arsitektur Security Operation Center (SOC) berbasis *open source* yang efisien dan dapat diimplementasikan oleh organisasi skala menengah ke bawah maupun menengah ke atas?
2. Bagaimana cara mengintegrasikan berbagai *tools open source* seperti Wazuh, Suricata, Zabbix, Grafana, TheHive, Cortex, Zammad, dan n8n ke dalam satu sistem SOC yang utuh dan fungsional?
3. Sejauh mana sistem SOC berbasis *open source* ini mampu mendeteksi, memantau, dan merespons ancaman siber secara *real-time* dengan keterbatasan sumber daya?
4. Apakah solusi SOC berbasis *open source* ini dapat menjadi alternatif yang efektif dibandingkan SOC komersial dalam hal keandalan dan efisiensi operasional?

1.3 Tujuan

Adapun tujuan dari pembuatan Tugas Akhir ini adalah untuk mencapai beberapa hal yang menjadi fokus utama dalam penelitian ini, diantaranya:

1. Merancang dan mengimplementasikan sistem Security Operation Center (SOC) berbasis *open source* yang mudah dioperasikan, efisien, dan ekonomis.
2. Mengintegrasikan berbagai perangkat lunak open source (Wazuh, Suricata, Zabbix, Grafana, TheHive, Cortex, Zammad, dan n8n) untuk membentuk sistem SOC yang fungsional dan adaptif.
3. Menguji efektivitas dan keandalan sistem SOC dalam mendeteksi serta merespons ancaman siber secara real-time.

4. Memberikan solusi praktis dan dapat direplikasi oleh organisasi menengah ke bawah maupun menengah ke atas untuk meningkatkan keamanan informasi tanpa ketergantungan pada solusi komersial.

1.4 Manfaat

penilitian ini diharapkan dapat memberikan manfaat pada berbagai pihak sebagai berikut:

1. Bagi Akademisi

   Penelitian ini dapat menjadi referensi tambahan terkait penerapan SOC berbasis teknologi *open source*, khususnya bagi mahasiswa dan peneliti yang tertarik pada bidang keamanan informasi.
2. Bagi Praktisi Keamanan Siber

   Penelitian ini memberikan panduan praktis dalam membangun SOC berbasis *open source* yang ekonomis namun tetap efektif untuk mendeteksi dan menangani ancaman siber.
3. Bagi Organisasi

   Organisasi skala kecil hingga menengah dapat memanfaatkan hasil penelitian ini untuk meningkatkan tingkat keamanan informasi mereka yang hemat biaya namun tetap andal. Implementasi solusi ini diharapkan mampu meningkatkan postur keamanan siber tanpa bergantung pada solusi komersial yang mahal dan kompleks.
4. Bagi Peneliti dan Pengembang Sistem

   Hasil penelitian ini dapat menjadi dasar untuk penelitian lebih lanjut baik dalam segi otomatisasi, penguatan sistem, maupun integrasi dengan teknologi lain yang lebih cerdas dalam pengembangan solusi SOC yang lebih inovatif dan efisien.

1.5 Batasan Masalah

Untuk menetapkan fokus dari penelitian ini maka ditetapkan batasan masalah yang meliputi sebagai berikut:

1. Penggunaan *tools* dan Teknologi o*pen oource*

Penelitian ini difokuskan pada implementasi Security Operations Center (SOC) dengan menggunakan teknologi berbasis *open source* seperti Wazuh, Suricata, Zabbix, Grafana, TheHive, Cortex, Zammad, serta otomasi dengan n8n.

2. Lingkungan Simulasi

   Implementasi sistem dilakukan dalam lingkungan simulasi yang dirancang untuk merepresentasikan kondisi keamanan jaringan secara realistis. Data yang digunakan bersifat fiktif, dan skenario serangan ditentukan sebelumnya guna memastikan pengujian berlangsung secara terkontrol dan terukur.

3. Cakupan Evaluasi

   Evaluasi sistem dibatasi pada kemampuan SOC dalam mendeteksi dan memitigasi ancaman siber umum seperti *brute force attack*, *DDoS*, *malware detection*, dan ancaman lain yang memungkinkan untuk disimulasikan sesuai dengan kapasitas penelitian.

4. Aspek Penelitian

   Fokus utama penelitian berada pada aspek teknis, mencakup proses instalasi, konfigurasi, integrasi, dan optimalisasi dari seluruh teknologi yang digunakan. Penelitian ini tidak mencakup aspek manajerial, kebijakan keamanan informasi, maupun tata kelola organisasi.

5. Waktu dan Sumber Daya

   Penelitian dilakukan dalam rentang waktu yang terbatas, sehingga pengujian teknologi dan skenario ancaman difokuskan pada kasus prioritas yang relevan dan sering terjadi di dunia nyata. Penggunaan sumber daya juga disesuaikan dengan kapasitas yang tersedia dalam lingkungan simulasi.

6. Tidak Mencakup proses Fine-Tuning Rules

   Penelitian ini tidak mencakup proses *fine-tuning* secara mendalam terhadap aturan deteksi (*detection rules*) maupun pembuatan *custom ruleset* secara luas. Penelitian hanya menggunakan aturan *default* yang disediakan oleh masing-masing *tools*, serta beberapa *custom rules* yang dibuat secara terbatas sesuai kebutuhan skenario simulasi.

Batasan-batasan ini ditetapkan untuk menjaga fokus penelitian tetap berada pada aspek teknis implementasi SOC berbasis *open source*, serta memastikan bahwa proses perancangan, integrasi, dan pengujian dapat dilakukan secara optimal dalam ruang lingkup yang terukur dan sesuai dengan sumber daya serta waktu yang tersedia.

1.6 Metodologi

Metodologi yang akan digunakan pada Tugas Akhir mencakup tahapan sebagai berikut:

1. Studi Literatur

   Tahap ini dilakukan untuk mengumpulkan informasi dan teori yang relevan mengenai konsep Security Operations Center (SOC), keamanan siber, serta teknologi open source seperti Wazuh, Suricata, Zabbix, Grafana, TheHive, Cortex, Zammad, dan n8n. Referensi diperoleh dari buku, jurnal ilmiah, dokumentasi resmi, serta publikasi daring yang membahas topik serupa. Studi literatur ini bertujuan untuk membangun dasar teoritis yang kuat sebelum proses perancangan dan implementasi dilakukan.
2. Analisis Kebutuhan dan Perancangan Sistem

   Tahap ini melibatkan identifikasi kebutuhan teknis SOC serta spesifikasi tools *open source* yang akan digunakan. Selain itu, dirancang pula arsitektur sistem, topologi jaringan simulasi, skenario serangan, serta alur integrasi antar komponen SOC agar sesuai dengan tujuan penelitian. Perancangan mencakup diagram topologi, alur log data, serta rencana pengujian sistem.
3. Implementasi Sistem

   Pada tahap ini dilakukan instalasi dan konfigurasi tools *open source* dalam lingkungan simulasi yang dirancang. Setiap komponen SOC seperti Wazuh, Suricata, Zabbix, Grafana, TheHive, Cortex, dan Zammad akan diintegrasikan agar dapat saling terhubung dan bekerja secara sinergis dalam mendeteksi serta merespon ancaman. Otomatisasi alur kerja menggunakan n8n juga diterapkan untuk mempercepat respon insiden.
4. Simulasi dan Uji Fungsi

Dilakukan simulasi serangan siber umum seperti *brute force*, *malware injection*, dan *DDoS* terhadap lingkungan sistem yang telah dibangun. SOC akan diuji kemampuannya dalam mendeteksi, mengumpulkan log, memberikan notifikasi, serta menghasilkan tiket insiden. Pengujian ini bertujuan untuk menilai efektivitas dan keterpaduan sistem SOC yang diimplementasikan.

5. Evaluasi dan Dokumentasi Hasil

   Tahap akhir ini berupa analisis terhadap kinerja SOC berdasarkan hasil simulasi dan pengamatan. Evaluasi mencakup kemampuan deteksi, kecepatan respons, alur otomatisasi, serta kemudahan pengoperasian sistem secara keseluruhan. Hasil evaluasi didokumentasikan sebagai bagian dari laporan Tugas Akhir, sekaligus menjadi dasar untuk penyusunan kesimpulan dan saran.

BAB II
LANDASAN TEORI

2.1 Teori Tentang Sistem

2.1.1 Keamanan Siber (Cybersecurity)

Keamanan siber adalah praktik melindungi manusia, sistem, dan data dari serangan siber melalui penerapan teknologi, proses, dan kebijakan yang tepat, sebagai bagian dari strategi manajemen risiko organisasi. Dalam penerapannya, keamanan siber bertujuan untuk mencapai dan menjaga tujuan inti keamanan informasi *Confidentiality, Integrity, Availability* (*CIA triad*) serta memastikan pencegahan, deteksi, penahanan, dan pemulihan terhadap gangguan yang berpotensi mengganggu operasi dan layanan digital[4].

2.1.2 Security Operations Center (SOC)

Security Operations Center (SOC) adalah fungsi terpusat yang menyelenggarakan pemantauan berkesinambungan, korelasi, dan analisis peristiwa keamanan untuk mendeteksi serta menanggapi insiden secara terkoordinasi, guna menjaga tujuan keamanan informasi yaitu kerahasiaan, integritas, dan ketersediaan (*CIA Triad*). Operasional SOC berjalan 24/7 dengan memadukan unsur *people*, *process*, dan *technology* sebagai satu kesatuan yang saling melengkapi, dan dijalankan secara terpusat dengan visualisasi situasi terkini untuk mendukung pengambilan keputusan yang cepat. Indikator kinerja utama yang digunakan meliputi MTTD (*Mean Time to Detect*), MTTR (*Mean Time to Respond*), cakupan dan keluasan deteksi, tingkat *false positive*, jumlah insiden yang diselesaikan, jumlah eskalasi, serta beban kerja per insiden[5].

2.1.3 Security Information and Event Management (SIEM)

SIEM adalah sebuah sistem yang berfungsi untuk mengumpulkan, memusatkan, dan menganalisis data log dari berbagai sumber dalam suatu

jaringan atau sistem untuk tujuan keamanan siber. SIEM secara otomatis mengumpulkan dan memusatkan log penting yang sebelumnya tersebar di jaringan, sehingga memudahkan tim keamanan untuk melakukan navigasi dan investigasi. Platform ini menerapkan baseline aktivitas normal, aturan, dan filter untuk menganalisis serta mengorelasikan data log, sehingga dapat mendeteksi aktivitas tidak biasa yang berpotensi menjadi insiden keamanan. Analisis ini umumnya dilengkapi dengan intelijen ancaman terkini[6].

2.1.4 Intrusion Detection and Prevention System (IDS/IPS)

Intrusion Detection System (IDS) adalah komponen keamanan yang memantau trafik jaringan atau aktivitas host untuk mengidentifikasi indikasi serangan atau pelanggaran kebijakan dan menghasilkan peringatan untuk ditindaklanjuti misalnya oleh SIEM/orkestrasi respons. IDS dapat berupa NIDS atau HIDS dengan teknik deteksi berbasis rules, tanda tangan, anomali, dan validasi protokol. Berbeda dari IPS yang bekerja **inline** untuk memblokir trafik, fokus IDS adalah visibilitas dan pelaporan sehingga keputusan mitigasi dilakukan oleh kontrol lain seperti firewall sesuai arsitektur SOC yang digunakan[7].

2.1.5 Security Orchestration, Automation, and Response (SOAR)

SOAR adalah platform perangkat lunak yang mengotomatiskan respons terhadap aktivitas tidak biasa yang terdeteksi di jaringan. Respons ini dijalankan melalui *playbook dan workflow* yang telah ditentukan, yang memadukan rencana respons insiden dan rencana kesinambungan bisnis untuk menentukan tindakan yang diambil saat terjadi insiden keamanan tertentu. SOAR dapat terintegrasi dengan SIEM atau alat keamanan lainnya, seperti firewall, solusi keamanan endpoint, dan pemindai kerentanan[6].

2.1.6 Komponen Sistem SOC Berbasis Open Source

1) Wazuh (SIEM)

   Wazuh adalah platform open source untuk keamanan informasi dan manajemen peristiwa (SIEM) yang menyediakan deteksi ancaman,

monitoring integritas file, respons insiden, dan kepatuhan. Dalam arsitektur SOC, Wazuh berperan sebagai pusat korelasi log dan sumber utama security alert[8].

2) Suricata (IDS/IPS)

Suricata merupakan sistem deteksi dan pencegahan intrusi jaringan (IDS/IPS) open source yang mampu melakukan inspeksi paket mendalam (DPI) serta mendukung protokol modern. Dalam arsitektur SOC, Suricata berperan mendeteksi aktivitas berbahaya di tingkat jaringan[9].

3) Zabbix (Utilization Monitoring)

Zabbix adalah platform open source untuk monitoring infrastruktur TI yang mencakup host, layanan, dan aplikasi. Zabbix dalam SOC digunakan untuk menghasilkan utilization alert terkait performa sistem dan kapasitas infrastruktur[10].

4) Grafana (Dashboard)

Grafana adalah platform analitik open source yang digunakan untuk visualisasi data dalam bentuk dashboard interaktif. Dalam SOC, Grafana berfungsi menampilkan metrik Zabbix agar operator memiliki visibilitas real-time terhadap kondisi sistem[11].

5) TheHive (Case Management).

TheHive adalah platform open source untuk case management insiden keamanan, yang memungkinkan kolaborasi tim dalam menganalisis dan menindaklanjuti alert. Integrasi Wazuh dengan TheHive melalui n8n memungkinkan otomatisasi pembuatan kasus berdasarkan deteksi ancaman[12].

6) Cortex (Threat Intelligence & Enrichment)

Cortex merupakan platform analisis artefak dan enrichment berbasis open source yang dapat diintegrasikan dengan TheHive. Perannya adalah memperkaya informasi insiden menggunakan threat intelligence, sehingga tim SOC dapat melakukan analisis lebih komprehensif[13].

7) n8n (Automation / Mini-SOAR)

   n8n adalah alat orkestrasi otomatisasi open source yang memungkinkan integrasi berbagai aplikasi melalui API dan workflow. Dalam SOC, n8n berperan sebagai mini-SOAR untuk menghubungkan Wazuh, TheHive, Cortex, Telegram, dan Zammad secara otomatis[14].

8) Telegram (Notifikasi Alert)

   Telegram menyediakan Bot API yang memungkinkan integrasi dengan sistem eksternal. Dalam SOC, Telegram digunakan untuk menyampaikan notifikasi alert secara real-time kepada tim keamanan[15].

9) Zammad (Ticketing System)

   Zammad adalah sistem ticketing open source yang mendukung manajemen layanan TI dan keamanan. Dalam SOC, Zammad mencatat semua event keamanan sebagai tiket untuk pelacakan insiden[16].

2.1.7 Incident Handling & Ticketing System

Incident Handling & Ticketing System adalah proses dan alat untuk mencatat, memprioritaskan, menugaskan, dan melacak respons insiden melalui tiket sebagai unit kerja yang memuat status, SLA, artefak, dan jejak audit; di SOC, sistem ini mengalirkan tahapan deteksi–analisis hingga containment, eradication, recovery, dan pembelajaran, sehingga koordinasi lintas tim terdokumentasi dan terukur. Integrasi dengan SIEM/IDS/SOAR mengubah alert menjadi tiket/kasus yang dapat ditindaklanjuti serta mengotomasi langkah respons, memberi visibilitas end-to-end dan akuntabilitas terhadap standar organisasi[17].

2.1.8 Cyber Threat Intelligence

Cyber Threat Intelligence (CTI) adalah sekumpulan informasi tepercaya tentang ancaman yang relevan bagi organisasi yang mencakup indikator teknis dan konteks aktor, motif, taktik, serta kerentanan yang dikumpulkan

dan dianalisis untuk membantu pengambilan keputusan pertahanan yang cepat dan tepat. pada SOC, CTI digunakan untuk memperkaya alert dari SIEM/IDS dan menetapkan prioritas penanganan. CTI umumnya disajikan pada tiga tingkat yaitu taktis (misalnya daftar IP/domain berbahaya, hash malware), operasional (misalnya gambaran teknik attacker yang memandu aturan deteksi dan playbook respons), dan strategis (misalnya tren serangan serta dampaknya pada bisnis yang mengarahkan kebijakan dan alokasi sumber daya), sehingga informasi mudah ditindaklanjuti sesuai kebutuhan tim[18].

2.1.9 Amazon Elastic Compute Cloud (EC2)

Amazon Elastic Compute Cloud (EC2) adalah layanan komputasi awan dari AWS yang menyediakan server virtual elastis dengan fleksibilitas tinggi[19]. Pemanfaatan EC2 untuk SOC memungkinkan penerapan solusi open source dengan skalabilitas sesuai kebutuhan organisasi, tanpa investasi besar pada infrastruktur fisik.

BAB III
ANALISIS DAN PERANCANGAN

3.1 Analisis Sistem yang Sedang Berjalan

Saat ini, sebagian besar organisasi kecil hingga menengah masih belum memiliki sistem pemantauan keamanan yang terpusat dan terintegrasi. Pengelolaan keamanan informasi biasanya bersifat reaktif, tidak terdokumentasi dengan baik, serta terbatas pada penggunaan perangkat lunak antivirus atau firewall standar. Tidak adanya sistem pemantauan secara real-time menyebabkan banyak insiden keamanan yang tidak terdeteksi dan teraudit sejak dini, sehingga berpotensi menimbulkan kerugian besar baik dari segi operasional maupun reputasi.

Selain itu, kendala utama yang dihadapi oleh organisasi-organisasi ini adalah keterbatasan anggaran untuk membangun Security Operations Center (SOC) yang lengkap dan mahal secara komersial. Banyak solusi SOC di pasaran mengandalkan lisensi berbayar, perangkat keras khusus, dan tenaga ahli tersertifikasi yang tidak selalu tersedia di organisasi skala menengah ke bawah. Hal ini menyebabkan munculnya kesenjangan antara kebutuhan keamanan dan kemampuan implementasi di lapangan.

Dalam kondisi seperti ini, insiden seperti serangan siber, monitoring utilisasi hingga kebocoran data sensitif menjadi sulit untuk dideteksi dan ditanggulangi secara tepat waktu. tidak adanya sistem pelaporan insiden yang terstruktur, serta tidak adanya orkestrasi otomatis untuk penanganan ancaman semakin memperburuk situasi ini. Akibatnya organisasi hanya menyadari keberadaan serangan atau masalah utilitas ketika dampaknya sudah terjadi.

Oleh karena itu, dibutuhkan solusi yang mampu menjawab tantangan-tantangan tersebut, yaitu sistem SOC yang dapat berjalan di lingkungan terbatas namun tetap mampu memberikan deteksi dan respon ancaman secara efektif. Solusi berbasis open source dinilai menjadi alternatif yang paling relevan untuk menjembatani kebutuhan ini, tanpa harus mengorbankan efisiensi dan efektivitas dalam aspek teknis.

3.2 Analisa Kebutuhan Sistem

3.2.1 Kebutuhan Perangkat Keras

Implementasi sistem SOC ini dilakukan menggunakan beberapa instance AWS EC2 dengan pembagian per layanan. Setiap komponen memiliki kebutuhan minimum perangkat keras agar dapat berjalan secara optimal. Berikut adalah Spesifikasi yang digunakan

Tabel 3. 1 kebutuhan perangkat keras

No	Nama Instance	Spesifikasi	Sistem Operasi
1	HQ-Wazuh-Suricata	2 vCPU, 8 GB RAM, 80 GB SSD	Ubuntu Server 22.04 LTS
2	HQ-Zabbix-Grafana	2 vCPU, 4 GB RAM, 60 GB SSD	Ubuntu Server 22.04 LTS
3	HQ-Thehive-Cortex	2 vCPU, 8 GB RAM, 80 GB SSD	Ubuntu Server 22.04 LTS
4	HQ-ZAmmad-n8n	2 vCPU, 8 GB RAM, 60 GB SSD	Ubuntu Server 22.04 LTS

3.2.2 Kebutuhan Perangkat Lunak

Agar seluruh komponen dalam sistem Security Operations Center (SOC) dapat berjalan dengan optimal, dibutuhkan sejumlah perangkat lunak utama beserta dependensinya. Perangkat lunak yang digunakan dalam penelitian ini meliputi sistem operasi, tools open source utama, serta beberapa software pendukung.

a) Sistem Operasi

Tabel 3. 2 kebutuhan perangkat lunak

Komponen	Verzi	Deskripsi

Ubuntu Server	22.04 LTS	Sistem operasi utama seluruh instance

b) Tools Utama

Tabel 3. 3 tools utama

Komponen	Versi	Deskripsi
Wazuh	4.12	SIEM/log management & alerting
Suricata	6.x	IDS/IPS berbasis signature
Zabbix	7.x	Monitoring performa server & layanan
Grafana	9.x	Visualisasi data dari Zabbix
TheHive	5.x	Platform manajemen insiden
Cortex	3.x	Otomasi analisis observables dari insiden
Zammad	6.x	Sistem ticketing & layanan pengguna
n8n	1.x	Workflow otomasi antar tools SOC

c) Depedensi Tambahan

Tabel 3. 4 depedensi tambahan

Komponen	Versi	Deskripsi

Elasticsearch	7.x	Backend penyimpanan dan pencarian log (Wazuh)
Cassandra	4.x	Database backend untuk TheHive dan Cortex
Node.js	18.x	Runtime untuk Zammad dan n8n
Java (OpenJDK)	11	Runtime untuk TheHive dan Cortex
Docker (opsional)	24.x	Manajemen container (jika digunakan)
Docker Compose	2.x	Orkestrasi layanan multi-container (YAML-based)

3.2.3 Kebutuhan Jariangan

Agar seluruh komponen Security Operations Center (SOC) dapat Diintegrasikan dan dapat diakses secara stabil, diperlukan konfigurasi jaringan yang tepat. Pada penelitian ini menggunakan layanan AWS EC2 dengan default VPC sebagai lingkungan virtualisasi, sehingga kebutuhan jaringan difokuskan pada pengaturan Security Group (SG) dan Elastic IP (EIP).

Security Group berperan sebagai firewall virtual yang mengatur lalu lintas masuk dan keluar instance berdasarkan port tertentu, guna menjamin komunikasi antar layanan serta akses pengguna melalui protokol yang aman. Sementara itu, Elastic IP digunakan agar instance memiliki alamat IP publik yang statis, sehingga tidak berubah ketika instance dihentikan atau direstart. Hal ini sangat penting untuk menjaga kestabilan koneksi antar

tools serta integrasi yang membutuhkan alamat IP tetap (misalnya pengaturan log forwarding, web interface, dan webhook).

Seluruh instance menggunakan default VPC AWS dengan alokasi 1 Elastic IP per instance agar dapat diakses publik (jika perlu) dan tetap memiliki alamat tetap dan menggunakan custom Security group pada masing-masing Instance dengan pengaturan port yang pada inbound dan outbound sebagi berikut

a) Elastic IP

Tabel 3. 5 elastic ip

Instance	Security Group	Private IP	Elastic IP
HQ-Wazuh-Suricata	SG-Wazuh-Suricata	172.31. xx.xx	54.84.xx.xx
HQ-ZAbbix-Grafana	SG -ZAbbix-Grafana	172.31. xx.xx	98.84. xx.xx
HQ-Thehive-Cortex	SG -Thehive-Cortex	172.31. xx.xx	3.216. xx.xx
HQ-ZAmmad-n8n	SG-ZAmmad-n8n	172.31. xx.xx	3.221. xx.xx

b) Inbound Rule Security Group

Tabel 3. 6 Inbound security group

SG	Service	Port	Deskripsi
SG-Wazuh-Suricata	SSH	22/TCP	Akses ke server melalui protokol SSH
	HTTPS	443/TCP	Akses ke Wazuh Dashboard secara aman

	Wazuh API	55000/TCP	Komunikasi antara agent dan server Wazuh
	Wazuh Agent Auth	1515/TCP	Autentikasi awal antara Wazuh agent dan manager
	Wazuh Cluster	1516/TCP	Sinkronisasi antar node Wazuh cluster
	Elasticsearch Cluster	9300-9400/TCP	Komunikasi antar node Elasticsearch cluster
	Wazuh TCP Syslog	1514/TCP	Pengiriman log dalam format TCP ke Wazuh
	Suricata TCP Syslog	514/TCP	Forwarding log Suricata melalui TCP
	Wazuh Elasticsearch API	9200/TCP	Akses API Elasticsearch yang digunakan Wazuh
	Wazuh UDP Syslog	1514/UDP	Pengiriman log dalam format UDP ke Wazuh
	Suricata UDP Syslog	514/UDP	Forwarding log Suricata melalui UDP
SG-Zabbix-Grafana	SSH	22	Akses administrator via SSH.
	HTTP	80	Akses web tidak terenkripsi (Zabbix).
	HTTPS	443	Akses web aman dengan SSL/TLS.

	Zabbix Server	10051	Komunikasi Zabbix Server dengan agent dan API.
	Grafana UI	3000	Antarmuka web Grafana untuk visualisasi dashboard.
SG-Thehive-Cortex	SSH	22/TCP	Akses administrator ke server menggunakan protokol SSH.
	HTTP	80/TCP	Akses web tanpa enkripsi (opsional).
	HTTPS	443/TCP	Akses web aman melalui protokol SSL/TLS.
	TheHive UI	9000/TCP	Antarmuka web utama dari TheHive (Manajemen insiden siber).
	Cortex UI	9001/TCP	Antarmuka web dan API Cortex untuk analisis otomatis.
	Elasticsearch API	9200/TCP	REST API Elasticsearch, digunakan untuk query data log oleh TheHive & Cortex.
	Cassandra CQL	9042/TCP	Port komunikasi aplikasi dengan database Cassandra menggunakan Cassandra Query Lang.

	Cassandra Cluster	7000/TCP	Port komunikasi antar-node Cassandra (internode communication).
	Cortex Responder	9024/TCP	Port khusus responder otomatis Cortex (tergantung konfigurasi).
SG-Zammad-n8n	SSH	22/TCP	Akses administrator ke server melalui protokol SSH.
	HTTP	80/TCP	Akses web tanpa enkripsi (untuk Zammad).
	HTTPS	443/TCP	Akses web aman menggunakan protokol SSL/TLS (untuk Zammad).
	n8n UI/API	5678/TCP	Antarmuka web dan API untuk workflow automation n8n.
	Elasticsearch API	9200/TCP	Digunakan oleh Zammad untuk pencarian dan penyimpanan data log (jika terkoneksi).

c) Outbound Rule Secuirty Group

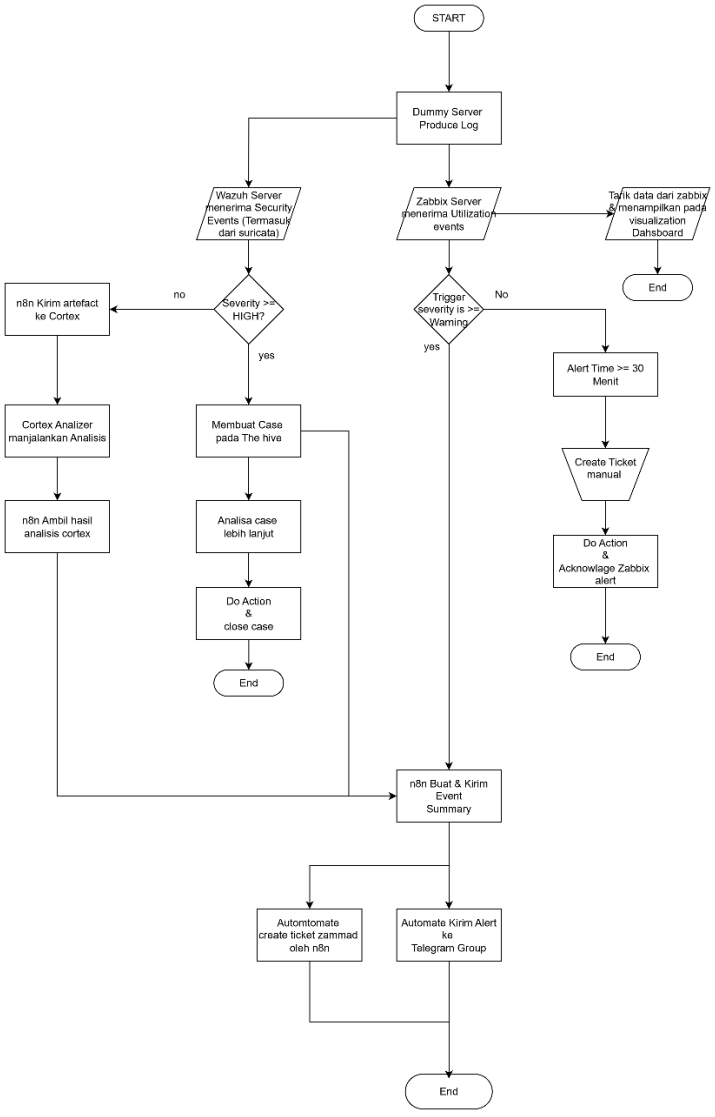
Tabel 3. 7 Outbound security Group

Instance	Port
SG-Wazuh-Suricata	ALL Traffic

SG-ZAbbix-Grafana	ALL Traffic
SG-Thehive-Cortex	ALL Traffic
SG-ZAmmad-n8n	ALL Traffic

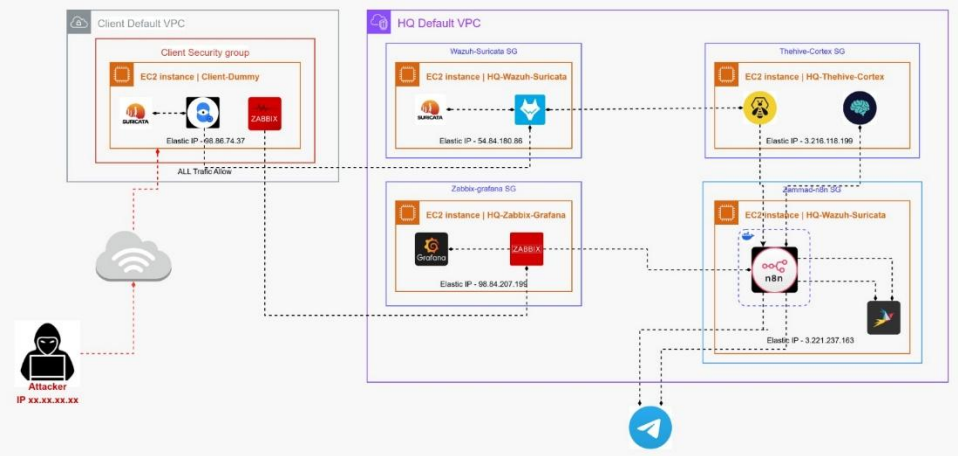
3.3 Rancangan Sistem

3.3.1 Rancangan Flowchart



Gambar 3. 1 Flowchart sistem

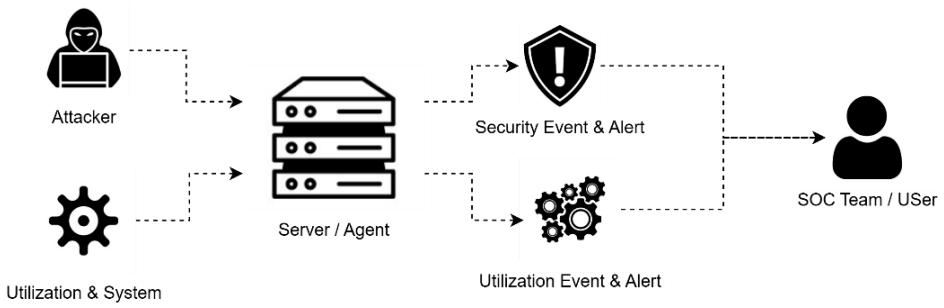
3.3.2 Rancangan Topologi



Gambar 3. 2 Rancangan topologi

3.3.3 Rancangan Arsitektur Sistem

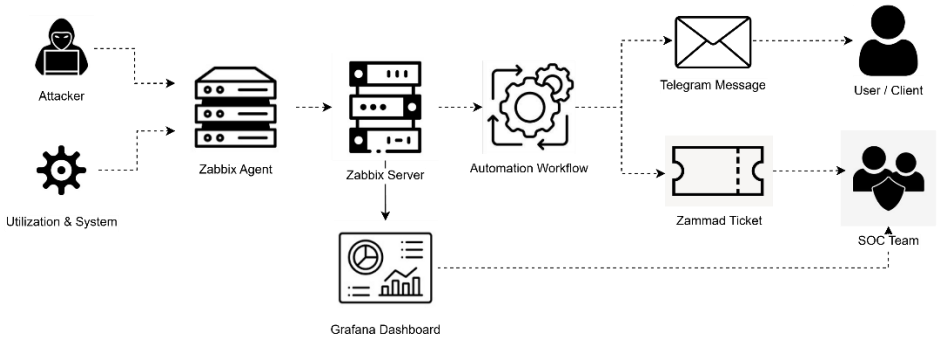
Arsitektur sistem yang dibangun pada Tugas Akhir ini dirancang untuk mendukung proses monitoring yang lebih komprehensif terhadap Agent atau server. Sistem ini secara sederhana akan melakukan klasifikasi event yang diperoleh dari agent atau server yang dimonitor menjadi dua kategori utama, yaitu **utilization alert** dan **security alert**. Dengan adanya pemisahan ini, administrator sistem dapat memperoleh gambaran yang lebih terstruktur mengenai kondisi infrastruktur, sehingga proses analisis, deteksi, maupun tindak lanjut dapat dilakukan dengan lebih tepat dan efisien.



Gambar 3. 3 rancangan arsitektur

a) Utilization Alert

Utilization alert akan difokuskan pada pemantauan sumber daya perangkat, seperti kondisi CPU, RAM, network traffic, serta layanan yang berjalan, misalnya web server dan service pendukung lainnya. Pemantauan ini dilakukan menggunakan platform Zabbix yang mampu memberikan informasi performa secara real-time sekaligus mendeteksi terjadinya anomali pada penggunaan sumber daya.



Gambar 3. 4 arsitektur utilization alert

Alur arsitektur utilization alert ini secara berurutan adalah sebagai berikut:

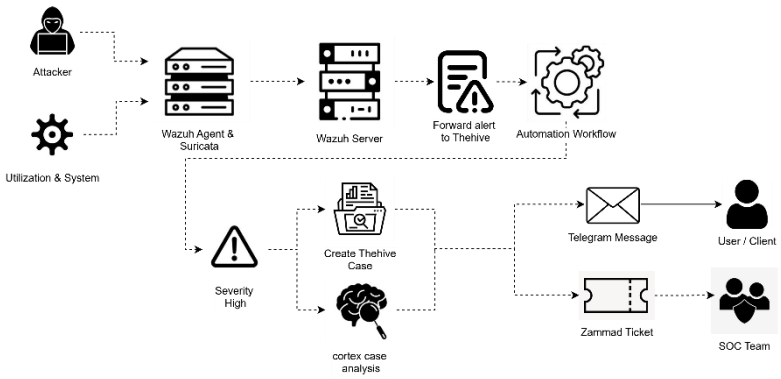
1) Agent atau server mengirimkan utilization event maupun system event, termasuk juga potensi serangan yang berfokus pada resource seperti DDoS atau aktivitas abnormal lainnya.
2) Event diterima oleh Zabbix server untuk dilakukan pemantauan dan analisis kondisi sumber daya.
3) Zabbix terhubung ke Grafana sehingga data monitoring divisualisasikan dalam bentuk dashboard yang lebih interaktif, user-friendly, dan modern untuk memudahkan tim dalam melakukan observasi.
4) Alert dikirimkan ke webhook automation workflow (n8n) agar dapat diproses secara otomatis sesuai skenario yang sudah ditentukan.

5) n8n memformat alert menjadi pesan informasi yang lebih jelas, ringkas, dan mudah dipahami.
6) Alert yang sudah diformat dikirim ke group Telegram sehingga tim bisa segera mengetahui kondisi yang terjadi secara real-time.
7) Secara paralel dibuatkan tiket di Zammad, sehingga setiap alert dapat ditindaklanjuti secara terstruktur sesuai dengan kategori maupun tingkat prioritas permasalahan.

Dengan alur ini, utilization alert tidak hanya berhenti pada deteksi anomali, tetapi juga disajikan dengan visualisasi yang lebih jelas dan disertai sistem notifikasi serta ticketing yang memudahkan tim dalam melakukan tindak lanjut secara cepat dan terorganisir.

b) Security Alert

Sementara itu, security alert difokuskan pada pemantauan sisi keamanan sistem. Monitoring ini dilakukan menggunakan Wazuh yang telah diintegrasikan dengan Suricata sehingga mampu mendeteksi berbagai aktivitas mencurigakan atau anomali yang berhubungan dengan keamanan jaringan maupun host. Beberapa contoh ancaman yang dapat dipantau meliputi percobaan koneksi tidak wajar, brute force, file integrity monitoring, dan aktivitas abnormal lain yang berpotensi mengganggu keamanan infrastruktur.



Gambar 3. 5 arsitektur security alert

Berikut adalah alur arsitektur security alert yang dapat dijelaskan sebagai berikut:

1) Terjadi serangan atau aktivitas mencurigakan pada agent/server yang sedang dimonitoring.
2) Event keamanan tersebut dideteksi oleh Wazuh server yang sudah terintegrasi dengan Suricata.
3) Event kemudian diteruskan ke TheHive untuk dikonversi menjadi sebuah alert.
4) Alert diproses oleh workflow automation (n8n).dimana jika alert memiliki severity high maka n8n akan mengubah alert tersebut menjadi sebuah case di TheHive, barulah kemudian pesan konfirmasi dikirim ke group telegram dan dibuatkan menajdi sebuah ticket zammad.
5) Dan jika alert memiliki severity dibawah HIGH, alert tersebut akan diteruskan ke Cortex untuk dilakukan analisis terhadap artefak atau observable yang terdapat pada case tersebut melalui automation workflow.
6) Alert diformat menjadi pesan informatif yang berisi ringkasan (summary) dan informasi penting dari alert beserta hasil analisis Cortex.
7) Pesan yang sudah diformat dikirimkan ke group Telegram agar tim dapat segera menerima notifikasi real-time.
8) Secara paralel dibuatkan tiket di Zammad, sehingga setiap alert memiliki jejak tindak lanjut yang jelas dan dapat diprioritaskan sesuai kebutuhan.

Dengan alur ini, security alert tidak hanya berhenti pada deteksi serangan, tetapi juga dilengkapi dengan proses investigasi, analisis otomatis, serta mekanisme notifikasi dan ticketing. Hal ini memastikan bahwa setiap ancaman dapat ditangani secara cepat, sistematis, dan terdokumentasi dengan baik.

Dengan adanya klasifikasi ini, infrastruktur yang dibangun diharapkan dapat memberikan pandangan yang lebih luas, baik terhadap kinerja maupun keamanan sistem yang dimonitor.

BAB IV
IMPLEMENTASI DAN PENGUJIAN

4.1 Persiapan Lingkungan Implementas

Proses instalasi dimulai dengan menyiapkan lingkungan implementasi, yang secara garis besar dibagi menjadi dua bagian utama. Pertama adalah **lingkungan utama**, yaitu tempat semua komponen SOC diinstal. Lingkungan ini ke depannya akan disebut sebagai **HQ** untuk mempermudah penyebutan. Kedua adalah **lingkungan client**, yaitu tempat di mana instance dummy diinstal sebagai client fiktif atau agent/server yang nantinya akan dimonitoring oleh HQ.

Pada lingkungan HQ, digunakan default VPC dari AWS agar fokus utama tetap pada proses instalasi dan integrasi antar tools, tanpa terlalu banyak konfigurasi jaringan yang kompleks. Setelah itu, dibuat empat buah custom Security Group yang disesuaikan dengan kebutuhan masing-masing instance, seperti yang telah dijelaskan pada bagian sebelumnya. Setelah Security Group disiapkan, proses dilanjutkan dengan melakukan launch instance EC2 mengikuti spesifikasi yang telah dijabarkan pada bagian 3.2.1. Setelah semua instance berjalan, masing-masing instance dialokasikan Elastic IP. Tujuannya agar setiap instance memiliki IP publik yang statis, sehingga proses konfigurasi dan integrasi antar layanan tidak terganggu oleh perubahan IP.

Sementara itu, pada lingkungan client dilakukan proses serupa, yaitu menggunakan default VPC, membuat Security Group, dan mengalokasikan Elastic IP. Namun, khusus untuk instance dummy agent, dilakukan pembukaan seluruh port (all traffic) dengan tujuan untuk mensimulasikan berbagai jenis serangan yang mungkin terjadi selama pengujian nanti.

4.2 Installasi dan konfigurasi komponen SOC

4.2.1 Wazuh Server

a) Installasi

Wazuh Manager sendiri memiliki beberapa metode instalasi seperti on-host, OVA, container, dan from source (tanpa menggunakan package manager) seperti yang dapat dilihat pada halaman resmi Wazuh Deployment Options. Untuk menjalankan wazuh perlu untuk menginstall 3 komponen utama yaitu:

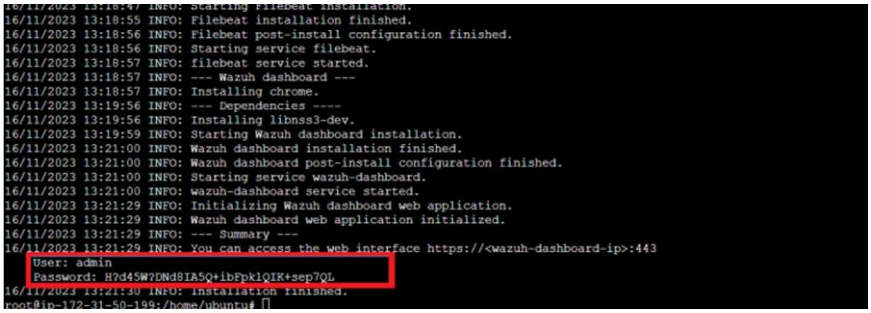
1) Wazuh Manager (Server)
   Berfungsi untuk mengumpulkan, menganalisis, dan mengelola data keamanan dari endpoint yang dipantau. Manager ini juga melakukan deteksi ancaman, korelasi log, dan penerapan rules untuk menentukan adanya insiden.
2) Wazuh Indexer
   merupakan fork dari OpenSearch dan digunakan untuk menyimpan dan mengindeks semua data log yang dikumpulkan oleh manager agar bisa ditelusuri dan dianalisis dengan cepat
3) Wazuh Dashboard
   merupakan antarmuka grafis berbasis web untuk memvisualisasikan data yang dikumpulkan. Melalui dashboard ini, pengguna bisa memantau status agent, melihat alert, dan membuat visualisasi log dalam bentuk grafik atau tabel.

Wazuh juga menyediakan **installation assistant command**, yaitu sebuah skrip otomatisasi instalasi yang memungkinkan pengguna untuk mengatur seluruh komponen Wazuh (manager, indexer, dan dashboard) hanya dengan menjalankan satu perintah. Dengan menggunakan metode ini, proses instalasi menjadi jauh lebih sederhana. Langkah-langkahnya dapat dilihat secara lengkap pada halaman Wazuh Quickstart Installation Guide. Untuk proses ini akan dilakukan pada isntance HQ-Wazuh-Suricata berikut adalah langkahnya.

1) Jalankan Instalation assistance Command

```
curl -sO https://packages.wazuh.com/4.12/wazuh-install.sh && sudo bash ./wazuh-install.sh -a
```

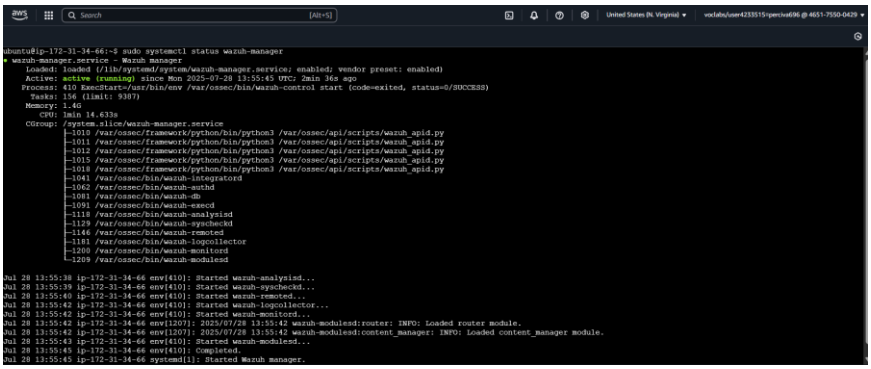
2) Seteah installasi selesai akan diberikan kredensial untuk login pada halaman wazuh dashboard dengan username admin dan password berupa String acak.



Gambar 4. 1 hasil download wazuh

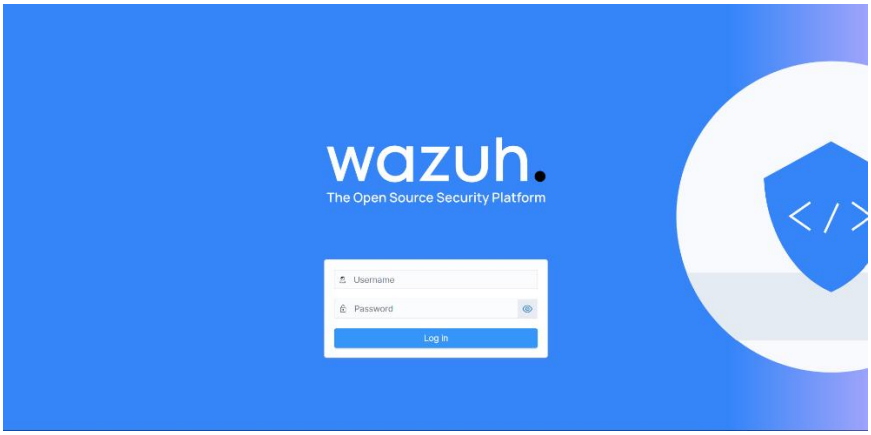
3) Untuk memastikan bahwa wazuh server telah berhasil berjalan gunakan perintah berikut:

```
sudo systemctl status wazuh-manager
```



Gambar 4. 2 cek status wazuh manager

4) Setelah proses selesai kita dapat login pada wazuh dashboard dengan akses melalui https://machineIP , Dimana pad infrastruktur ini adalah https:// 54.84.xx.xx



Gambar 4. 3 halaman login wazuh dashboard

b) Konfigurasi

Terdapat beberapa hal yang perlu dilakukan konfigurasi seperti, ……

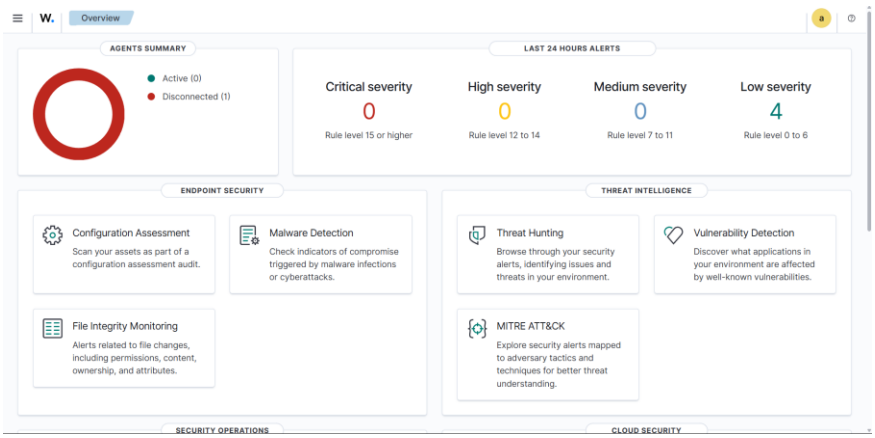
1) Untuk melakukan penggantian password default yang telah diberikan oleh wazuh dapat dilakukan langkah sebagai berikut, dimana juga dapat dilihat pada halaman Wazuh Password Management.
2) Jalankan perintah seperti berikut

```
curl -so wazuh-passwords-tool.sh https://packages.wazuh.com/4.12/wazuh-passwords-tool.sh
```

3) Ganti bagian Secr3tP4ssw*rd menjadi password yang diinginkan

```
bash wazuh-passwords-tool.sh -u admin -p Secr3tP4ssw*rd
```

4) Setelahnya dapat Kembali login ke halaman wazuh dashboard dengan password yang telah diubah



Gambar 4. 4 Halama utama wazuh dahsboard

4.2.2 Suricata

Suricata adalah IDS (Intrusion Detection System) berbasis open source yang mampu melakukan analisis lalu lintas jaringan secara real-time serta mendeteksi berbagai jenis serangan. Dalam implementasi ini, Suricata akan dipasang pada instance *HQ-Wazuh-Suricata* dan juga pada cinstanc *Client-Dummy*. Untuk langkahnya sama dan cukup menyesuaikan dengan alamt ip dan interface dari masing-masing instance. Untuk contoh kali ini akan dilakukan pada instance *HQ-Wazuh-Suricata* Berikut ini adalah langkah-langkah instalasinya:

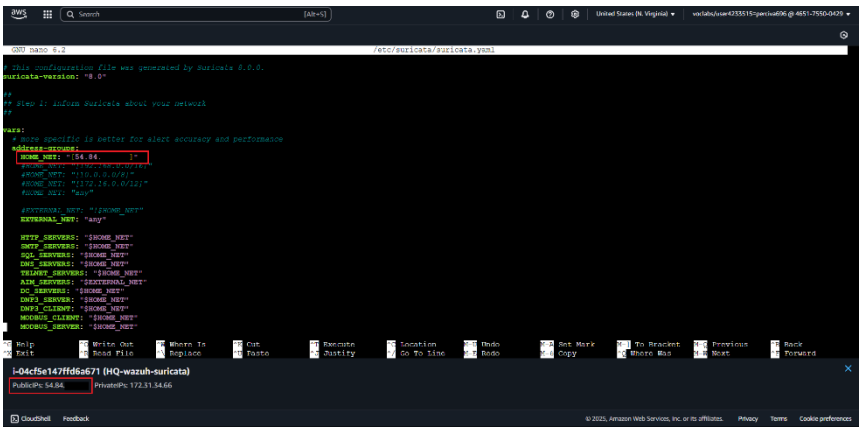
a) Installasi

1) Hal pertama yang perlu dilakukan adalah menambahkan repository Suricata versi stabil agar kita bisa mengakses paket versi terbaru dari pengembang resmi.
2) Setelah repository ditambahkan, lakukan update pada sistem agar paket terbaru bisa dikenali oleh sistem operasi.
3) Selanjutnya, install Suricata dengan perintah sudo **`apt-get install suricata -y`**
4) Setelah proses instalasi selesai, unduh dan ekstrak ruleset dari Emerging Threats agar Suricata memiliki basis aturan untuk mendeteksi ancaman jaringan. Aturan yang di unduh nantinya akan ditempatkan pada direktori /etc/suricata/rules.

```
cd /tmp/
curl                                                    -LO
https://rules.emergingthreats.net/open/suricata-
6.0.8/emerging.rules.tar.gz
sudo tar -xvzf emerging.rules.tar.gz
sudo mv rules/*.rules /etc/suricata/rules/
sudo chmod 640 /etc/suricata/rules/*.rules
```

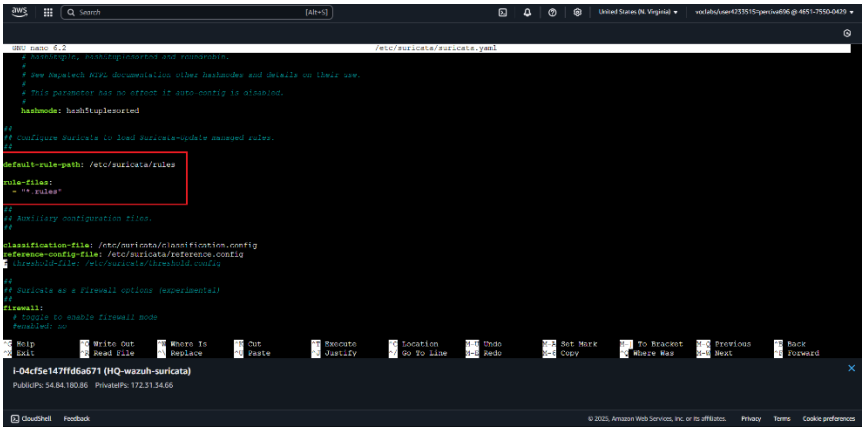
b) Konfigurasi

1) Setelah ruleset berhasil diinstall, buka file konfigurasi utama Suricata yang berada di /etc/suricata/suricata.yaml untuk menyesuaikan pengaturan jaringan dan path rules.
2) Pada bagian HOME_NET, isikan IP dari instance Ubuntu tempat Suricata dijalankan. Sementara EXTERNAL_NET bisa diatur ke nilai "any".

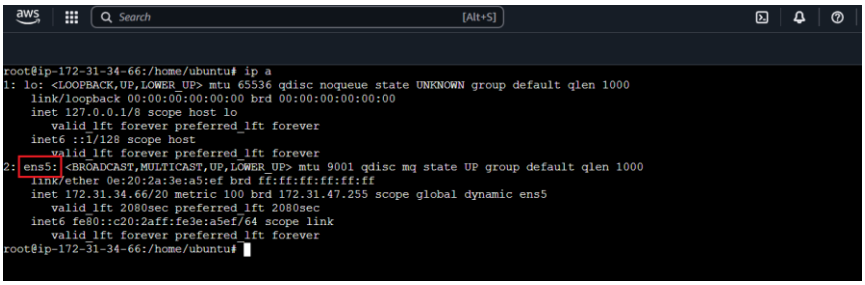


Gambar 4. 5 configurasi homenet & external net

3) Pastikan juga path default ke file rules sudah benar, dan aktifkan semua file dengan ekstensi .rules. Dimana nantinya suricata akan membaca setiap file dengan ekstensi .rules pada direktory tersebut dikarenakan sudah diset menjadi “*.rules”

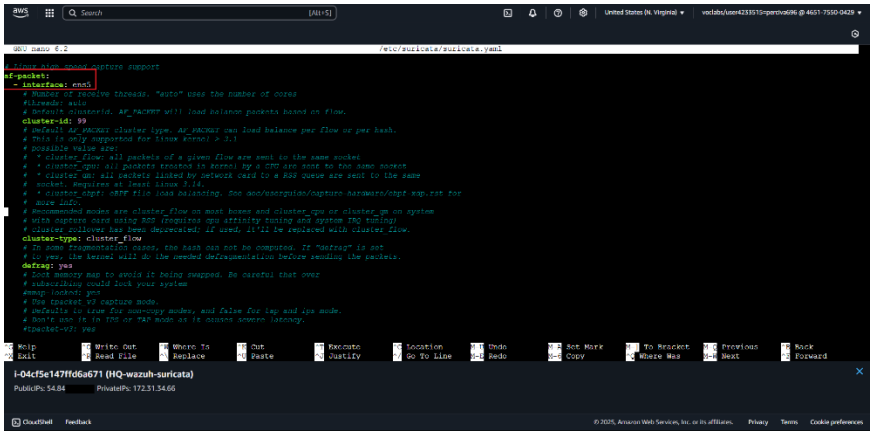


4) Sesuaikan Interface yang aktif pada machine dengan melakukan pengecekan menggunakan perintah ip a.



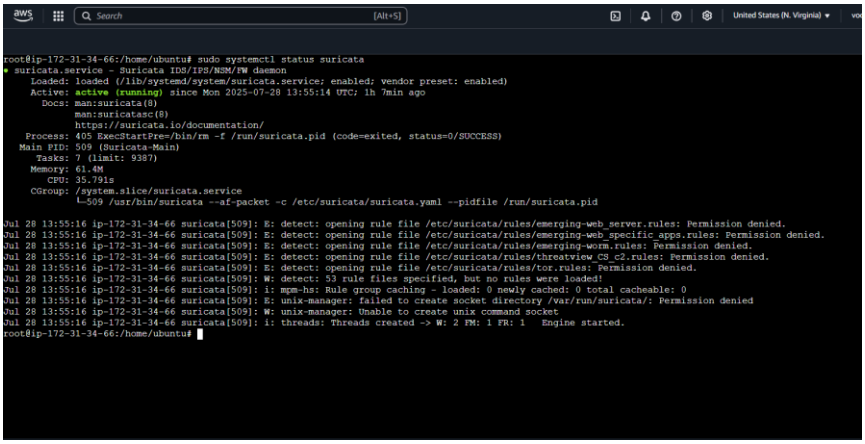
Gambar 4. 6 cek interface

5) Aktifkan statistik global dan dukungan untuk interface jaringan yang sesuai, pada machine ini adalah ens5 interface menggunakan af-packet:



Gambar 4. 7 set static global

6) Setelah konfigurasi selesai dan file telah disimpan, restart Suricata agar perubahan dapat diterapkan. Dan cek status apakah sudah berjalan dengan benar.



Gambar 4. 8 cek status suricata

4.2.3 Zabbix Server

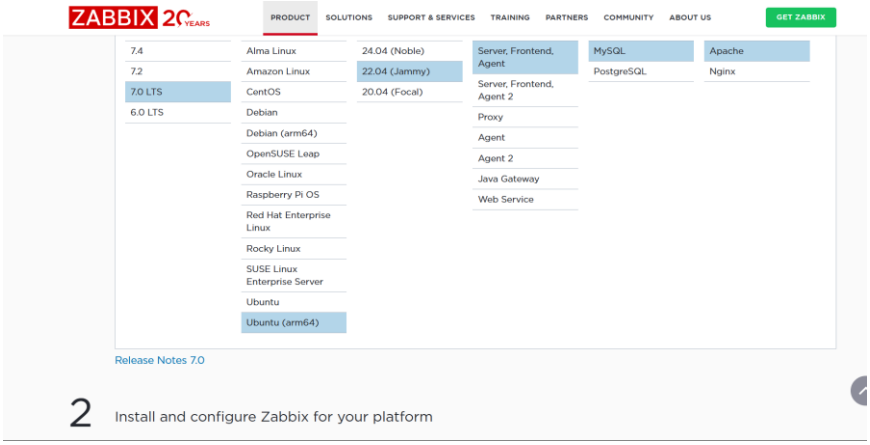
Zabbix adalah salah satu sistem monitoring open source yang digunakan untuk memantau kinerja jaringan, server, dan layanan secara real-time. Pada implementasi ini, Zabbix digunakan untuk memantau berbagai komponen dari infrastruktur Security Operations Center (SOC). Proses instalasi akan dilakukan pada instance *HQ-Zabbix-Grafana* yang berjalan di atas Ubuntu Server 22.04 LTS.

Zabbix menyediakan paket instalasi untuk berbagai distribusi Linux. Dalam hal ini, instalasi dilakukan menggunakan paket resmi dari repository Zabbix versi 7.0 untuk Ubuntu.

a) Installasi

1) Pertama pilih platform dimana zabbix akan diintall, untuk bagian ini sudah disediakan UI untuk memermudah pada halaman Zabbix.com/download. Sesuaikan dengan kebutuhan, disini kita akan menggunakan versi 7.0 pada ubuntu 22.04 dengan mendownload 3 komponen yaitu server, frontend dan agent, untuk

database akan menggunakan Mysql dengan apache sebagai webserver.



Gambar 4. 9 zabbix installation assistant

2) Untuk memudahkan proses installsi dan menghindari kendala permission selama proses installasi disarankan untuk menggunakan user root dengan menggunakan perintah **`sudo su.`**

3) Unduh dan install paket repository Zabbix ke dalam sistem dengan menggunakan perintah sebagai berikut:

```
wget
https://repo.zabbix.com/zabbix/7.2/release/ubuntu/pool
/main/z/zabbix-release/zabbix-
release_latest_7.2+ubuntu22.04_all.deb dpkg -i zabbix-
release_latest_7.2+ubuntu22.04_all.deb apt update
```

4) Setelah repository berhasil ditambahkan, install komponen utama dari Zabbix dengan menjalankan perintah

```
apt install zabbix-server-mysql zabbix-frontend-php
zabbix-apache-conf zabbix-sql-scripts zabbix-agent
```

5) Pastikan MySQL atau MariaDB sudah terinstall dan berjalan. Jalankan perintah berikut pada host database:

```
mysql -uroot -p
create database zabbix character set utf8mb4 collate
utf8mb4_bin;
```

```
create user zabbix@localhost identified by 'password';
grant all privileges on zabbix.* to zabbix@localhost;
set global log_bin_trust_function_creators = 1; quit;
```

6) Setelah database dibuat, impor skema dan data awal ke dalam database dengan menjalankan perintah berikut:

```
zcat /usr/share/zabbix/sql-scripts/mysql/server.sql.gz
| mysql --default-character-set=utf8mb4 -uzabbix -p
zabbix
```

7) Jika sudah selesai, nonaktifkan kembali opsi log_bin_trust_function_creators dengan menjalankan perintah seperti dibawah:

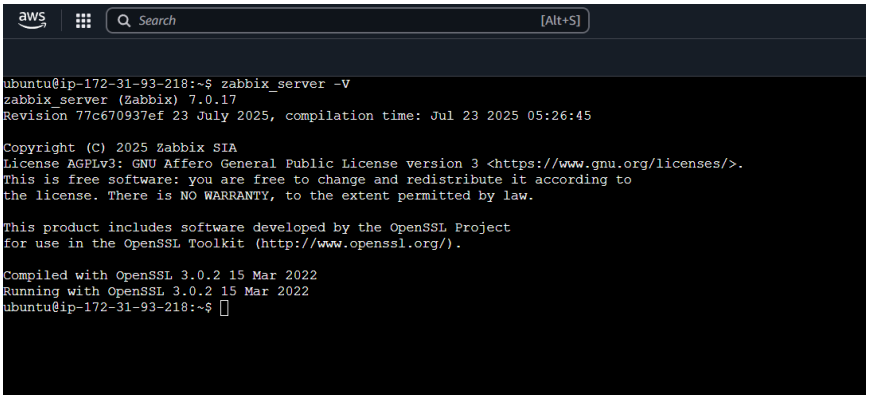
```
mysql -uroot –p
set global log_bin_trust_function_creators = 0;
quit;
```

b) Konfigurasi

1) Hal pertama yang perlu dikonfigurasi adalah password yang berada pada file /etc/zabbix/zabbix_server.conf, dan sesuikan dengan password database yang telah dibuat pada langkah installasi 5. Hal yang perlu diingat adalah untuk selalu restart service setalah melakukan modifikasi pada file konfigurasi agar konfigurasi dapat disimpan dan dijalankan oleh service.

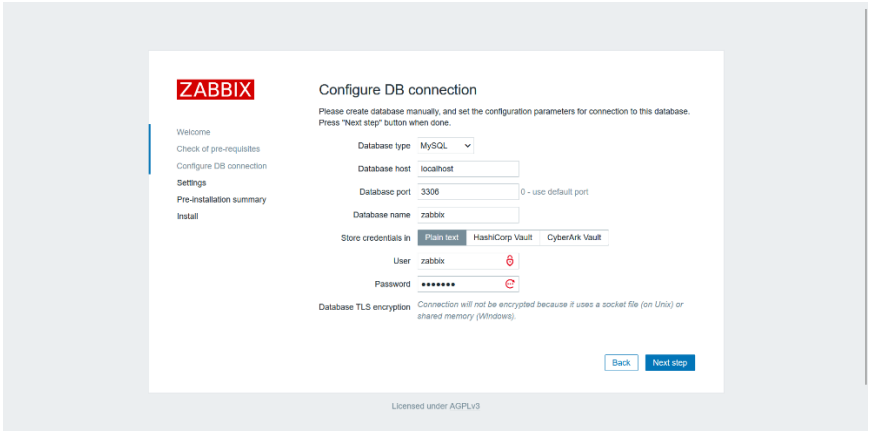
```
systemctl restart zabbix-server zabbix-agent apache2
systemctl enable zabbix-server zabbix-agent apache2
```

2) Untuk memastikan apakah zabbix sudah terinstall jalankan perintah Zabbix_server -V yang akan menampilkan informasi tentang Zabbix server yang terinstall dan berjalan pada instance.



Gambar 4. 10 cek status Zabbix server

3) Setelah Zabbix server dan agen berjalan, antarmuka Zabbix dapat diakses melalui browser menggunakan URL default: http://<ip-server>/zabbix.
4) Ikuti panduan konfigurasi awal di web UI untuk menyelesaikan instalasi, seperti memilih bahasa, konfigurasi koneksi database, dan set administrator account. Setelah konfigurasi awal selesai, Zabbix siap digunakan untuk monitoring infrastruktur SOC.



Gambar 4. 11 configurasi awal wazuh server

5) Setalah konfigurasi login menggunakan default user dan password dari zabbix
6) Secara default zabbix akan melakukan self monitoring

4.2.4 Grafana

Grafana adalah tool visualisasi open source yang digunakan untuk memantau data metrik dan membuat dashboard interaktif dari berbagai sumber data seperti Zabbix, Prometheus, dan database lainnya. Pada implementasi ini, Grafana digunakan untuk menampilkan hasil monitoring utilization dari sistem SOC secara real-time. Proses instalasi dilakukan pada instance *HQ-Zabbix-Grafana* yang menggunakan sistem operasi Ubuntu Server 22.04.

Grafana dapat diinstal melalui beberapa metode seperti APT repository, file .deb, maupun binary .tar.gz. Namun dalam implementasi ini, instalasi dilakukan melalui APT repository resmi Grafana agar lebih praktis dan bisa mendapatkan update otomatis saat melakukan apt-get update.

a) Installasi

1) Hal pertama yang harus dilakukan adalah menambahkan dependensi repository Grafana agar sistem dapat mengambil paket dari repositori resmi. Jalankan perintah berikut:

```
sudo apt-get install -y apt-transport-https software-properties-common wget
```

2) Setelah itu, import GPG key milik Grafana yang berfungsi untuk memverifikasi integritas paket yang akan diunduh.

```
sudo mkdir -p /etc/apt/keyrings/
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee /etc/apt/keyrings/grafana.gpg > /dev/null
```

3) Langkah selanjutnya yaitu menambahkan repository stable Grafana ke dalam sistem agar bisa mendownload versi OSS terbaru.

```
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo tee -a /etc/apt/sources.list.d/grafana.list
```

4) Setelah repository ditambahkan, lakukan update agar sistem mengenali paket-paket terbaru dari Grafana.

```
sudo apt-get update
```

5) Terakhir, install Grafana versi OSS menggunakan perintah berikut:

```
sudo apt-get install grafana
```

b) Konfigurasi

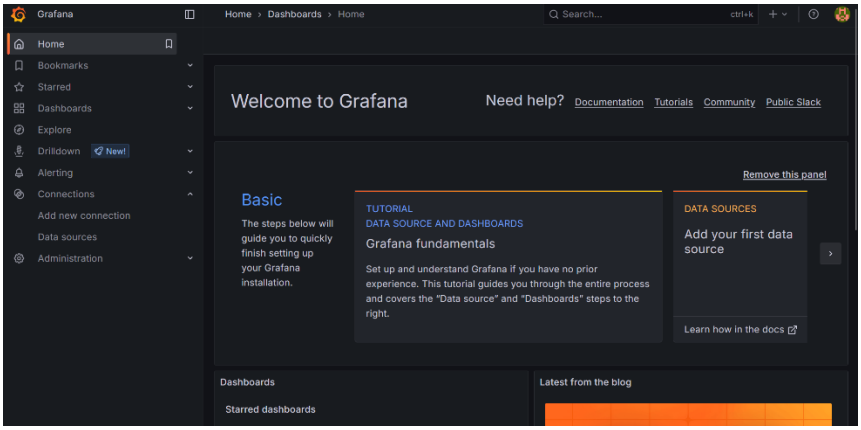
1) Setelah proses instalasi selesai, hal pertama yang harus dilakukan adalah menjalankan layanan Grafana agar bisa mulai diakses melalui browser.

```
sudo systemctl start grafana-server
```

2) Agar layanan Grafana otomatis berjalan setiap kali server direstart, aktifkan layanan tersebut menggunakan perintah berikut:

```
sudo systemctl enable --now grafana-server
```

3) Setelah Grafana berjalan, antarmuka web Grafana dapat diakses melalui browser dengan mengetikkan alamat http://<ip-server>:3000. Port 3000 adalah port default yang digunakan oleh Grafana.
4) Pada saat pertama kali membuka halaman Grafana, pengguna akan diminta login dengan username dan password default, yaitu admin untuk keduanya.
5) Setelah berhasil login, sistem akan meminta pengguna untuk mengganti password admin default demi alasan keamanan. Gunakan password baru yang aman dan mudah diingat, namun untuk projek ini akan tetap menggunakan password default.



Gambar 4. 12 halaman utama grafana

Setelahnya kita dapat mulai menambahkan data source seperti Zabbix dan membuat dashboard pemantauan sesuai kebutuhan, untuk tahapan ini akan dilakukan pada bagian integrasi antara kedua komponen.

4.2.5 The Hive

Instalasi TheHive akan dilakukan pada instance *HQ-TheHive-Cortex*, di mana proses ini cukup kompleks karena membutuhkan beberapa dependensi seperti Cassandra dan Elasticsearch sebagai backend storage serta Java sebagai runtime environment. Semua langkah di bawah mengikuti panduan resmi dari StrangeBee Docs dan disesuaikan dengan kebutuhan lingkungan dan telah dibuat sesederhana mungkin. Secara lengkap berikut langkahnya.

a) Installasi

1) Pertama, lakukan update pada daftar repositori agar semua paket yang akan dipasang adalah versi terbaru dengan menggunakan perintah **`sudo apt update`**.
2) Tambahkan repository universe yang kadang diperlukan, terutama saat menginstall pip.

```
sudo add-apt-repository universe
sudo apt update
```

3) Install pip3 jika belum terpasang (pastikan Python3 sudah tersedia).

```
sudo apt install python3-pip -y
```

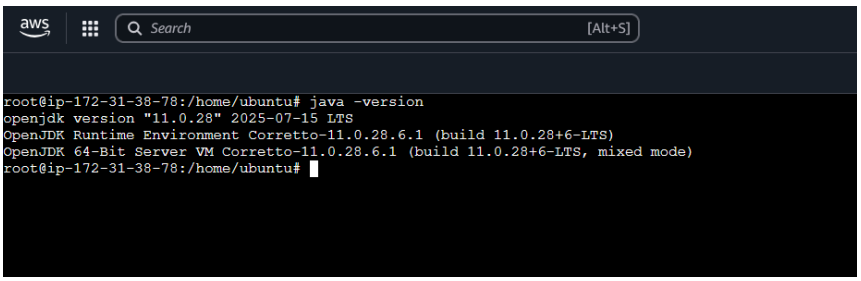
4) Install dependensi tambahan yang dibutuhkan untuk proses instalasi keseluruhan.

```
sudo apt install wget gnupg apt-transport-https git ca-certificates ca-certificates-java curl software-properties-common python3-pip lsb-release -y
```

5) Install Java Virtual Machine (Amazon Corretto JDK 11), yang merupakan runtime yang diperlukan oleh TheHive.

```
wget -qO- https://apt.corretto.aws/corretto.key | sudo gpg --dearmor -o /usr/share/keyrings/corretto.gpg
echo "deb [signed-by=/usr/share/keyrings/corretto.gpg] https://apt.corretto.aws stable main" | sudo tee -a /etc/apt/sources.list.d/corretto.sources.list
sudo apt update
sudo apt install java-common java-11-amazon-corretto-jdk -y
echo  JAVA_HOME="/usr/lib/jvm/java-11-amazon-corretto" | sudo tee -a /etc/environment
export         JAVA_HOME="/usr/lib/jvm/java-11-amazon-corretto"
```

6) Untuk memastikan bahwa JVM telh terinstall jalankan perintah **`java -version`**.



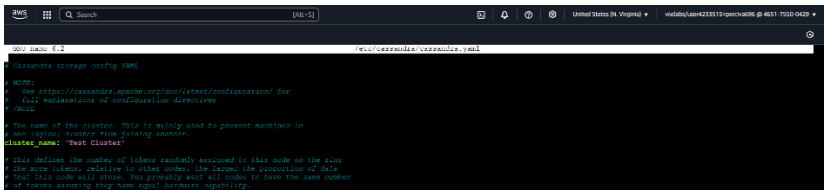
Gambar 4. 13 java version

7) Lanjut install Apache Cassandra yang merupakan database utama untuk TheHive.

```
wget -qO - https://downloads.apache.org/cassandra/KEYS
| sudo gpg --dearmor -o /usr/share/keyrings/cassandra-
archive.gpg
echo "deb [signed-by=/usr/share/keyrings/cassandra-
archive.gpg] https://debian.cassandra.apache.org 41x
main" | sudo tee -a
/etc/apt/sources.list.d/cassandra.sources.list
sudo apt update
sudo apt install cassandra -y
```

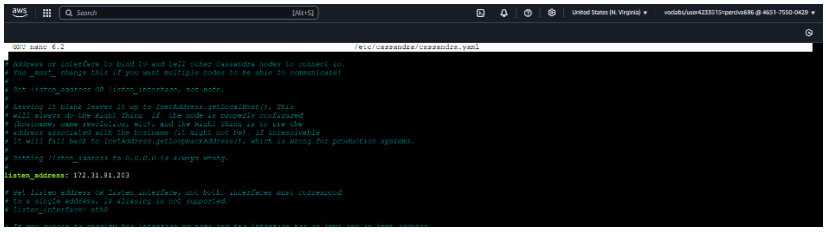
8) Ada beberapa hal yang perlu diubah pada file configurasi milik casandra yang berada pada direktori **`/etc/cassandra/cassandra.yaml`**, Diantaranya:

- cluster_name, Dimana pada kasus ini diubah menjadi ‘Test Cluster’, bagian ini bebas untuk diubah sesuai dengan kebutuhan.



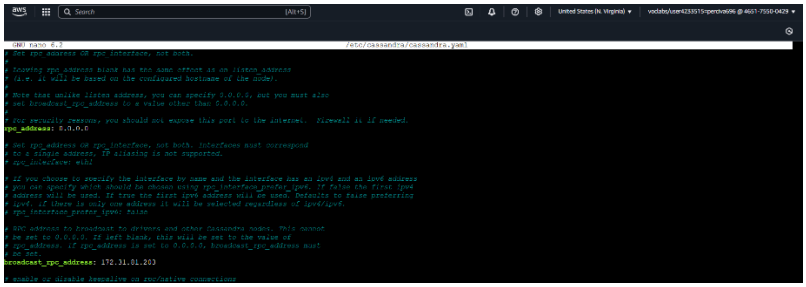
Gambar 4. 14 cluster name thehive

- listen_address diubah menjadi ip privat instance.



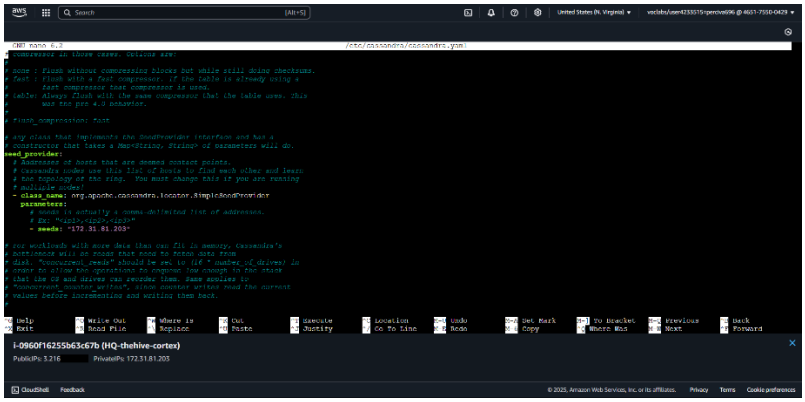
Gambar 4. 15 listen address thehive

- Ubah rpc_address menjadi 0.0.0.0 dengan tujuan Memungkinkan Cassandra menerima koneksi dari semua interface. Dan Ubah broadcast_rpc_address menjadi private IP dari instance.



Gambar 4. 16 rpc address thehive

- Untuk bagian seed_provider, sesuaikan seeds dengan IP private dari instance yang akan menjadi node awal.

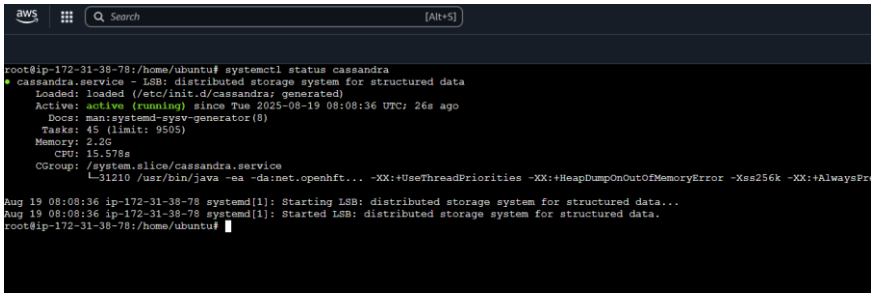


Gambar 4. 17 seed provider thehive

9) Setelah konfigurasi selesai hapus file yang sudah tidak dibutuhkan, dan restart service Cassandra:

```
sudo systemctl stop cassandra
sudo rm -rf /var/lib/cassandra/*
sudo systemctl start cassandra
sudo systemctl enable cassandra
```

10) Cek status dari casandra untuk memastikan apakah service sudah berjalan dengan benar dengan menggunakan perintah **`sudo systemctl status Cassandra.`**



Gambar 4. 18 status cassandra

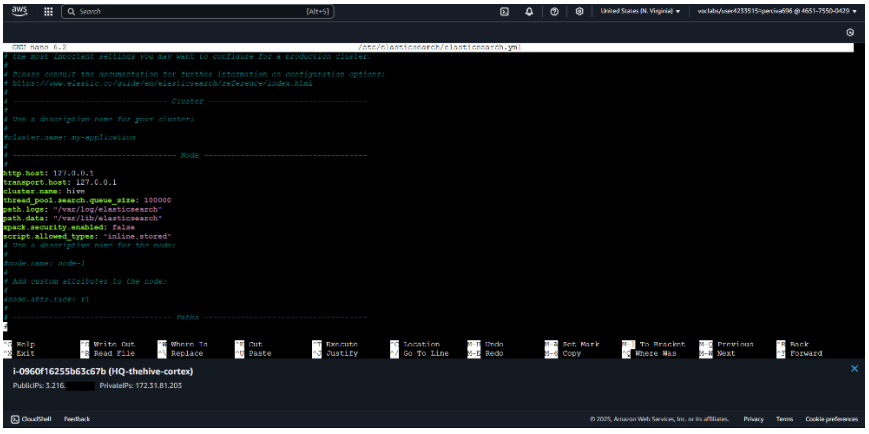
11) Komponen selanjutnya adalah Elasticsearch yang akan digunakan sebagai backend pencarian. Versi yang akan digunakan adalah versi 7.x.

```
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg
sudo apt-get install apt-transport-https -y
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/7.x/apt stable main" | sudo tee -a /etc/apt/sources.list.d/elastic-7.x.list
sudo apt update
sudo apt install elasticsearch -y
```

12) Konfigurasi Elasticsearch dengan mengedit file /etc/elasticsearch/elasticsearch.yml Dimana terdapat beberapa bagian yang perlu diubah yaitu:

- **`http.host: 127.0.0.1`** : Agar Elasticsearch hanya menerima koneksi HTTP dari localhost (untuk keamanan, tidak bisa diakses dari luar).
- **`transport.host: 127.0.0.1`**: Membatasi komunikasi internal antar node Elasticsearch (jika ada) hanya di localhost.
- **`cluster.name: hive`** : Menamai klaster Elasticsearch, digunakan oleh TheHive untuk identifikasi klaster yang digunakan.

- **`thread_pool.search.queue_size: 100000`** : Meningkatkan antrean pencarian untuk menangani banyak permintaan search dari TheHive, agar tidak cepat penuh.
- **`path.logs: /var/log/elasticsearch`**: Menentukan lokasi penyimpanan log Elasticsearch.
- **`path.data: /var/lib/elasticsearch`**: Menentukan lokasi penyimpanan data Elasticsearch.
- **`xpack.security.enabled: false`**: Menonaktifkan fitur keamanan X-Pack (seperti autentikasi dan enkripsi), biasanya untuk kemudahan integrasi dengan TheHive dalam lingkungan non-produksi.
- **`script.allowed_types: "inline, stored"`** (Opsional): Mengizinkan Elasticsearch menjalankan dua jenis script yaitu inline (Script langsung ditulis di dalam query) dan stored (Script disimpan sebelumnya di Elasticsearch dan dipanggil saat dibutuhkan)



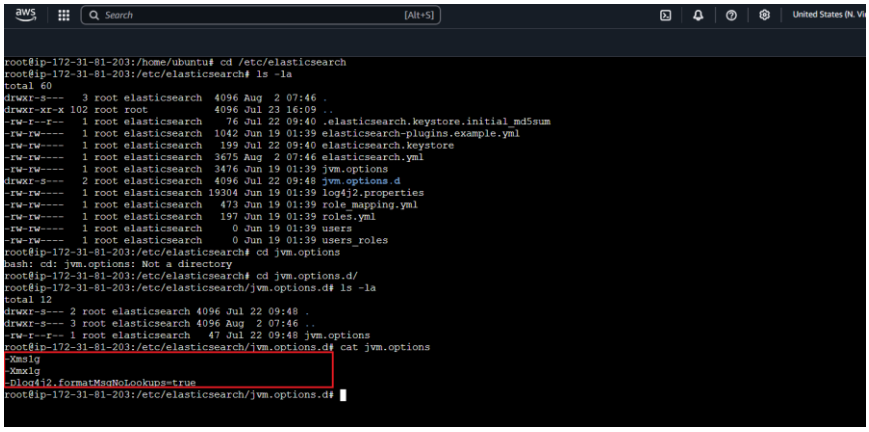
Gambar 4. 19 script allowed thehive

13) Buat direktori khusus untuk konfigurasi JVM jika belum ada, kemudian tambahkan pengaturan memori dan keamanan dengan menggunakan perintah:

```
sudo mkdir -p /etc/elasticsearch/jvm.options.d
sudo nano /etc/elasticsearch/jvm.options.d/jvm.options
```

dan isi dengan:

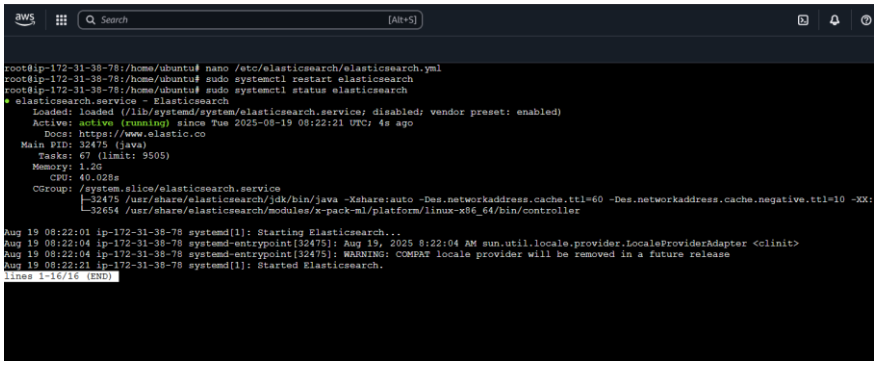
```
-Xms1g
-Xmx1g
-Dlog4j2.formatMsgNoLookups=true
```



Gambar 4. 20 configurasi jvm

14) Restart Elasticsearch dan pastikan berjalan normal.

```
sudo systemctl restart elasticsearch
sudo systemctl status elasticsearch
```



Gambar 4. 21 status elasticsearch

15) Buat user dan group untuk TheHive.

```
sudo adduser --system --no-create-home --group --
disabled-login thehive
```

16) TheHive sendiri menyediakan dua cara untuk menyimpan file yaitu secara lokal dan menggunakan minio, untuk projek ini kit akan

menyimpan pada local storage. Buat direktori penyimpanan file lokal untuk TheHive pada /opt/thp/thehive/files lalu ubah permission agar dimiliki oleh user thehive:

```
sudo mkdir -p /opt/thp/thehive/files
sudo chown -R thehive:thehive /opt/thp/thehive/files
```

17) Terakhir, install TheHive dengan menambahkan repository StrangeBee dan kemudian mulai proses installasi.

```
echo 'deb [arch=all signed-by=/usr/share/keyrings/strangebee-archive-keyring.gpg] https://deb.strangebee.com thehive-5.5 main' | sudo tee -a /etc/apt/sources.list.d/strangebee.list
sudo apt-get update
sudo apt-get install -y thehive
```

b) Konfigurasi

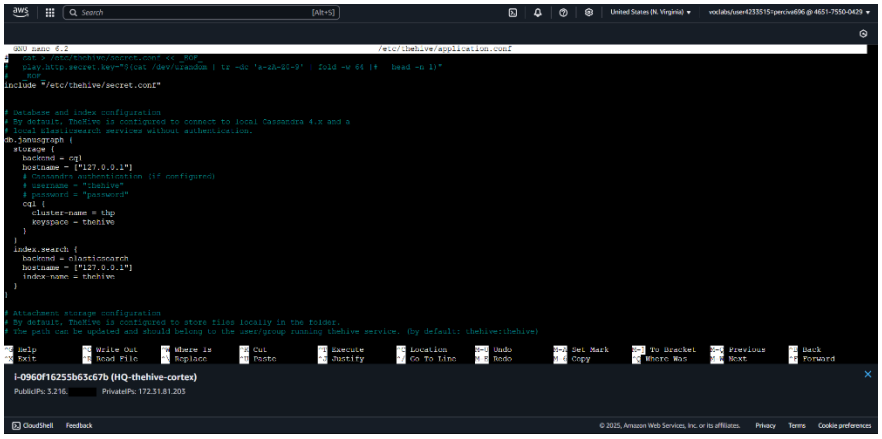
1) Setelah berhasil terinstall, konfigurasikan baseURL TheHive agar sesuai dengan IP publik (Elastic IP)instance. File configurasi untuk the hive sendiri berapa di etc/thehive/application.conf

```
sudo nano /etc/thehive/application.conf
```



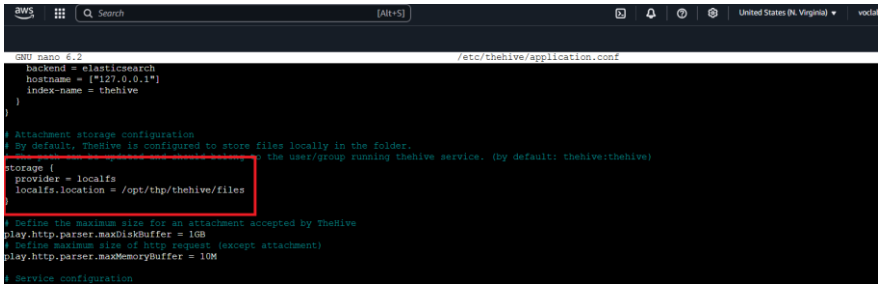
Gambar 4. 22 baseUrl thehive

2) Pada bagian Databse tidak perlu dilakukan perubahan cukup mengikuti default setting pada application.conf saja.



Gambar 4. 23 database config thehive

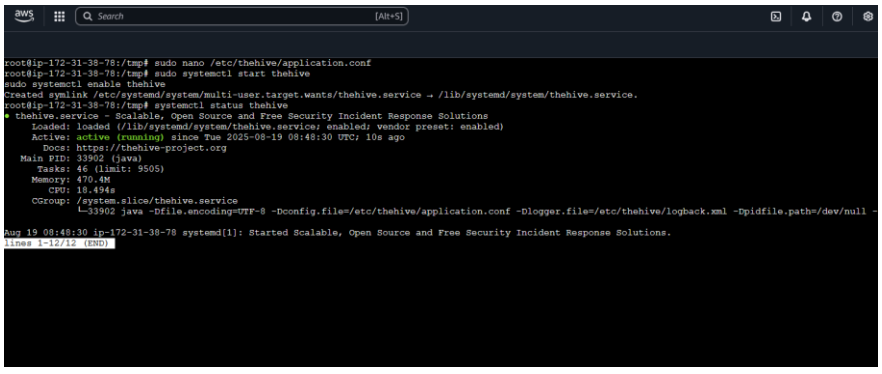
3) Pada bagian Storage juga sudah disesuaikan dengan lokasi path dimana nantinya file akan disimpan oleh TheHive.



Gambar 4. 24 lokasi storage thehive

4) Setelah semua proses selasai langkah terakhir adalah dengan menjalankan the hive dengan perintah:

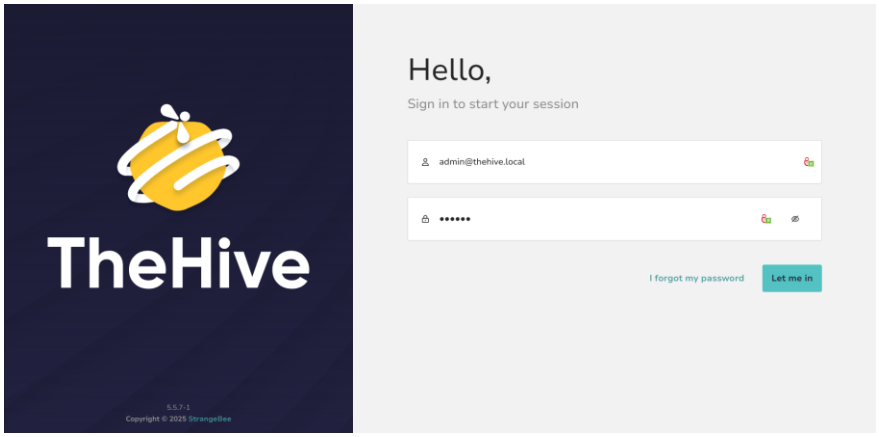
```
sudo systemctl start thehive
sudo systemctl enable thehive
```



Gambar 4. 25 status thehive

5) Setelah service berjalan dengan sukses masuk ke browser dan buka halaman the hive dengan menggunakan http://ElasticIP-Instance:9000/. Setelahnya kitab isa login untuk melakukan konfigurasi lebih lanjut seperi membuat user, group ataupun API key yang akan digunakan untuk proses Automation nantinya. Default kredensial admin dari the hive adalah:

```
Username: admin@thehive.local
Password: secret
```



Gambar 4. 26 halaman login thehive

4.2.6 Cortex

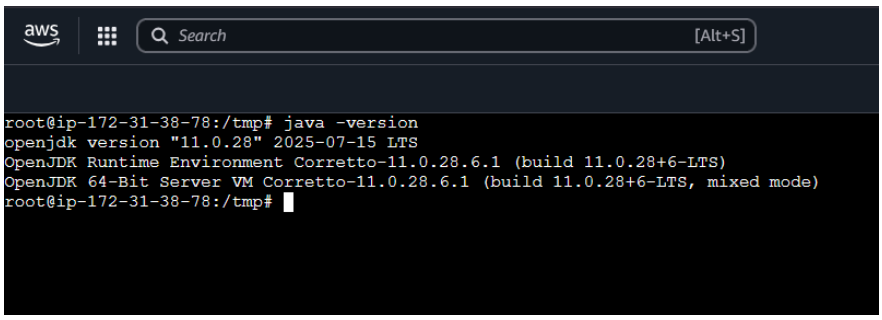
Untuk proses instalasi Cortex ini dilakukan pada instance *HQ-TheHive-Cortex*, yaitu instance yang sama dengan tempat instalasi TheHive sebelumnya. untuk langkah lengkap dari installasi dapat mengunjungi halaman Cortex Installation. Karena beberapa dependensi sudah tersedia dari proses sebelumnya, maka ada beberapa langkah yang bisa dilewati.

a) Installasi

1) Langkah awal adalah menginstall beberapa package pendukung yang dibutuhkan oleh Cortex agar dapat berjalan dengan lancar di sistem. Jalankan perintah berikut:

```
sudo apt install wget gnupg apt-transport-https git ca-certificates ca-certificates-java curl software-properties-common python3-pip lsb-release -y
```

2) Karena Java Virtual Machine (JVM) sudah diinstal sebelumnya saat proses setup TheHive, maka di sini cukup lakukan verifikasi dengan menjalankan java -version.



Gambar 4. 27 java version

3) Begitu juga dengan Elasticsearch yang sudah diinstall saat menginstall Thehive sebelumnya.
4) Selanjutnya, tambahkan GPG key dan repository resmi milik TheHive Project.

```
curl -sSL https://raw.githubusercontent.com/TheHive-Project/Cortex/master/PGP-PUBLIC-KEY | sudo gpg --dearmor -o /usr/share/keyrings/thehive-project.gpg
echo "deb [arch=all signed-by=/usr/share/keyrings/thehive-project.gpg] https://deb.thehive-project.org release main" | sudo tee /etc/apt/sources.list.d/thehive-project.list
```

5) Lakukan update repository dan install Cortex

```
sudo apt update
sudo apt install cortex -y
```

6) Setelah instalasi selesai, ubah kepemilikan direktori Cortex agar user yang kita gunakan (pada contoh ini Adalah ubuntu) memiliki hak akses penuh.

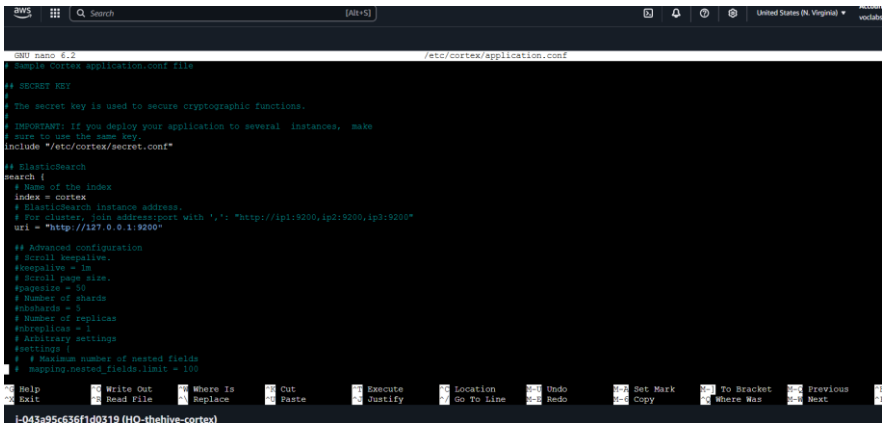
```
sudo chown -R ubuntu:ubuntu /opt/cortex
```

b) Konfigurasi

1) Langkah pertama pada tahap konfigurasi adalah membuat file secret.conf yang berisi secret key yang dibutuhkan oleh Cortex

```
sudo tee /etc/cortex/secret.conf > /dev/null << _EOF_
play.http.secret.key="$(cat /dev/urandom | tr -dc 'a-zA-Z0-9' | fold -w 64 | head -n 1)"
_EOF_
```

2) Setelah itu, edit file konfigurasi utama Cortex yang ada di /etc/cortex/application.conf, dan tambahkan baris berikut agar file secret.conf dapat dimuat dengan benar sesuai engan lokasi dari secret key yang telah dibuat sebelumnya.
3) Untuk konfigurasi koneksi ke Elasticsearch yang digunakan sebagai database Cortex, sesuaikan dengan pengaturan berikut:



Gambar 4. 28 configurasi elasticsearch cortex

4) Selanjutnya adalah proses konfigurasi analyzer dan responder. Di sini digunakan dua sumber sekaligus, yaitu katalog online resmi dari StrangeBee dan katalog lokal hasil clone dari GitHub. Sebelumnya, install dulu Docker karena akan digunakan untuk menjalankan analyzer dan responder dengan cara yang lebih aman.

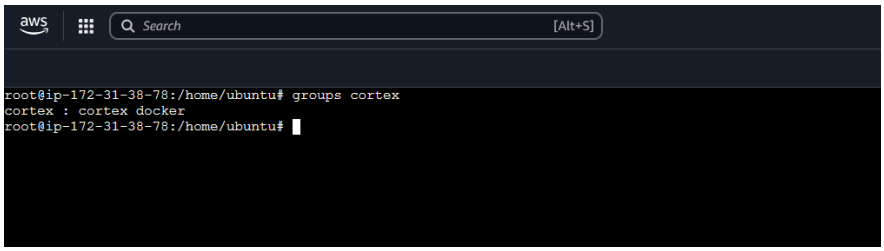
```
sudo apt update
sudo apt install -y docker.io
```

```
sudo systemctl enable --now docker
```

5) Lalu pastikan user cortex masuk dalam grup docker agar bisa menjalankan container tanpa error:

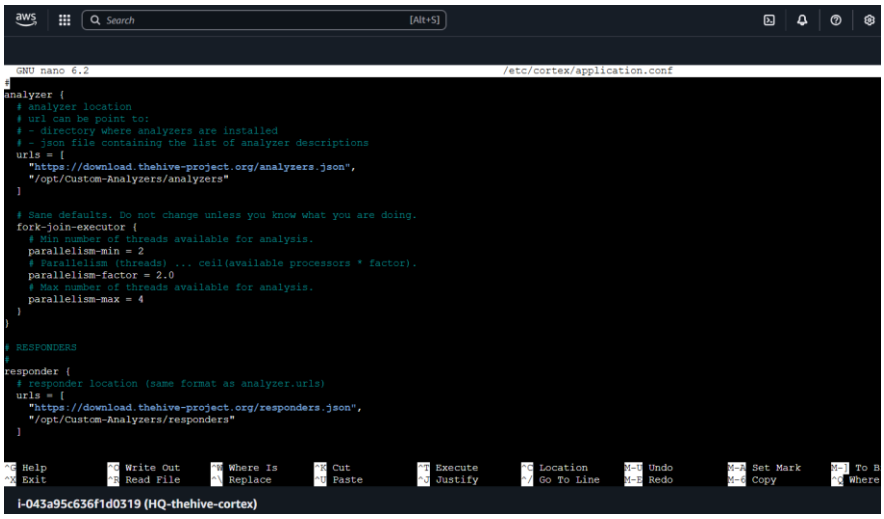
```
sudo usermod -aG docker cortex
sudo reboot
```

6) Setelah reboot, pastikan user cortex sudah berada di grup docker dengan perintah



Gambar 4. 29 group user cortex

7) Edit kembali file application.conf dan tambahkan konfigurasi berikut untuk memuat katalog analyzer dan responder dari dua sumber:



Gambar 4. 30 cortex catalog

8) clone repository analyzer/responder ke direktori local

```
cd /opt
```

```
sudo git clone https://github.com/TheHive-Project/Cortex-Analyzers
sudo chown -R cortex:cortex /opt/Cortex-Analyzers
```

9) Install seluruh dependencies Python yang dibutuhkan oleh analyzer menggunakan perintah berikut:

```
for I in $(find Cortex-Analyzers -name 'requirements.txt'); do sudo -H pip3 install -r $I || true; done
```

10) Jalankan skrip fix-ubuntu.sh untuk memperbaiki kompatibilitas analyzer terhadap Python 3.10+:

```
cd /opt/Cortex-Analyzers
sudo bash fix-ubuntu.sh
```

11) Lakukan penggantian sintaks Mapping agar tidak error saat analyzer dijalankan:

```
find . -type f -name "*.py" -exec sed -i 's/from collections import Mapping/from collections.abc import Mapping/g' {} +
```

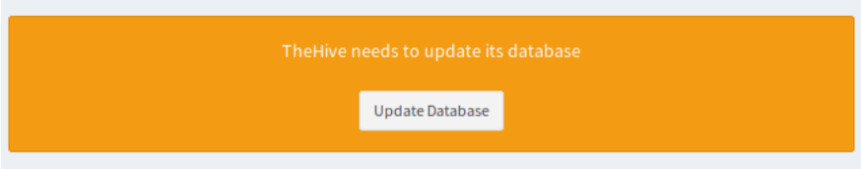
12) Terakhir, upgrade library urllib3 dan requests ke versi yang lebih baru agar semua analyzer bisa berjalan dengan baik:

```
sudo pip install --upgrade urllib3 requests
```

13) Jika semua konfigurasi sudah selesai, restart dan cek apakah Cortex sudah berjalan:

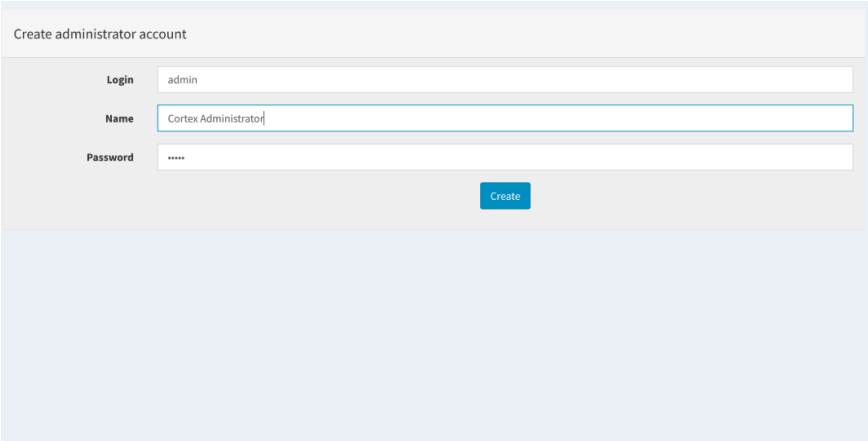
```
sudo systemctl restart cortex
sudo systemctl status cortex
```

14) Kemudian masuk ke halaman ui dari cortex menggunakan http://Instance-IP:9001, dan akan diminta untuk membuat database, cukup klik update database maka cortex otomatis akan membuat databse secra otomatis.



Gambar 4. 31 update database cortex

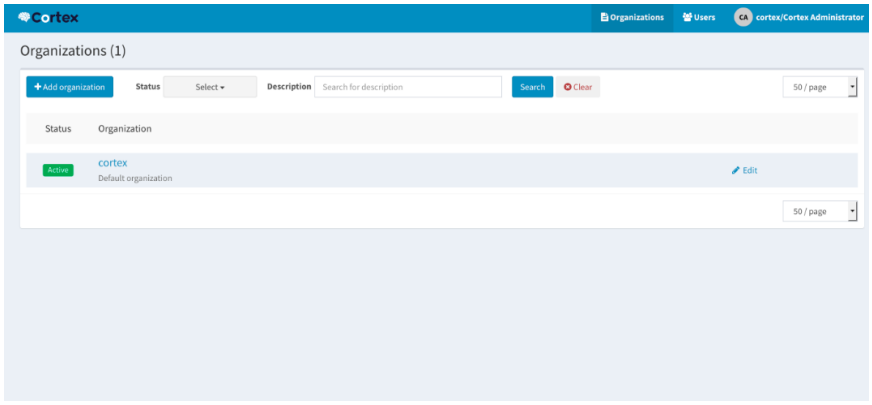
15) Setelah database berhasil dibuat selanjutnya cortex akan menampilkan halaman utuk membuat akun administrator, Dimana akun inilah yang nantinya akan digunakan untuk membuat organisai dan dan user untuk cortex.



Gambar 4. 32 buat akun administrator cortex

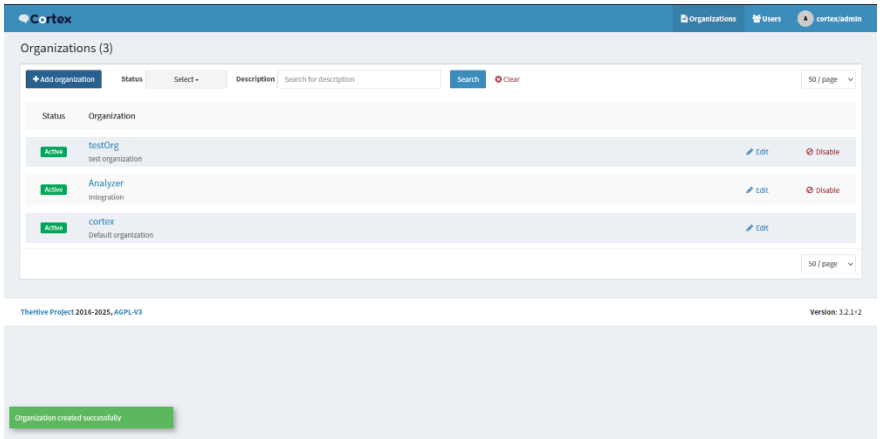
16) Kemudian gunakan user tersebut untuk login kedalam cortex

17) Saat pertama kali login, akan terlihat bahwa sudah otomatis dibuat satu organisasi default bernama cortex dan akun admin kita sudah termasuk di dalamnya dengan role sebagai Cortex Global Administrator.



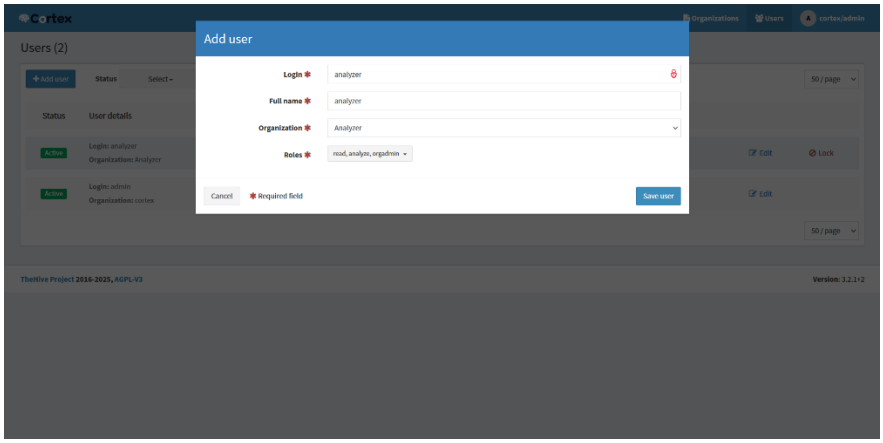
Gambar 4. 33 organisasi default cortex

18) Kita bisa menambahkan beberapa grup organisasi baru sesuai kebutuhan. Caranya cukup klik tombol "Add Organization" dan isi form yang disediakan seperti nama organisasi dan deskripsinya.



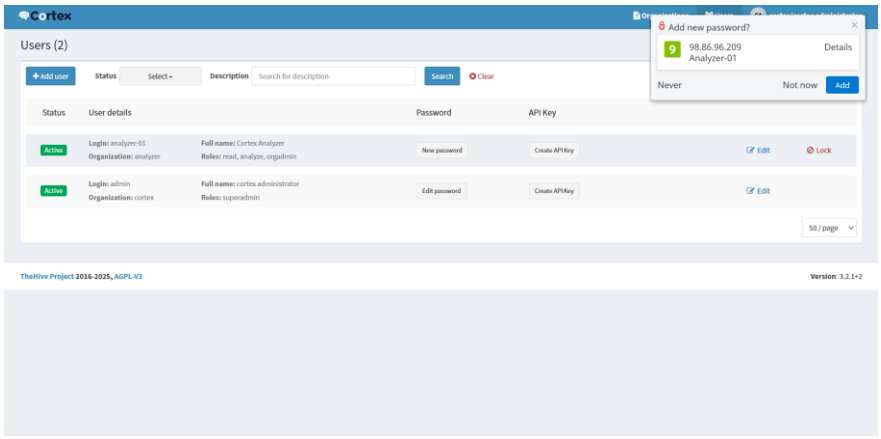
Gambar 4. 34 buat organisasi cortex

19) Selanjutnya buat user baru bernama analyzer dan menempatkannya pada organisasi yang diinginkan, disini user aakn dimasukan ke organisasi analyzer yang telah dibuat sebelumnya. Untuk role, berikan akses read, analyze, dan orgadmin agar user ini dapat menjalankan analyzer sekaligus mengelola organisasi tersebut.



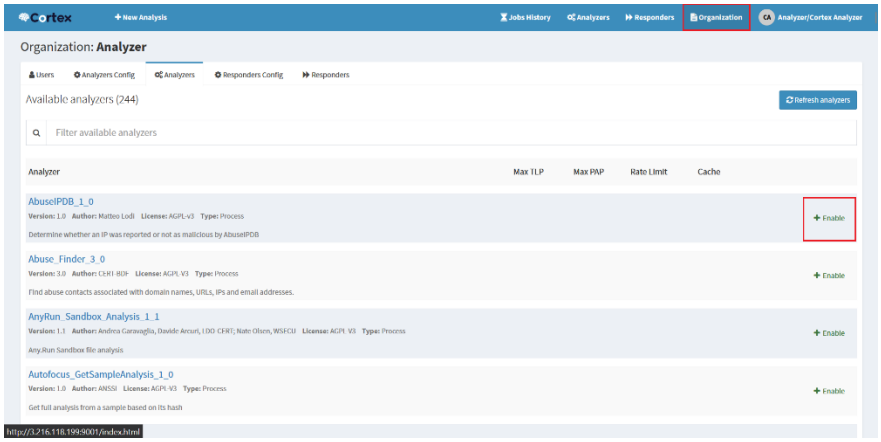
Gambar 4. 35 buat user baru cortex

20) Atur password untuk user yang telah dibuat dengan klik pada new password.



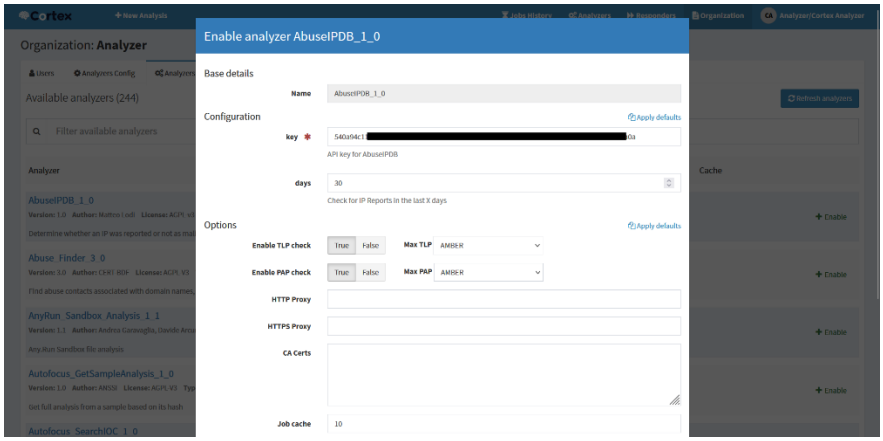
Gambar 4. 36 set user passowrd

21) Setelah itu, kita bisa logout dari akun admin, lalu login menggunakan akun analyzer yang telah kita buat tadi. Untuk mencoba penggunaan analyzer, klik pada tab organization kemudian klikl tab Analyzer. Di sini akan ditampilkan daftar semua analyzer yang tersedia. pilih salah satu analyzer, contohnya **AbuseIPDB**, karena kita akan mencoba melakukan analisis terhadap sebuah IP address. Klik tombol enable untuk mengaktifkan analyzer tersebut.



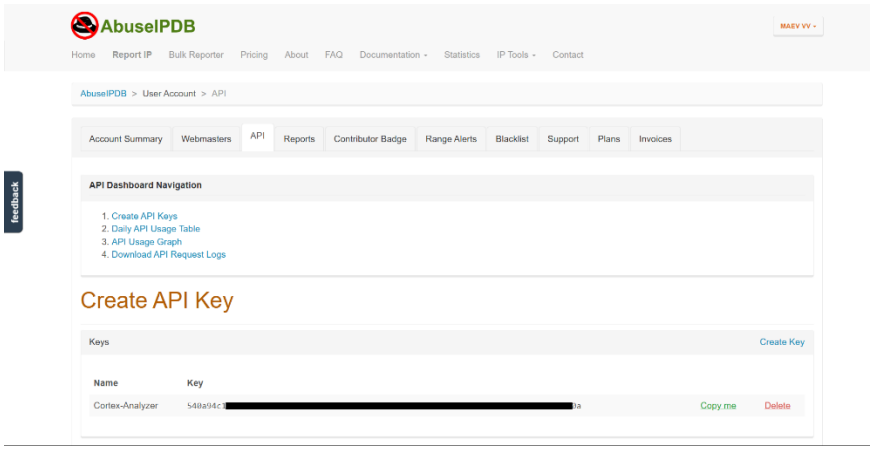
Gambar 4. 37 enable analyzer

22) Saat mengaktifkan analyzer, akan muncul form konfigurasi yang perlu diisi. Biasanya akan ada beberapa parameter penting, dan yang ditandai dengan tanda bintang merah wajib diisi. Dalam kasus AbuseIPDB, yang wajib hanya **API Key**.



Gambar 4. 38 set api key analyzer

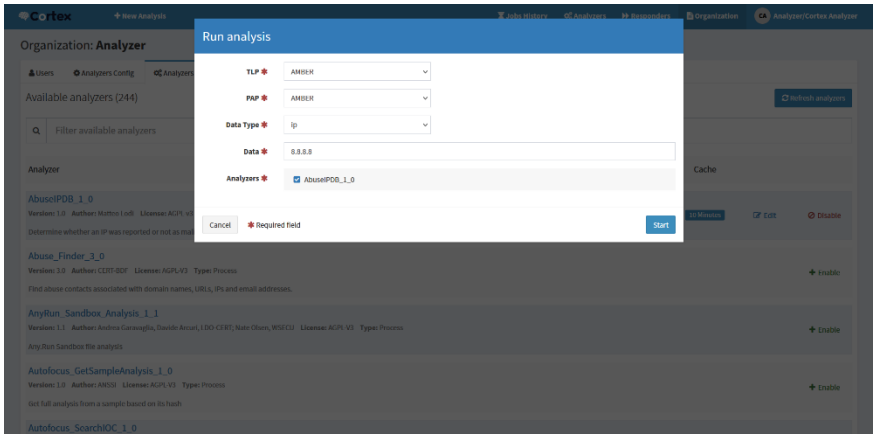
23) Kita bisa mendapatkan API Key ini dengan mendaftar di situs **abuseipdb.com**, kemudian paste API Key tersebut ke kolom yang tersedia dan klik Save.



Gambar 4. 39 mengambil api key abuseIpDb

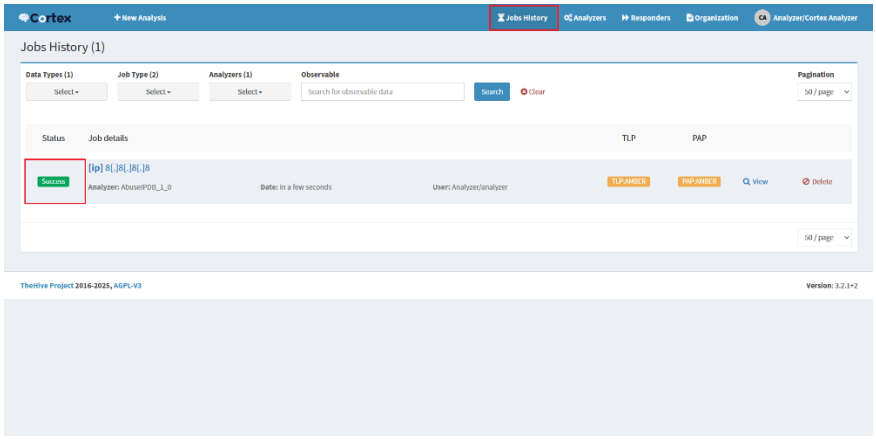
24) Setelah berhasil disimpan, sekarang kita bisa mencoba menjalankan analyzer. Klik tombol New Analysis di bagian kanan atas, tepat di sebelah logo Cortex.

25) Pada bagian ini, kita pilih Data Type menjadi ip, kemudian masukkan IP yang ingin kita analisis, misalnya 8.8.8.8 (IP milik Google DNS). Pada bagian Analyzers, centang AbuseIPDB sebagai analyzer yang akan dijalankan, kemudian klik tombol Start untuk memulai proses analisis.



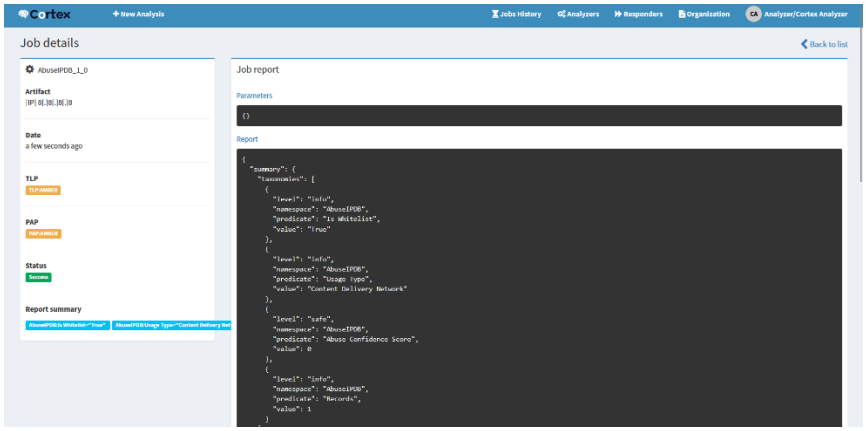
Gambar 4. 40 test analysis

26) Untuk melihat hasilnya, buka tab Job History. Di sana akan terlihat status dari job analyzer yang kita jalankan tadi. Jika prosesnya berhasil, statusnya akan muncul sebagai Success.



Gambar 4. 41 hasil analisys cortex

27) Klik Detail pada job tersebut untuk melihat hasil lengkap dari analisis. Di dalamnya akan ditampilkan informasi-informasi penting seperti level abuse, catatan pelaporan sebelumnya terhadap IP tersebut, dan berbagai metrik lain yang sangat berguna nantinya dalam proses **enrichment** otomatis saat terjadi insiden keamanan di sistem.



Gambar 4. 42 detail analysis cortex

4.2.7 Zammad

a) Installasi

1) Langkah pertama adalah menginstall tools pendukung dasar seperti curl, gnupg, dan apt-transport-https yang diperlukan untuk proses instalasi repositori.

```
sudo apt install curl apt-transport-https gnupg -y
```

2) Setelah itu, tambahkan repository Elasticsearch secara resmi menggunakan metode keyring di direktori /usr/share/keyrings agar lebih aman dan terstandarisasi.

```
curl -fsSL https://artifacts.elastic.co/GPG-KEY-elasticsearch | \
gpg --dearmor | sudo tee /usr/share/keyrings/elasticsearch-keyring.gpg > /dev/null

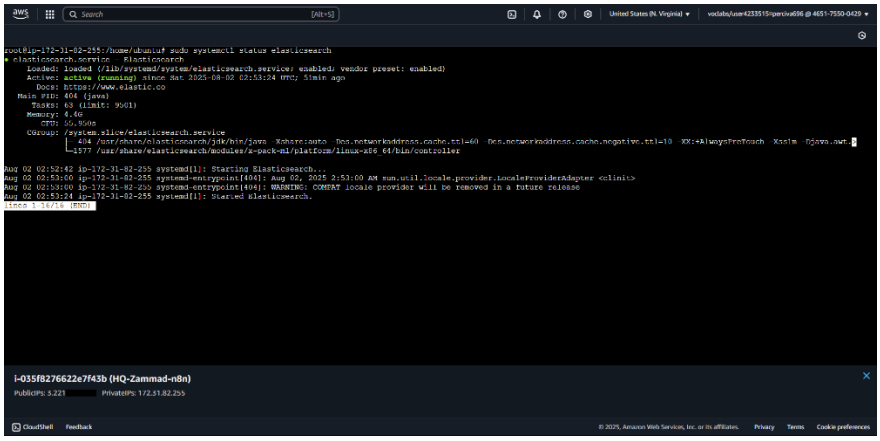
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/7.x/apt stable main" | \
 sudo tee /etc/apt/sources.list.d/elastic-7.x.list
```

3) Lanjutkan dengan update paket dan install Elasticsearch versi 7. Sesuai dengan yang kita tambahkan pada repository sebelumnya.

```
sudo apt update
sudo apt install elasticsearch -y
```

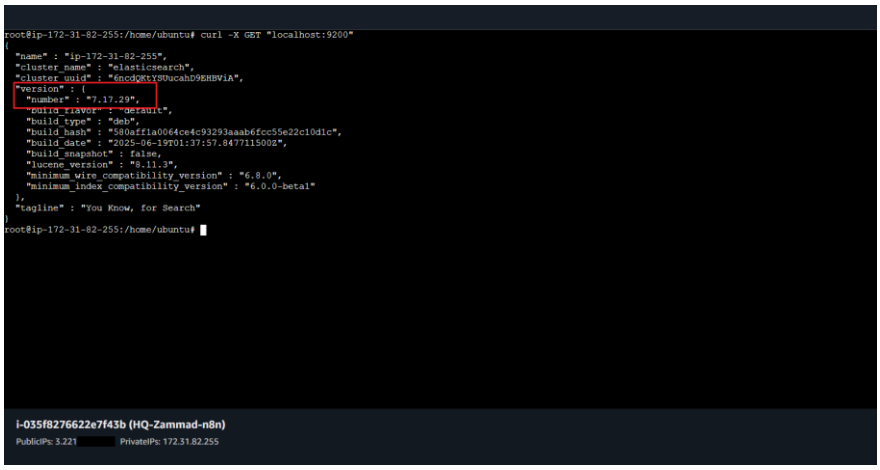
4) Aktifkan dan jalankan service Elasticsearch, lalu pastikan statusnya running.

```
sudo systemctl enable --now elasticsearch
sudo systemctl start elasticsearch
sudo systemctl status elasticsearch
```



Gambar 4. 43 enable elasticsearch

5) Setelah service jalan, kita bisa cek versi Elasticsearch dengan menggunakan perintah curl -X GET "localhost:9200", dan pada terminal dapat dilihat versi elasticsearch yang berjalan pada instance adalah 7.17



Gambar 4. 44 versi elasticsearch

6) Karena kita menggunakan Elasticsearch versi 7, maka perlu install plugin ingest-attachment agar Elasticsearch bisa melakukan parsing file berbasis attachment.

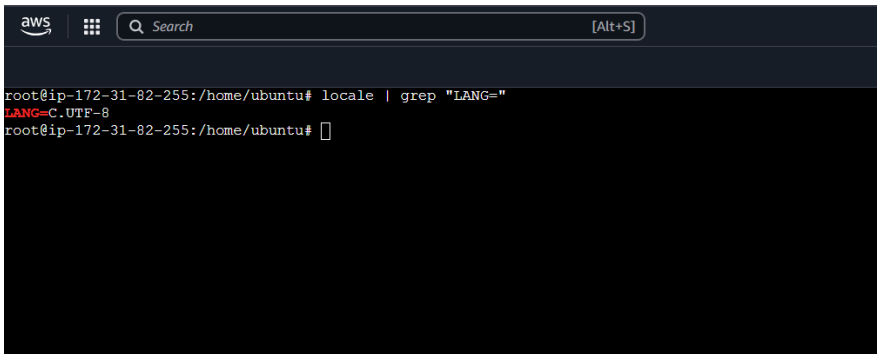
```
sudo /usr/share/elasticsearch/bin/elasticsearch-plugin
install ingest-attachment
```

7) Kemudian tambahkan setting untuk menaikkan limit memory map virtual agar Elasticsearch bisa berjalan stabil saat beban tinggi dan

menghindari error saat elasticsearch membuat banyak index dalam memory dengan menggunakan perintah:

```
sudo sysctl -w vm.max_map_count=262144
```

8) Cek setting lokal sistem untuk memastikan tidak ada masalah regionalisasi.



Gambar 4. 45 cek local system

9) Sekarang tambahkan repository Zammad ke sistem agar paket Zammad dapat diambil langsung dari repo resminya.

```
echo "deb [signed-by=/etc/apt/keyrings/pkgr-zammad.gpg] https://dl.packager.io/srv/deb/zammad/zammad/stable/ubuntu 22.04 main" | \
sudo tee /etc/apt/sources.list.d/zammad.list > /dev/null
```

10) Setelah repo ditambahkan, jalankan update dan install Zammad:

```
sudo apt update
sudo apt install zammad -y
```

b) Konfigurasi

1) Setelah Zammad berhasil diinstall, kita perlu menghubungkannya dengan Elasticsearch. Karena pengaturan es_url disimpan di dalam konfigurasi internal Zammad (bukan di file .env), maka kita perlu menjalankan perintah berikut:

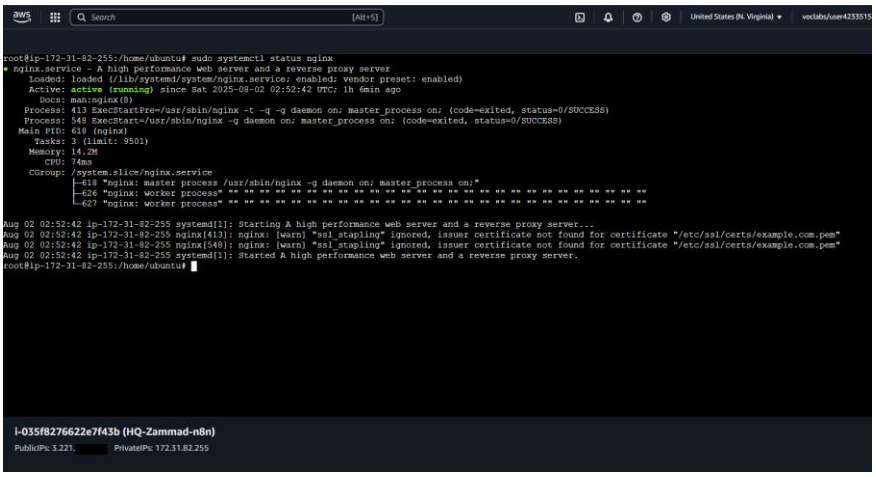
```
zammad run rails r "Setting.set('es_url',
'http://localhost:9200')"
```

2) Untuk web server-nya, digunakan NGINX karena lebih ringan, modern, dan umum digunakan dalam implementasi Zammad. Selain itu, konfigurasi untuk proxy maupun SSL-nya juga lebih simpel dan fleksibel. Langsung install NGINX dengan perintah berikut:

```
sudo apt install nginx -y
```

3) Setelah itu aktifkan dan jalankan servicenya dan cek apakah nginx sudah berjalan dan tidak terdapat masalah.

```
sudo systemctl enable nginx
sudo systemctl start nginx
sudo systemctl status nginx
```



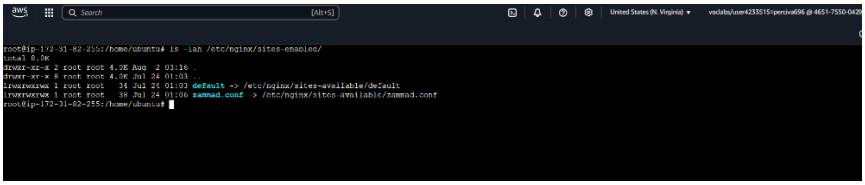
Gambar 4. 46 cek status nginx

4) Salin konfigurasi default dari Zammad agar kita bisa langsung punya struktur file zammad.conf yang sudah disiapkan oleh developer.

```
cp /opt/zammad/contrib/nginx/zammad_ssl.conf
/etc/nginx/sites-available/zammad.conf
```

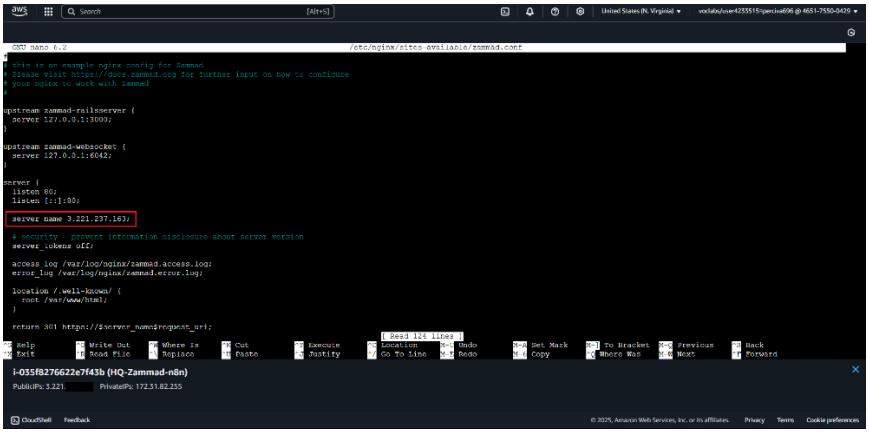
5) Cek apakah file konfigurasi sudah muncul di folder sites-enabled

```
ls -lah /etc/nginx/sites-enabled/
```



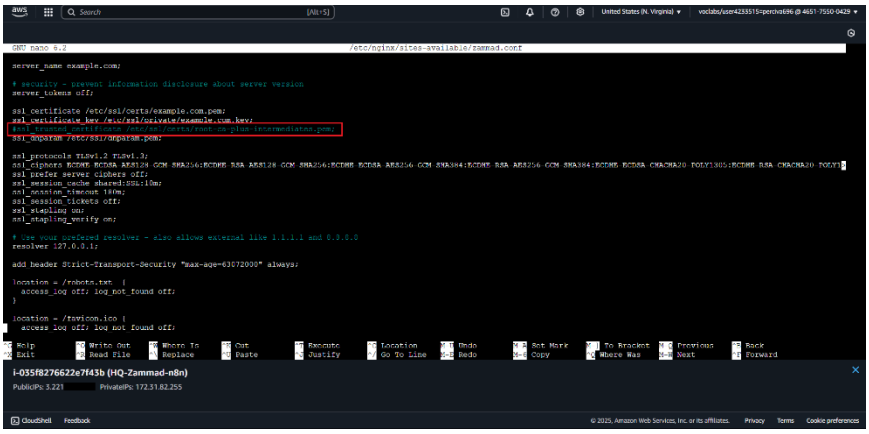
Gambar 4. 47 cek file konfigurasi pada sites-enabled

6) Edit file konfigurasi zammad.conf untuk menyesuaikan bagian server_name dengan IP dari instance kita, agar layanan bisa diakses dengan benar dengan menjalankan perintah **`nano /etc/nginx/sites-available/zammad.conf`**



Gambar 4. 48 set server name

7) Masih dalam file yang sama, cari bagian konfigurasi sertifikat SSL dan komentari baris trusted certificate karena tidak wajib.



Gambar 4. 49 konfigurasi trusted certificate

8) Karena tidak menggunakan SSL dari Let's Encrypt, maka kita perlu buat sertifikat **self-signed** sebagai pengganti untuk keperluan testing ini, kita akan menyesuaikan dengan nama file yang sudah ada pada file configurasi agar lebih mudah dan sederhana, untuk membuat self-signed certificate jalankan perintah berikut:

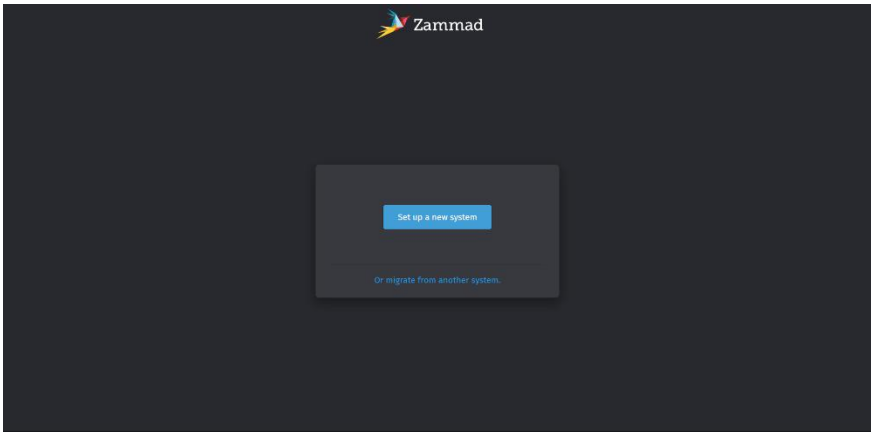
```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048
\
  -keyout /etc/ssl/private/example.com.key \
  -out /etc/ssl/certs/example.com.pem
```

9) Setelah itu buat file dhparam.pem yang digunakan untuk memperkuat keamanan SSL handshake pada NGINX:

```
sudo openssl dhparam -out /etc/ssl/dhparam.pem 2048
```

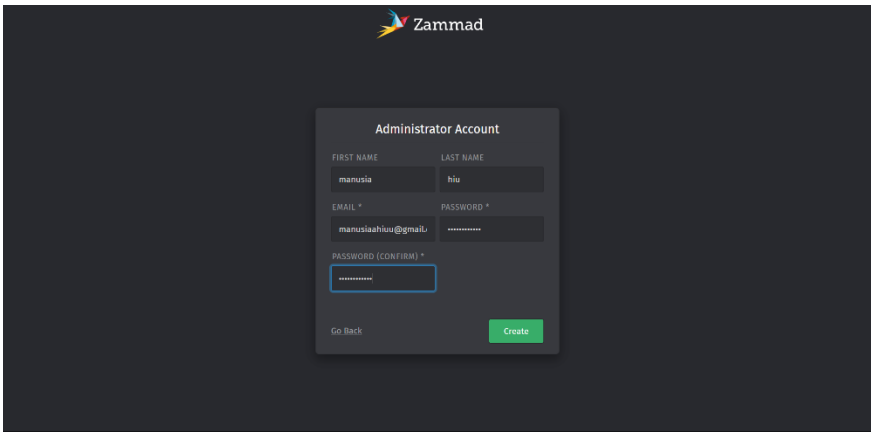
10) Terakhir, lakukan pengecekan ulang pada konfigurasi nginx dengan menjalankan perintah **`sudo nginx -t`**, Jika tidak ada error (*syntax is ok, test is successful*), lanjutkan dengan restart service agar konfigurasi terbaru diterapkan dan dijalankan.

11) Setelahnya kita dapat mengakses zammad pada halaman browser dengan memasukan alamt https://instanceip, Dimana disini adalah https://3.221.xx.xx dan akan muncul tampilan sebagai berikut.



Gambar 4. 50 satup awal zammad

12) Setelahnya akan diminta untuk membuat akun administrator dan zammad bisa digunakan untuk keperluan infrastruktur kita.



Gambar 4. 51 buat akun administrator zammad

4.2.8 N8N

a) Installasi

1) Hal pertama yang dilakukan adalah memperbarui repository sistem dan menginstal beberapa dependensi dasar seperti apt-transport-https, ca-certificates, curl, dan software-properties-common agar sistem siap menjalankan container.

```
sudo apt update && sudo apt install -y apt-transport-
https ca-certificates curl software-properties-common
```

2) Buat directory n8n server pada directory kerja saat ini.
3) Setelah itu, buat direktori baru bernama n8n-data yang nantinya digunakan sebagai volume penyimpanan data n8n. Kemudian ubah kepemilikan folder tersebut agar dapat digunakan oleh user dalam container n8n.

```
mkdir n8n-data
sudo chown -R 1000:1000 n8n-data/
```

4) Selanjutnya, atur permission folder n8n-data menjadi 755 agar bisa diakses dengan baik oleh container saat dijalankan.

```
sudo chmod -R 755 n8n-data/
```

5) Buat file .env di direktori kerja untuk menyimpan variabel lingkungan yang akan digunakan oleh container. File ini digunakan untuk mengatur autentikasi dasar, IP host, port, dan konfigurasi cookie.

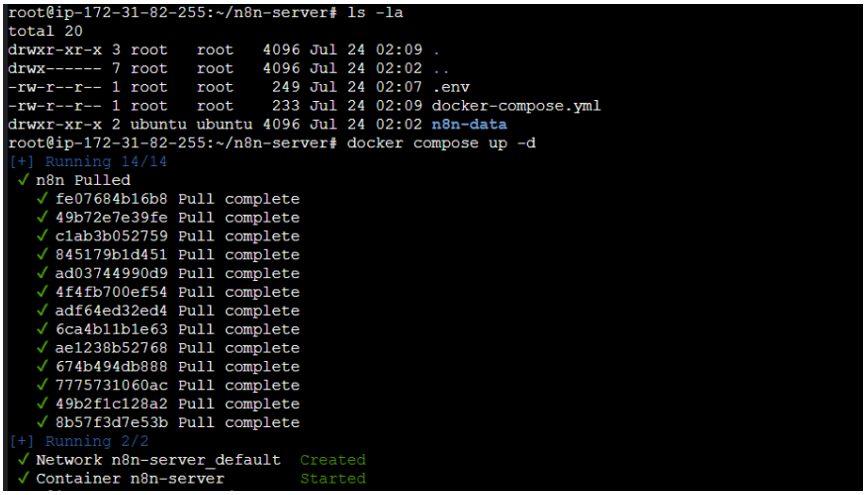
```
N8N_BASIC_AUTH_ACTIVE=true
N8N_BASIC_AUTH_USER=admin
N8N_BASIC_AUTH_PASSWORD=#Password
N8N_HOST=#machineIP
N8N_PORT=5678
N8N_PROTOCOL=http
N8N_SECURE_COOKIE=false  # Temporarily disable secure
cookie to avoid HTTP access issues
NODE_ENV=production
```

6) Setelah itu, buat file docker-compose.yml yang berisi konfigurasi container untuk menjalankan n8n. Konfigurasi ini akan menghubungkan container dengan file .env dan volume penyimpanan n8n-data.

```
services:
  n8n:
    image: n8nio/n8n
    restart: always
    container_name: n8n-server
    ports:
      - "5678:5678"
    env_file:
      - .env
    volumes:
      - ./n8n-data:/home/node/.n8n
```

7) Setelah semuanya siap, jalankan container n8n dengan perintah berikut agar layanan langsung berjalan di background.

```
docker compose up -d
```



Gambar 4. 52 proses docker compose

8) Untuk memastikan bahwa container sudah berjalan, lakukan pengecekan dengan perintah berikut

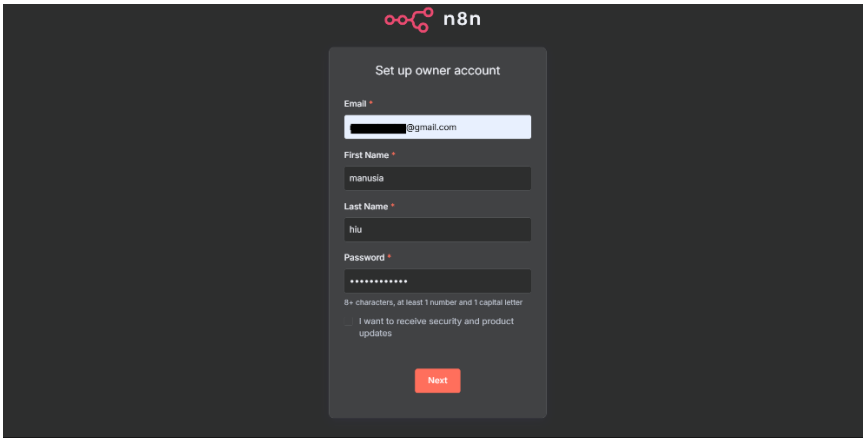
```
docker ps -a
```



Gambar 4. 53 cek status container

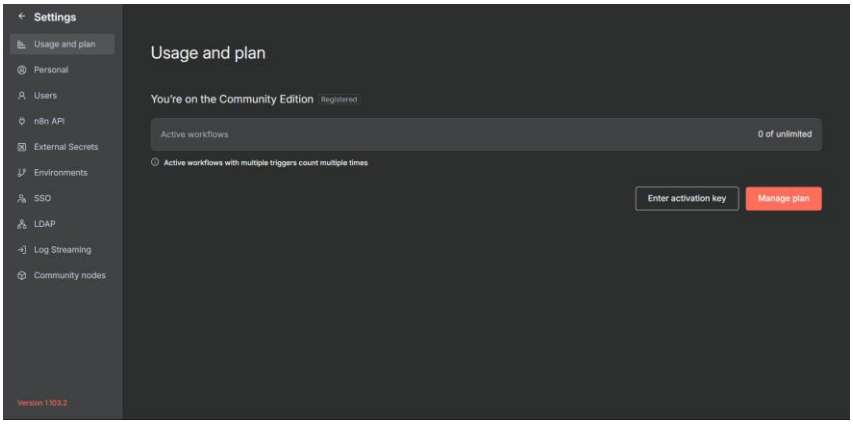
b) Konfigurasi

1) Daftar meggunakan alamat email yang diinginkan, kemudian atur password untuk akun



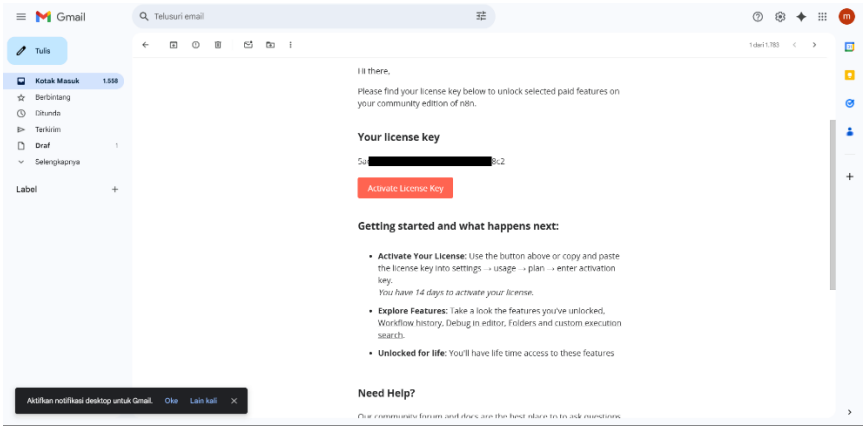
Gambar 4. 54 buat akun owner n8n

2) N8n sendiri membutuhkan lisence key yang dapat di request pada halaman berikut agar bisa membuat workflow nantinya



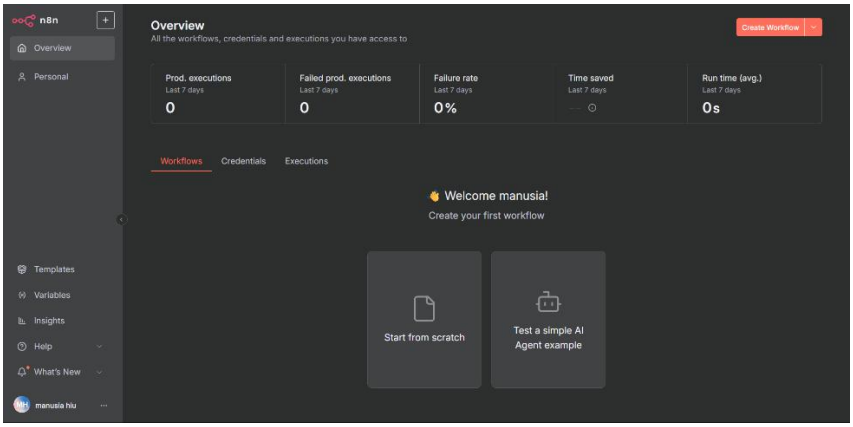
Gambar 4. 55 mengaktifkan lisence key n8n

3) Berikut adalah contoh lisence key yang akan dikirim ke Alamat email yang didaftarkan saat melakukan pembuatan owner account



Gambar 4. 56 n8n lisence key

4) N8n sudah siap digunakan untuk membuat workflow automation nantinya



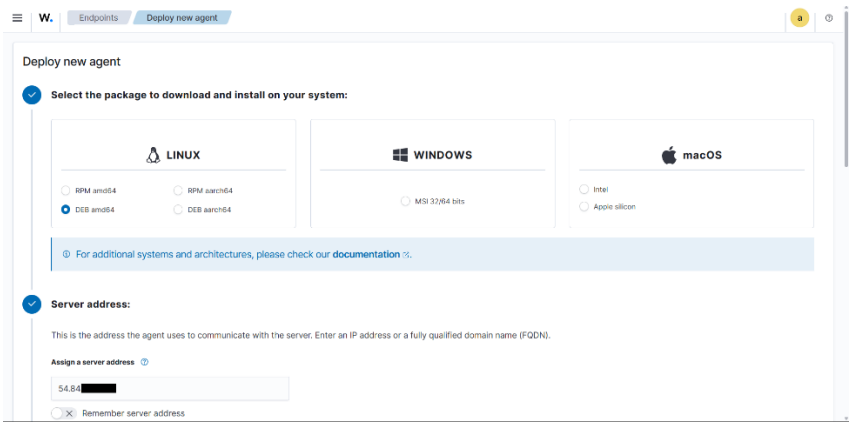
Gambar 4. 57 halaman utama n8n

4.2.9 Wazuh Agent

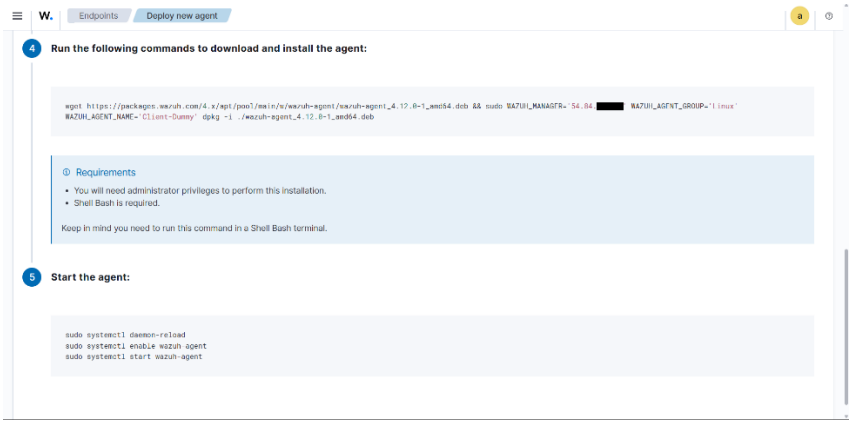
a) Installasi

Installasi untuk Wazuh Agent sendiri cukup mudah, karena perintah instalasinya bisa langsung didapat dari dashboard Wazuh Manager. Prosesnya dilakukan dengan cara masuk ke menu *Deploy Agent*, kemudian mengisi form yang diperlukan. Setelah itu, Wazuh akan secara otomatis menghasilkan command yang sesuai, sehingga cukup disalin dan dijalankan di mesin target tanpa perlu konfigurasi manual yang rumit. Instalasi ini dilakukan pada instance *Client-dummy*. Langkah-langkahnya adalah sebagai berikut:

1) Pada Wazuh Manager, masuk ke bagian *Deploy Agent*, kemudian isi setiap form yang dibutuhkan seperti nama agen, grup agen, serta alamat IP dari Wazuh Server.

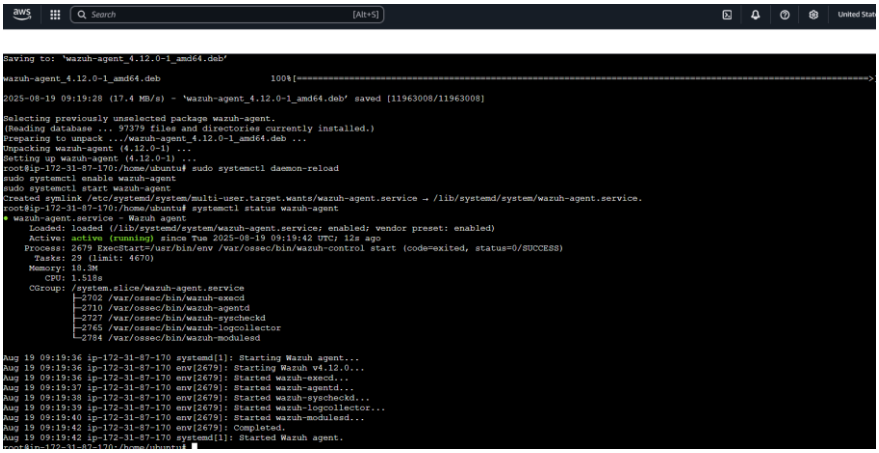


Gambar 4. 58 wazuh agent installation assistant 1



Gambar 4. 59 wazuh agent installation assistant 2

2) Setelah form diisi, Wazuh akan secara otomatis menghasilkan perintah instalasi. Salin perintah tersebut dan jalankan pada mesin di mana Wazuh Agent akan dipasang, dalam hal ini adalah *Client-dummy*.
3) Tunggu sampai proses installasi selesai kemudian konfirmasi apakah agent sudah berjalan menggunakan perintah `sudo systemctl status wazuh-agent`.



Gambar 4. 60 wazuh agent status

4.2.10 Zabbix Agent

a) Installasi

Instalasi Zabbix Agent bertujuan agar server pusat Zabbix dapat melakukan monitoring terhadap server target atau klien. Pada implementasi ini, Zabbix Agent akan dipasang pada instance *Client-dummy*. Prosesnya cukup sederhana dan dilakukan melalui beberapa langkah berikut:

1) Hal pertama yang perlu dilakukan adalah memperbarui sistem operasi pada instance client agar semua paket berada pada versi terbaru. Perintah yang dijalankan adalah:

```
sudo apt update
sudo apt upgrade
```

2) Setelah proses update selesai, langkah berikutnya adalah menambahkan repository Zabbix ke dalam sistem agar kita bisa menginstal agent-nya dari sumber resmi. Jalankan perintah berikut:

```
wget
https://repo.zabbix.com/zabbix/6.0/ubuntu/pool/main/z/
zabbix-release/zabbix-release_6.0-
4+ubuntu22.04_all.deb
sudo dpkg -i zabbix-release_6.0-4+ubuntu22.04_all.deb
sudo apt update
```

3) Setelah repository berhasil ditambahkan, install Zabbix Agent dengan menggunakan perintah berikut:

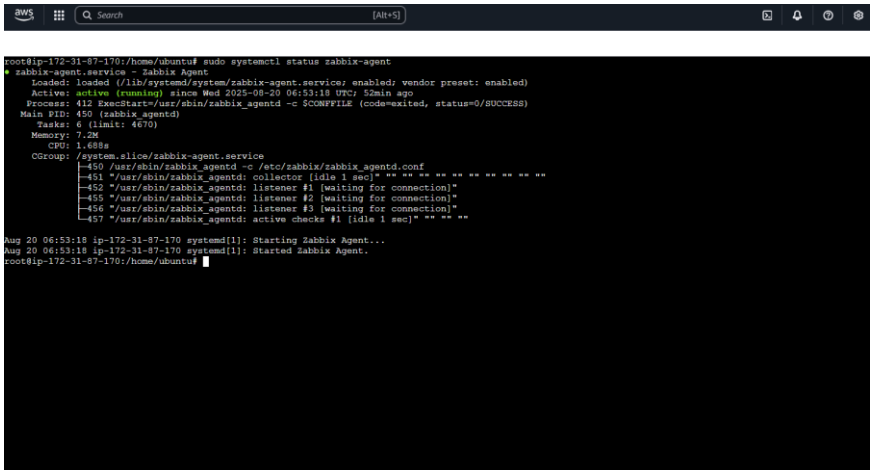
```
sudo apt install zabbix-agent
```

b) Konfigurasi

1) Setelah Zabbix Agent berhasil terinstal, tahap selanjutnya adalah melakukan konfigurasi agar agent dapat terhubung ke server pusat. File konfigurasi dapat ditemukan di path /etc/zabbix/zabbix_agentd.conf, untuk melakukan modifikasi gunakan perintah berikut:

```
sudo nano /etc/zabbix/zabbix_agentd.conf
```

2) Temukan baris konfigurasi Server dan Hostname. Ganti nilainya sesuai dengan pengaturan jaringan yang digunakan. Pada kasus ini, IP dari server pusat Zabbix adalah 98.84.xx.xx.
3) Jika konfigurasi sudah disesuaikan, simpan file dan keluar dari editor yang digunakan, Dimana pada langkah ini adalah nano dengan menekan CTRL+S kemudian CTRL+X.
4) Jalankan Zabbix Agent dengan perintah berikut, lalu aktifkan agar otomatis berjalan saat server di-restart:
5) Untuk memastikan bahwa Zabbix agent sudah berjalan dengan benar, cek status service dengan perintah **`sudo systemctl status zabbix-agent.`**



Gambar 4. 61 zabbix agent status

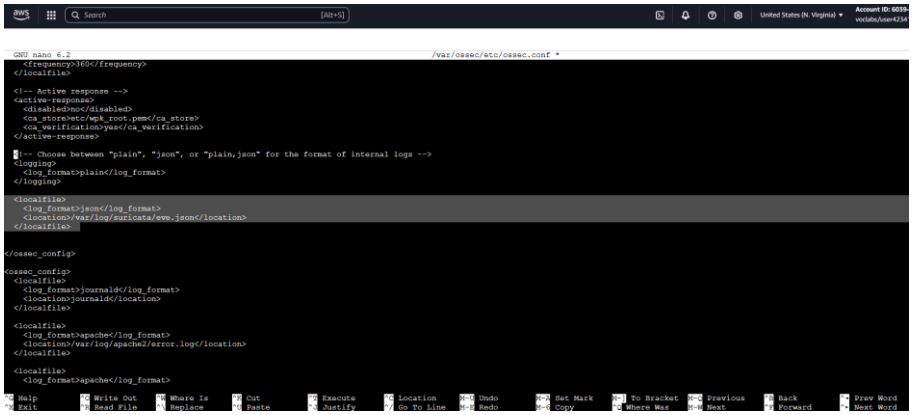
4.3 Integrasi Antar Komponen

4.3.1 Integrasi Wazuh Dan Suricata

Tujuan dari dilakukan integrasi antara Wazuh dan Suricata adalah agar setiap alert yang dihasilkan oleh Suricata dapat muncul pada modul *Threat Hunting* pada Wazuh Dashboard. Dengan begitu, proses monitoring terhadap aktivitas mencurigakan dapat dilakukan secara terpusat melalui satu platform. Proses ini dilakukan pada dua instance, yaitu HQ-Wazuh-Suricata sebagai server dan Client-Dummy sebagai agen.

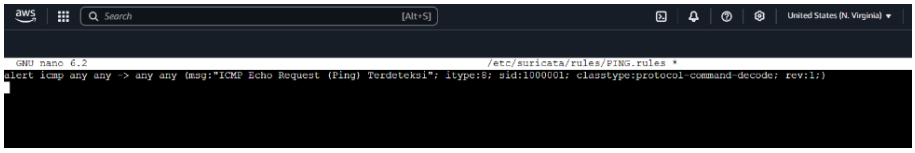
1) Langkah pertama yang dilakukan adalah memastikan bahwa log atau alert dari Suricata bisa terbaca oleh Wazuh. Untuk itu, perlu menambahkan konfigurasi baru berupa localfile pada file /var/ossec/etc/ossec.conf. Penambahan ini memungkinkan Wazuh untuk membaca file log JSON dari Suricata yang berada di /var/log/suricata/eve.json, dengan konfigurasi sebagai berikut:

```
<ossec_config>
  <localfile>
    <log_format>json</log_format>
    <location>/var/log/suricata/eve.json</location>
  </localfile>
</ossec_config>
```



Gambar 4. 62 set ossec.conf

2) Setelah konfigurasi ditambahkan, langkah selanjutnya adalah me-restart service Wazuh agar perubahan konfigurasi bisa diterapkan. Karena *HQ-Wazuh-Suricata* berperan sebagai server, maka perintah yang dijalankan adalah **`systemctl restart wazuh-manager`**. Sedangkan untuk *Client-Dummy* sebagai agen, perintahnya adalah **`systemctl restart wazuh-agent`**.
3) Untuk menguji apakah integrasi sudah berjalan, dibuat sebuah rule Suricata sederhana yang dapat mendeteksi aktivitas ping. Rule ini disimpan dalam file baru bernama PING.rules yang dibuat di direktori /etc/suricata/rules/.



Gambar 4. 63 suricata custom ping rules

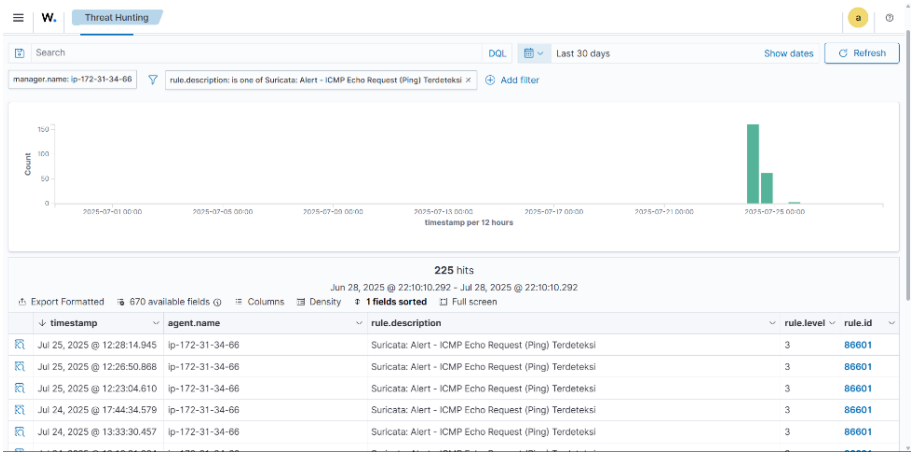
4) Isi dari rule tersebut adalah sebagai berikut:

```
alert icmp any any -> any any (msg:"ICMP Echo Request (Ping) Terdeteksi"; itype:8; sid:1000001; classtype:protocol-command-decode; rev:1;)
```

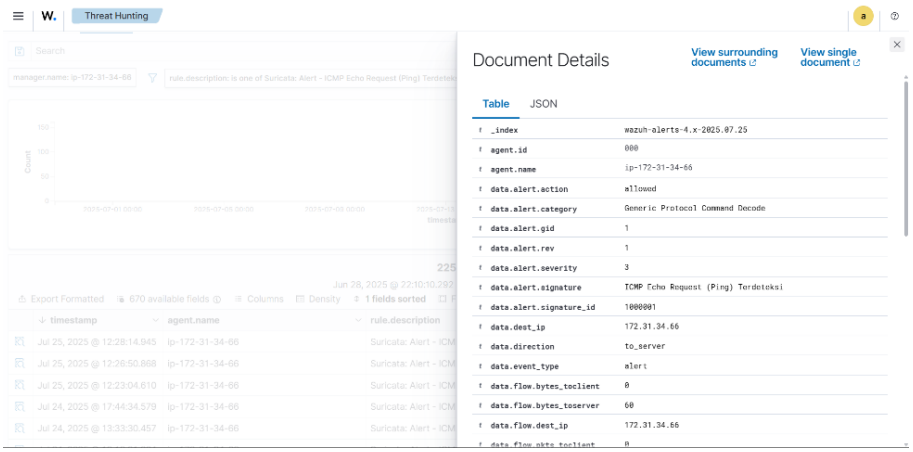
Rule ini bertujuan untuk mengeluarkan alert setiap kali sistem mendeteksi adanya paket ICMP Echo Request (ping), baik dari maupun

ke mesin tersebut. Hal ini bisa digunakan untuk memantau aktivitas pemindaian jaringan maupun koneksi dasar.

5) Selanjutnya dilakukan pengujian dengan cara menjalankan perintah ping dari mesin lain menuju instance *HQ-Wazuh-Suricata* yang telah diinstal Suricata sebelumnya.
6) Setelah ping dilakukan, dilakukan pengecekan melalui dashboard Wazuh pada modul *Threat Hunting*. Hasilnya menunjukkan bahwa alert dari Suricata berhasil ditangkap dan ditampilkan di dalam Wazuh, yang berarti integrasi berjalan dengan baik dan proses monitoring sudah terpusat.



Gambar 4. 64 suricata alert pada wazuh dahsboard



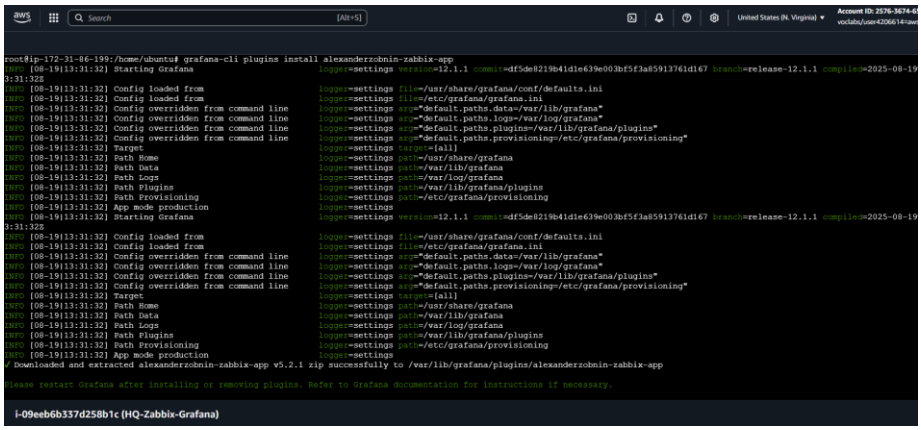
Gambar 4. 65 detail suricata alert

4.3.2 Integrasi Zabbix Dan Grafana

Integrasi Zabbix dan Grafana bertujuan menghadirkan tampilan monitoring yang lebih komprehensif, interaktif, dan fleksibel dibanding dashboard bawaan Zabbix. Dengan Grafana, data dapat divisualisasikan secara dinamis dalam bentuk grafik dan panel interaktif, sehingga analisis utilisasi menjadi lebih cepat dan informatif, serta memudahkan pembuatan dashboard tematik sesuai kebutuhan tanpa mengganggu data yang ada.

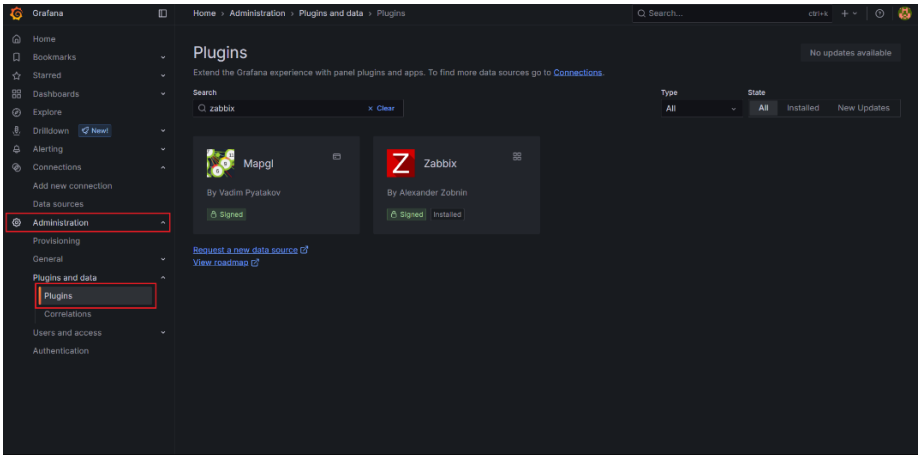
1) Pada instance HQ-Zabbix-Grafana, langkah pertama yang perlu dilakukan adalah menginstall plugin “Zabbix Plugin for Grafana”. Plugin ini berfungsi sebagai jembatan agar Grafana dapat membaca dan menampilkan data dari Zabbix, Proses instalasi ini akan menggunakan perintah Grafana CLI dengan mengetikkan pada terminal.

```
grafana-cli plugins install alexanderzobnin-zabbix-app
```



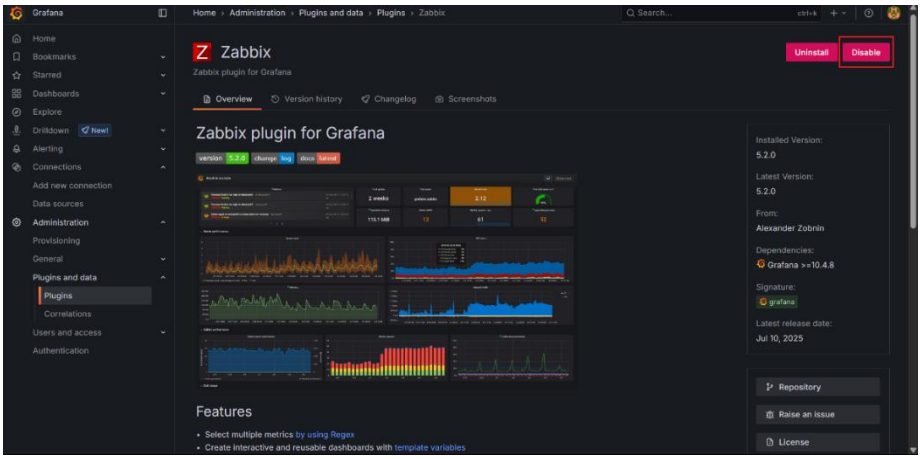
Gambar 4. 66 install Zabbix plugin

2) Setelah plugin berhasil terinstal, masuk ke UI Grafana untuk mengaktifkan plugin sekaligus melakukan koneksi awal dengan Zabbix. Pada menu Grafana, klik Administrator kemudian pilih Plugins, lalu pada kolom pencarian ketikkan “zabbix” untuk memfilter plugin yang kita butuhkan.



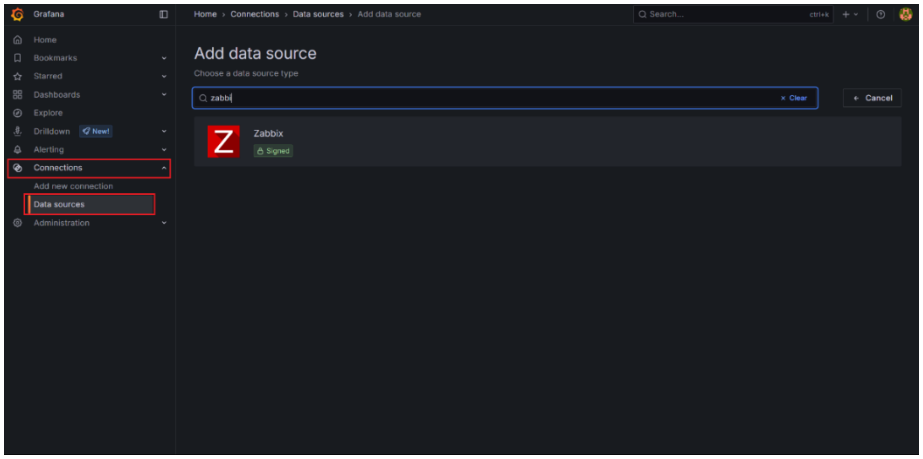
Gambar 4. 67 cari plugin Zabbix pada grafana UI

3) Klik pada plugin Zabbix tersebut untuk masuk ke halaman detailnya, lalu aktifkan plugin dengan menekan tombol Enable.



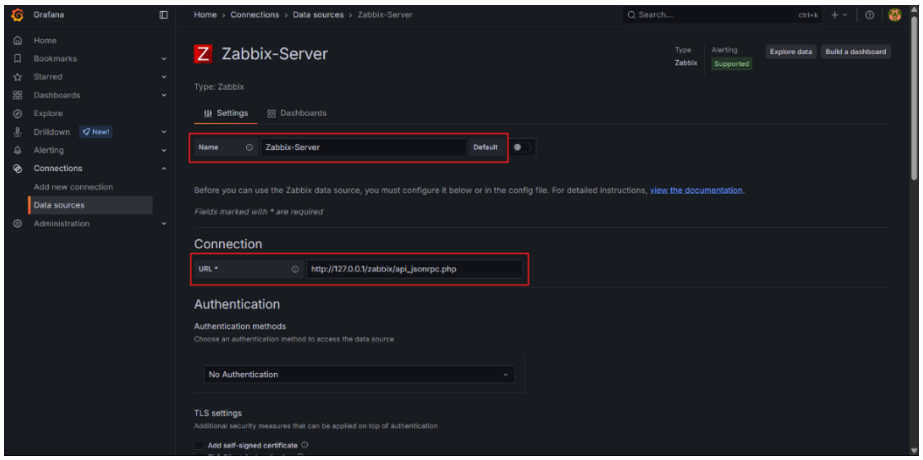
Gambar 4. 68 Enable Grafana plugin

4) Selanjutnya masuk ke bagian Connections, kemudian klik Data Sources untuk menambahkan sumber data baru. Langkah ini bertujuan agar Grafana dapat terhubung langsung dengan server Zabbix yang sudah kita siapkan.



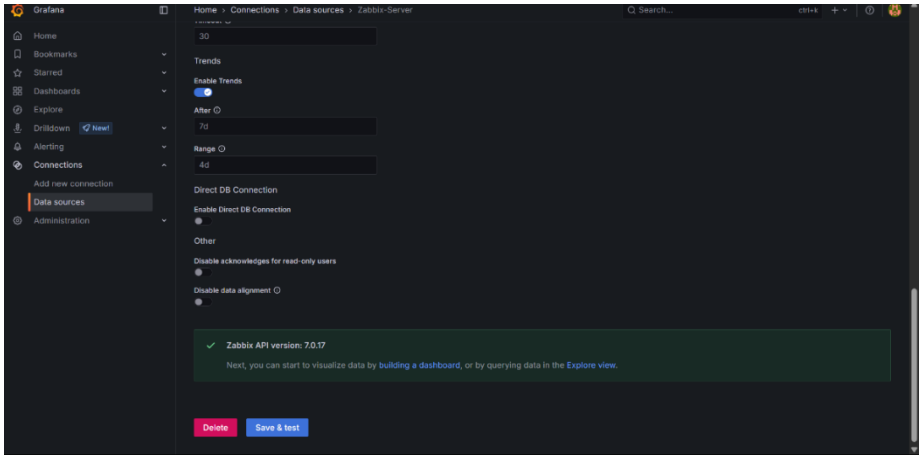
Gambar 4. 69 menambahkan datasource zabbix

5) Isikan informasi koneksi seperti Name sesuai keinginan, kemudian pada kolom URL masukkan **`http://127.0.0.1/zabbix/api_jsonrpc.php`** (alamat ini digunakan untuk mengakses API Zabbix secara lokal). Masukkan username dan password sesuai dengan kredensial admin Zabbix yang digunakan.



Gambar 4. 70 set data source

6) Setelah semua data koneksi terisi, klik tombol Save & Test untuk memverifikasi koneksi. Jika konfigurasi benar, akan muncul notifikasi bahwa koneksi berhasil.



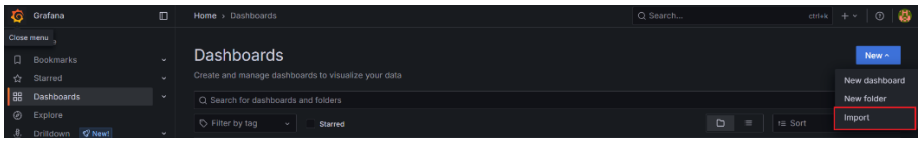
Gambar 4. 71 test data source

7) Langkah berikutnya adalah membuat dashboard baru. Pada tahap ini, kita akan menggunakan template bawaan dengan nama [Zabbix - Full Server Status], agar tampilan lebih informatif dan siap digunakan tanpa perlu membangun dari nol. Kemudian salin Dashboard



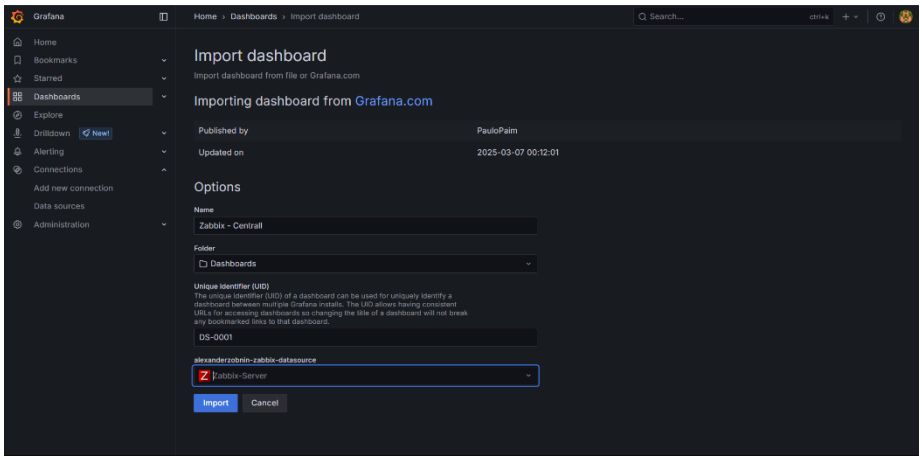
Gambar 4. 72 grafana template zabbix dashboard

8) Pada UI Grafana, masuk ke halaman Dashboard, lalu klik New dan pilih Import kemudian paste kan id dari template dashboard yang sudah didapat sebelumnya.



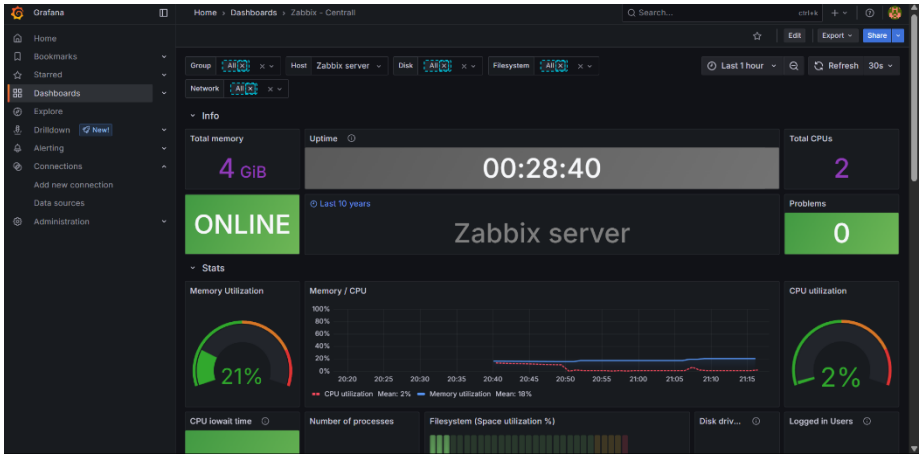
Gambar 4. 73 import dashboard

9) Pada halaman import dashboard berikan nama dashboard yang akan dibuat, Masukkan Dashboard UID sesuai keinginan, kemudian pada bagian Data Source pilih data source Zabbix yang sudah kita tambahkan sebelumnya, setelah semua data telah dilengkapi klik import.



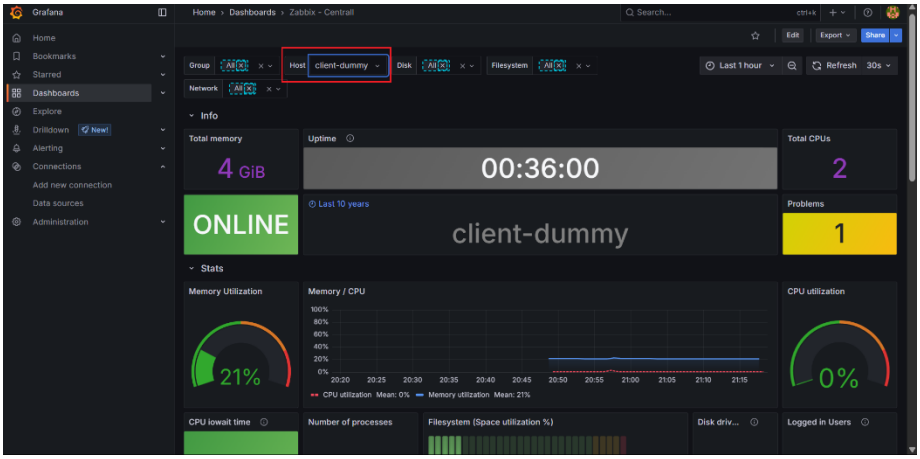
Gambar 4. 74 setup dashboard

10) Dashboard akan otomatis terbuat, dan jika berhasil, akan langsung menampilkan berbagai panel monitoring dari Zabbix.



Gambar 4. 75 Grafana - zabbix dashboard

11) Kita juga dapat menyesuaikan tampilan dashboard untuk server HQ maupun client dummy. Caranya adalah dengan mengklik pada bagian **Host** lalu memilih host yang ingin ditampilkan datanya.



Gambar 4. 76 client-dummy dashboard

Dengan adanya integrasi ini, proses monitoring tidak hanya menjadi lebih informatif dan efisien, tetapi juga memberikan fleksibilitas visual yang membantu admin mengambil keputusan dengan lebih cepat dan tepat.

4.3.3 Automate Alert Wazuh Ke Thehive

Tujuan dari dilakukannya integrasi ini adalah agar wazuh mengirimkan semua alert ke the hive, dimana nantinya alert dengan severity HIGH ataulebih bisa dijadikan sebuah case untuk ditindak lanjuti dan di Analisa lebih mendalam, Langkah integrasinya yaitu:

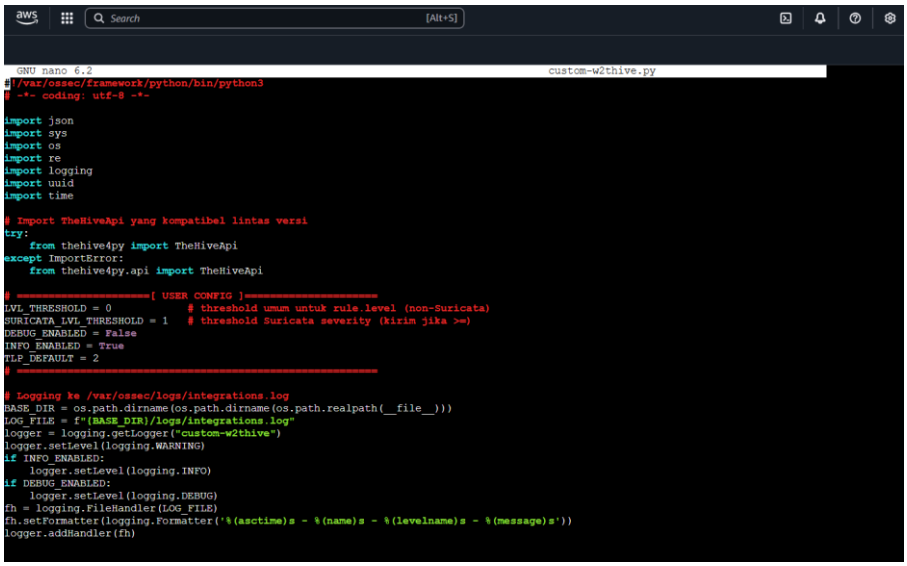
1) Langkah pertama saya melakukan update package dan instalasi dependensi utama yang diperlukan, yaitu Python3. Hal ini dilakukan agar environment di dalam Wazuh siap digunakan untuk menjalankan integrasi ke TheHive dengan perintah sudo yum update lalu sudo yum install python3.

```
sudo apt update
sudo apt install python3
```

2) Setelah itu saya melanjutkan dengan instalasi modul Python khusus untuk TheHive, yaitu thehive4py. Instalasi dilakukan langsung pada environment Python yang ada di dalam Wazuh dengan perintah **sudo /var/ossec/framework/python/bin/pip3 install**

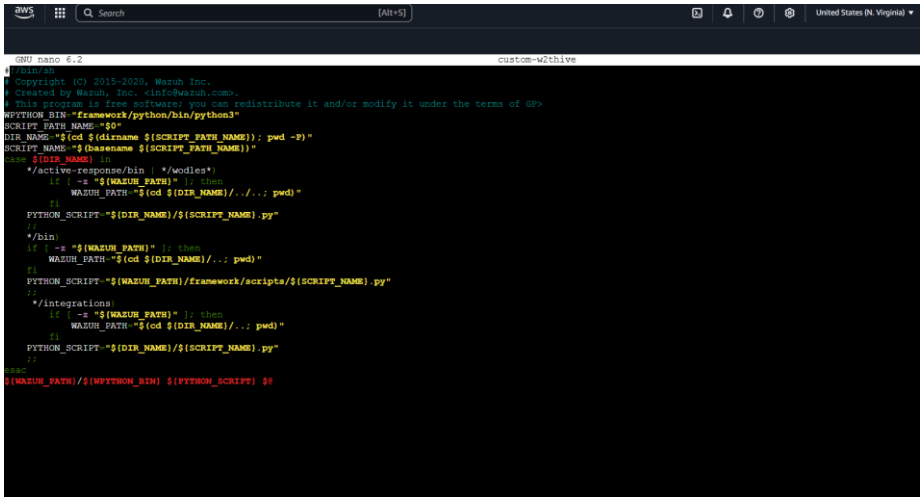
thehive4py. Modul ini berfungsi sebagai jembatan komunikasi antara Wazuh dengan API TheHive sehingga setiap alert bisa dikirim otomatis.

3) Kemudian buat custom integration script dengan nama custom-w2thive.py yang ditempatkan di direktori /var/ossec/integrations/. Isi script ini sudah mengatur agar semua alert yang levelnya di atas threshold tertentu bisa dikirimkan ke TheHive. Script bisa dilihat pada Wazuh-Thehive-Integration.



Gambar 4. 77 custom-w2thive.py

4) Supaya script Python yang sudah dibuat tadi bisa dieksekusi oleh Wazuh, perlu dibuat sebuah bash script dengan nama custom-w2thive yang diletakkan di path /var/ossec/integrations/custom-w2thive. File bash ini bertugas memanggil file Python yang sudah dibuat sebelumnya sehingga integrasi dapat berjalan sesuai alur. Script bisa dilihat pada Wazuh-Thehive-Integration.



Gambar 4. 78 custom-w2thive

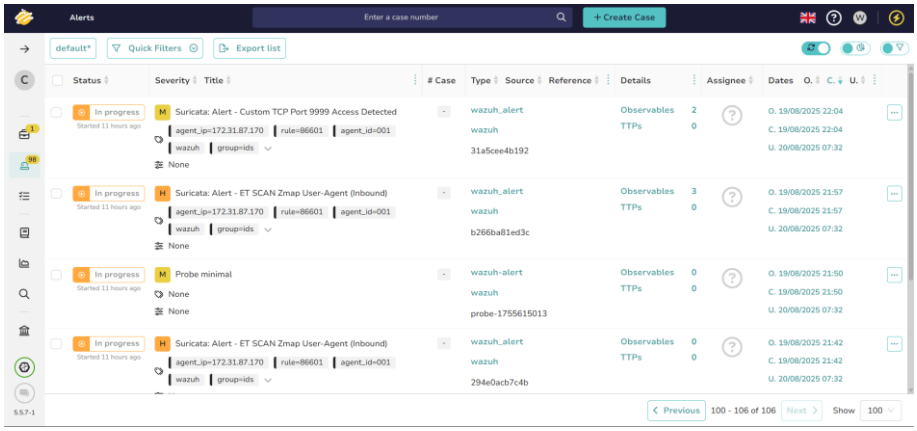
5) Setelah kedua script selesai dibuat atur permission dan ownership agar Wazuh memiliki hak akses penuh untuk menjalankannya. Tahap ini penting karena tanpa permission yang benar, integrasi tidak akan bisa dieksekusi dengan baik.

```
sudo chmod 755 /var/ossec/integrations/custom-w2thive.py
sudo chmod 755 /var/ossec/integrations/custom-w2thive
sudo chown root:wazuh /var/ossec/integrations/custom-w2thive.py
sudo chown root:wazuh /var/ossec/integrations/custom-w2thive
```

6) Langkah terakhir yaitu mengaktifkan integrasi langsung pada konfigurasi Wazuh Manager dengan cara mengedit file /var/ossec/etc/ossec.conf. Di dalam file konfigurasi tersebut saya menambahkan konfigurasi integrasi baru yang berisi

```
<integration>
  <name>custom-w2thive</name>
  <hook_url>http://TheHive_Server_IP:9000</hook_url>
  <api_key>API-KEY user thehive</api_key>
  <alert_format>json</alert_format>
</integration>
```

7) Setelah konfigurasi selesai dimasukkan, saya melakukan restart pada service Wazuh Manager menggunakan perintah sudo **`systemctl restart wazuh-manager`**. Restart ini diperlukan supaya perubahan konfigurasi terbaca dan integrasi antara Wazuh dengan TheHive bisa berjalan sebagaimana mestinya.
8) Terlihat alert wazuh sudah berhasil dikirim ke thehive



Gambar 4. 79 alert wazuh ke thehive

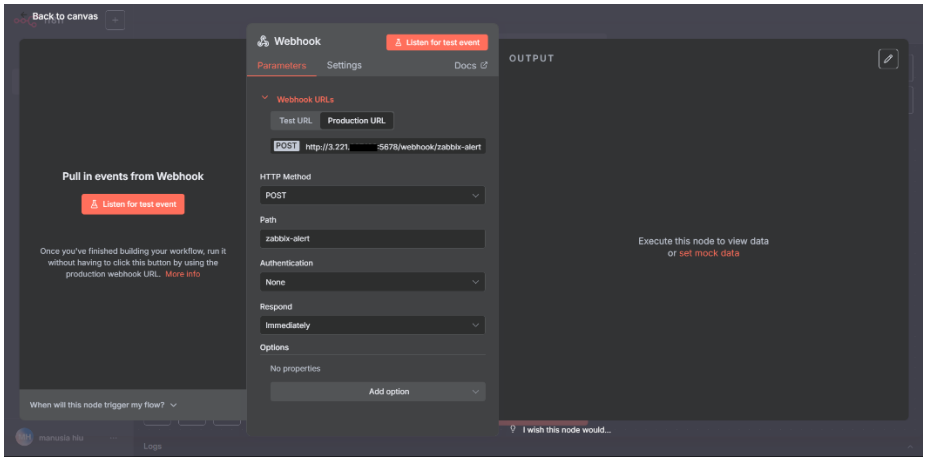
4.3.4 Utilization Alert & Ticket Automation

Pada tahap ini dilakukan integrasi antara Zabbix dengan sistem komunikasi dan pelaporan insiden, yaitu Telegram dan Zammad, menggunakan workflow otomatis dari n8n. Tujuan dari integrasi ini adalah agar setiap alert yang muncul di Zabbix dapat secara otomatis dikirimkan sebagai notifikasi ke Telegram dan juga dibuatkan tiket secara langsung ke Zammad. Dengan begitu, tim SOC ataupun klien bisa mendapatkan notifikasi secara real-time dan segera melakukan tindakan terhadap insiden yang terdeteksi. Berikut Adalah tahapan dalam integrasi komponen:

1) Masuk ke dashboard n8n melalui alamat IP publik instance HQ-Zammad-n8n menggunakan URL http://3.221.xx.xx:5678, lalu buat sebuah workflow baru dan beri nama bebas seperti Zabbix Alert Integration agar mudah dikenali.
2) Tambahkan node Webhook sebagai node pertama dalam workflow dan konfigurasikan sebagai berikut:

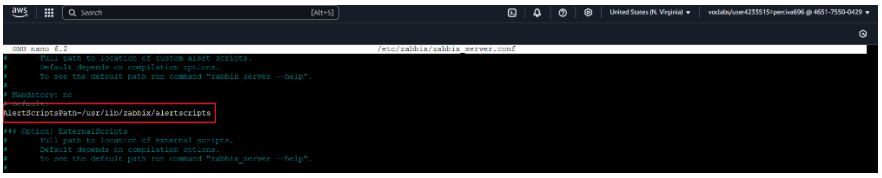
- HTTP Method POST
- URL: http://3.221.xx.xx:5678/webhook/zabbix-alert
- Path: zabbix-alert
- Authentication None
- Respond Immediately

Webhook ini akan menjadi titik masuk (endpoint) bagi data alert dari Zabbix dalam bentuk JSON.



Gambar 4. 80 node webhook

3) Buka konfigurasi Zabbix Server dengan mengedit file /etc/zabbix/zabbix_server.conf, pastikan baris AlertScriptsPath=/usr/lib/zabbix/alertscripts tidak dikomentari agar Zabbix dapat menjalankan script kustom sebagai metode pengiriman alert.



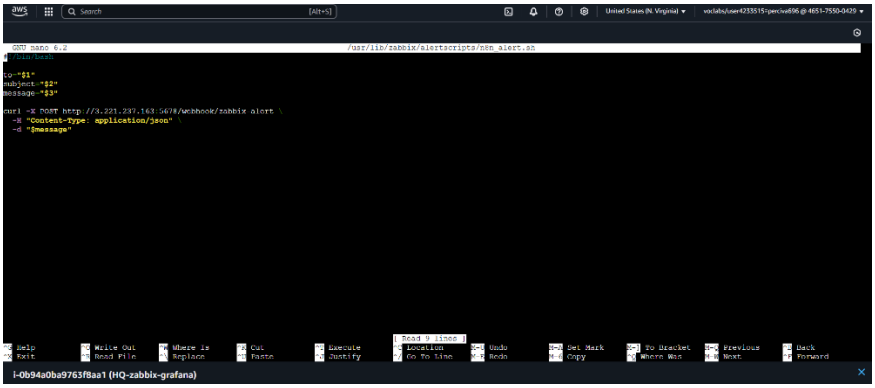
Gambar 4. 81 konfigurasi AlertScriptsPath

4) Setelah Buat file script baru bernama n8n_alert.sh di dalam direktori /usr/lib/zabbix/alertscripts dengan isi script sebagai berikut:

```
#!/bin/bash
```

```
to="$1"
subject="$2"
message="$3"

curl -X POST http://3.221.237.163:5678/webhook/zabbix-
alert \
  -H "Content-Type: application/json" \
  -d "$message"
```



Gambar 4. 82 n8n_alert.sh

Penjelasan dari script:

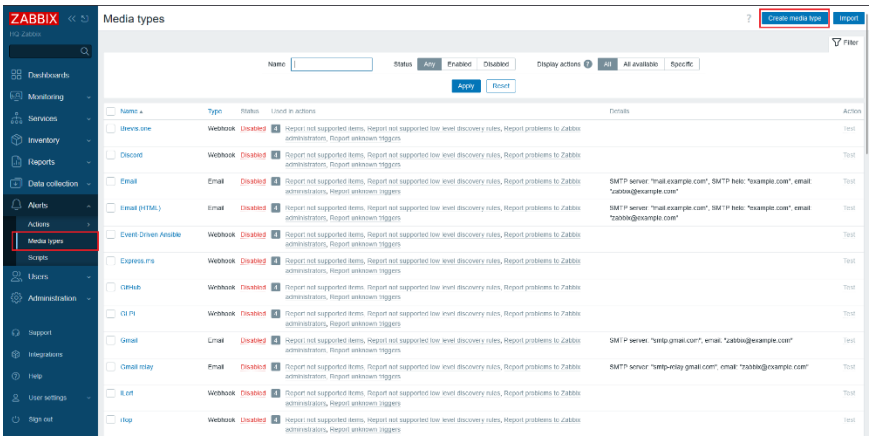
- $1, $2, $3 adalah parameter yang diterima dari Zabbix
- $1: to / alamat tujuan
- $2: subject / subjek pesan
- $3: message / isi utama berupa JSON, yang akan diteruskan ke webhook n8n.

Kesimpulannya Script ini akan menerima parameter dari Zabbix, yaitu tujuan, subjek, dan isi pesan, kemudian mengirimkan payload JSON dari parameter $message ke endpoint Webhook milik n8n menggunakan curl.

5) Ubah permission file script tersebut agar bisa dieksekusi dengan menjalankan perintah berikut:

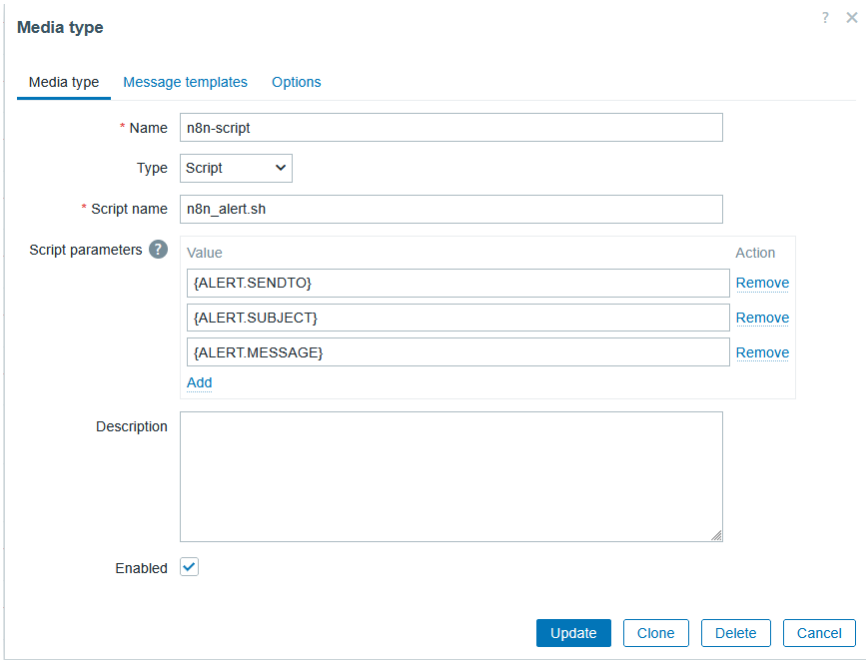
```
chmod +x /usr/lib/zabbix/alertscripts/n8n_alert.sh
```

6) restart Zabbix Server menggunakan perintah **`sudo systemctl restart zabbix-server`** agar konfigurasi dan script dapat dijalankan.
7) Pada antarmuka web Zabbix, buat sebuah media type baru, Dimana media type sendiri digunakan untuk Menentukan cara Zabbix mengirim notifikasi secara custom, contoh yang akan dipakai Adalah menggunakan script buatan sendiri. dengan cara klik menu alert kemudian media type, setelahnya klik create media type.



Gambar 4. 83 buat media type baru

8) Adapun hal yang perlu diatur pada media type yang dibuat adalah:
   - Name: n8n-script
   - Type: Script
   - Script name: n8n_alert.sh (pastikan sama dengan nama script yang telah dibuat sebelumnya)
   - Script parameters:
     - {ALERT.SENDTO} → parameter $1
     - {ALERT.SUBJECT} → parameter $2
     - {ALERT.MESSAGE} → parameter $3

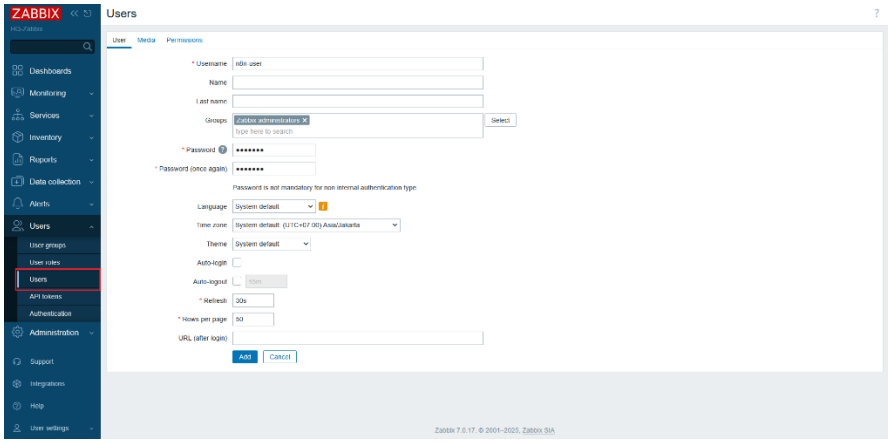


Gambar 4. 84 detail media type

Media type ini memberitahu Zabbix untuk menjalankan skrip dengan parameter yang sesuai ketika ada alert. Parameter ketiga ({ALERT.MESSAGE}) adalah kunci karena di sinilah JSON alert dikirim ke n8n

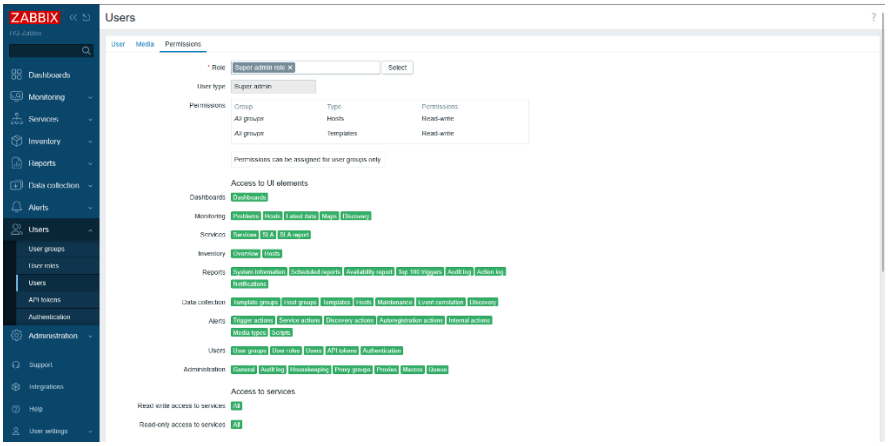
9) Setelahnya buat sebuah user yang akan digunakan untuk menerima dan memicu script media type yang mengirim alert ke n8n. untuk membuat sebuah user klik pada menu Users, kemudian Users dan create user. Pada tab user atur sebagi berikut:

- Username: n8n-user
- Group: Zabbix administrators
- Password: isi sesuai keinginan



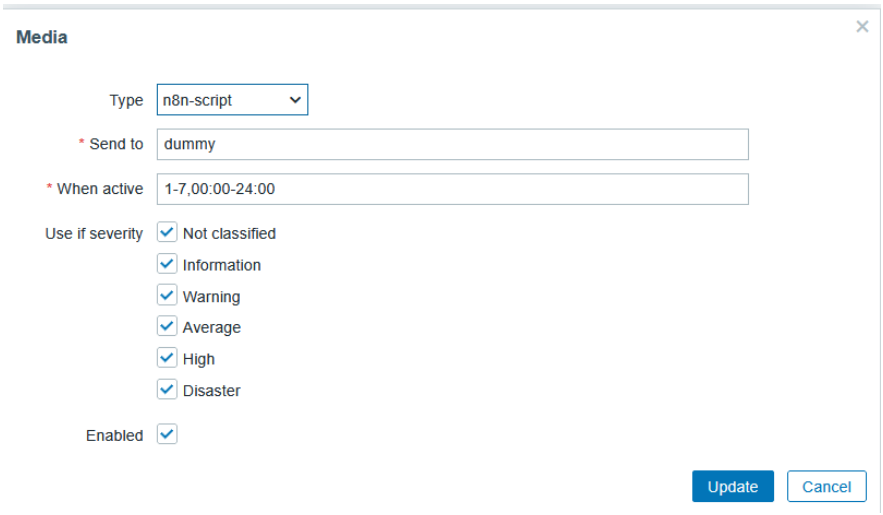
Gambar 4. 85 buat user baru

10) Selanjutnya klik pada tab permission, dan pada kolom role pilih Super admin role.



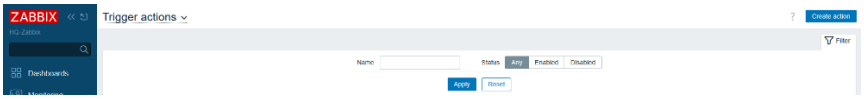
Gambar 4. 86 set permission untuk user

11) Dan pengaturan terakhir saat membuat user Adalah dengan menambahkan media type yang telah kita buat yaitu n8n-script ke user agar user bida mengakses media type tersebut. Untuk send to bisa diisi sesuai kebutuhan dan ceklist semua bagian pada bagian use if severity, setelahnya click update.



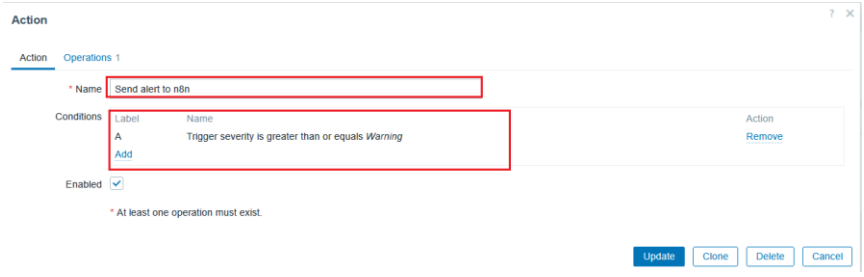
Gambar 4. 87 set media type user

12) Sebagai pemicu alert, buat sebuah action baru pada menu Configuration > Actions > Create Action.



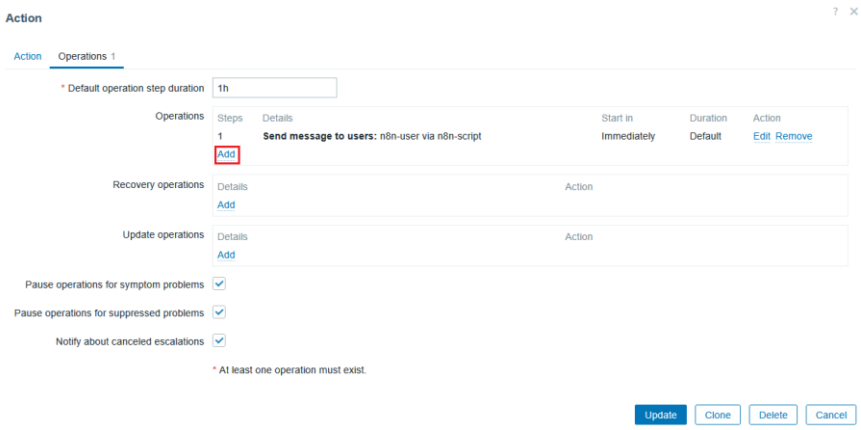
Gambar 4. 88 buat action baru

13) Pada tab action beri nama Send alert to n8n dan tambahkan sebuah kondisi trigger-nya adalah severity lebih besar atau sama dengan Warning.



Gambar 4. 89 buat condition baru

14) Kemudian lanjut ke tab Operation, Dimana jika belum ada klick add



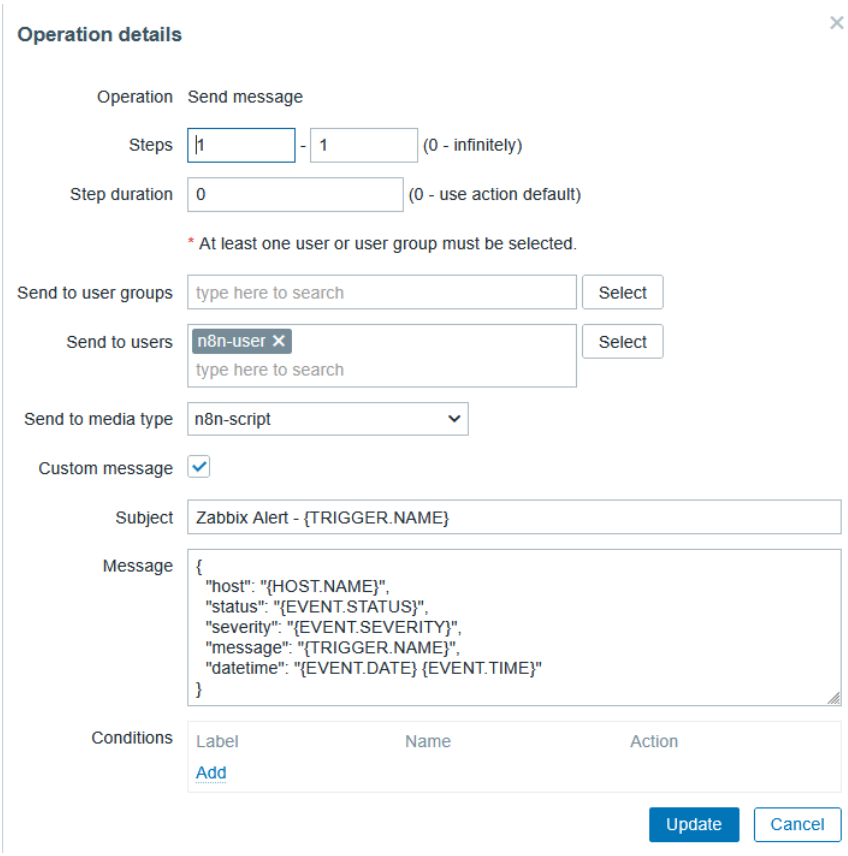
Gambar 4. 90 buat operation baru

15) Kemudian akan muncul menu operation details, Dimana bagian yang perlu diatur Adalah memastikan bahwa bagian berikut ini sesuai dengan:

- Send to Users: n8n-user (user yang telah dibuat sebelumnya)
- Send to Medi type: n8n-script (media type yang telah dibuat sebelumnya)
- Custom Message dalam keaadan aktif dengan pengaturan
    - Subject: Zabbix Alert - {TRIGGER.NAME}
    - Message:

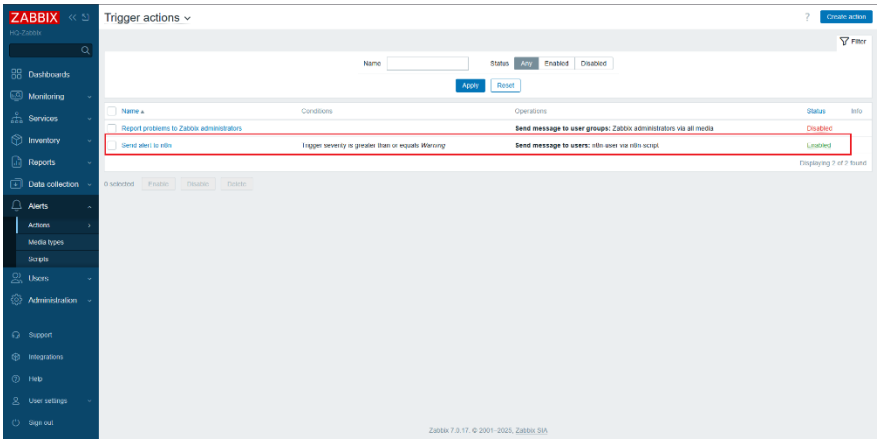
      {
      "host": "{HOST.NAME}",
      "status": "{EVENT.STATUS}",
      "severity": "{EVENT.SEVERITY}",
      "message": "{TRIGGER.NAME}",
      "datetime": "{EVENT.DATE} {EVENT.TIME}"
      }

      Nantinya Pesan JSON inilah yang akan diterima webhook n8n dan diproses untuk diteruskan ke telegram dan zammad.



Gambar 4. 91 detail operation baru

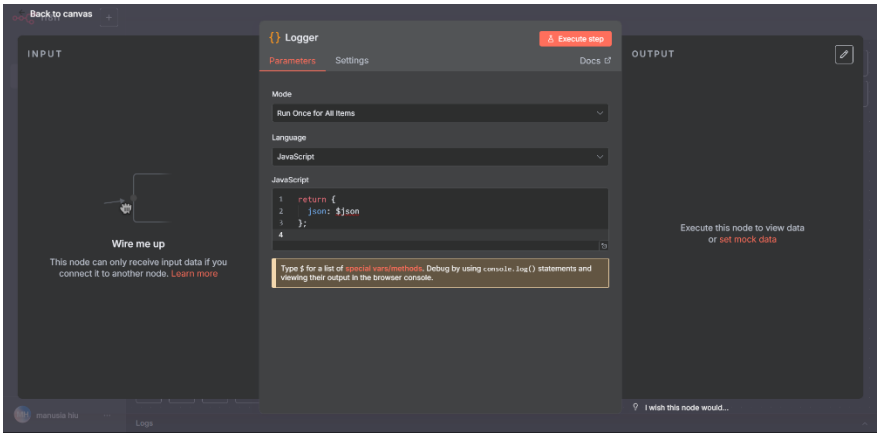
16) Action yang telah dibuat akan muncul pada list trigger action yang tersedia, dan pastikan bahwa action dalam keadaan enable.



Gambar 4. 92 status trigger action

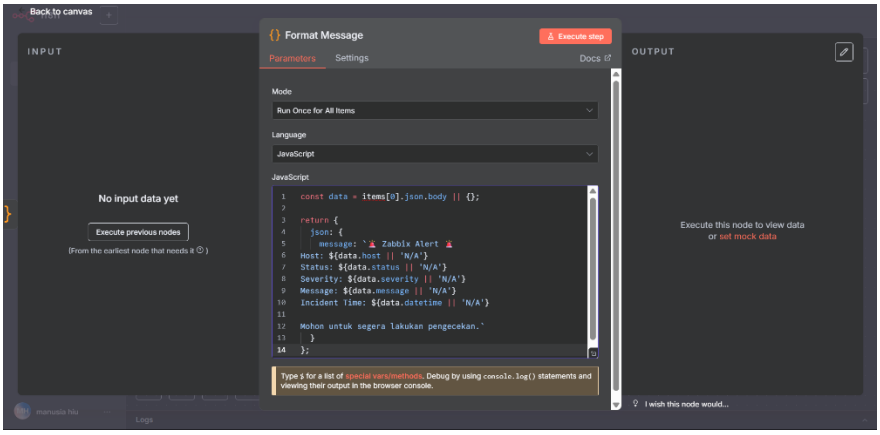
17) Kembali ke workflow n8n, setelah node Webhook, ditambahkan node baru dengan nama logger yang menggunakan mode JavaScript.

Node ini hanya berfungsi untuk meneruskan data tanpa mengubah apapun, dan kodenya berisi return { json: $json }. Fungsi node ini berguna sebagai log visual awal dan untuk mempermudah debugging jika terjadi error di tahap berikutnya. Berikut Adalah pengaturan dari node logger.



Gambar 4. 93 node logger

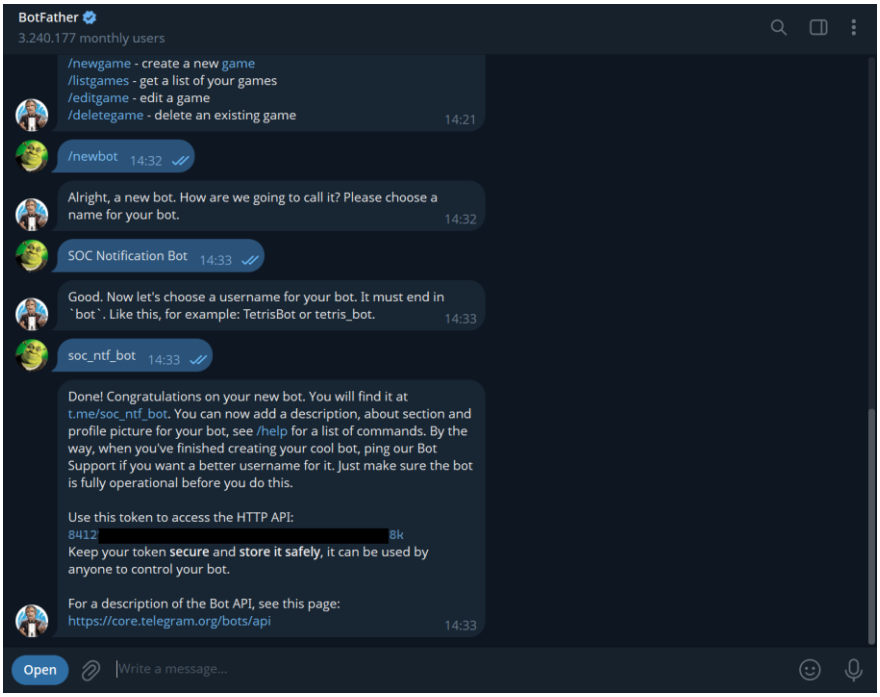
18) Setelah itu ditambahkan node format message, masih menggunakan JavaScript, untuk menyusun struktur pesan yang lebih rapi. Di dalam node ini, data JSON yang masuk akan diurai dan kemudian dibentuk menjadi teks alert yang terformat. Berikut Adalah pengaturan yang digunakan:



Gambar 4. 94 node format message

Pemilihan struktur teks seperti ini bertujuan agar pesan terlihat jelas dan langsung terbaca saat dikirim ke Telegram dan Zammad.

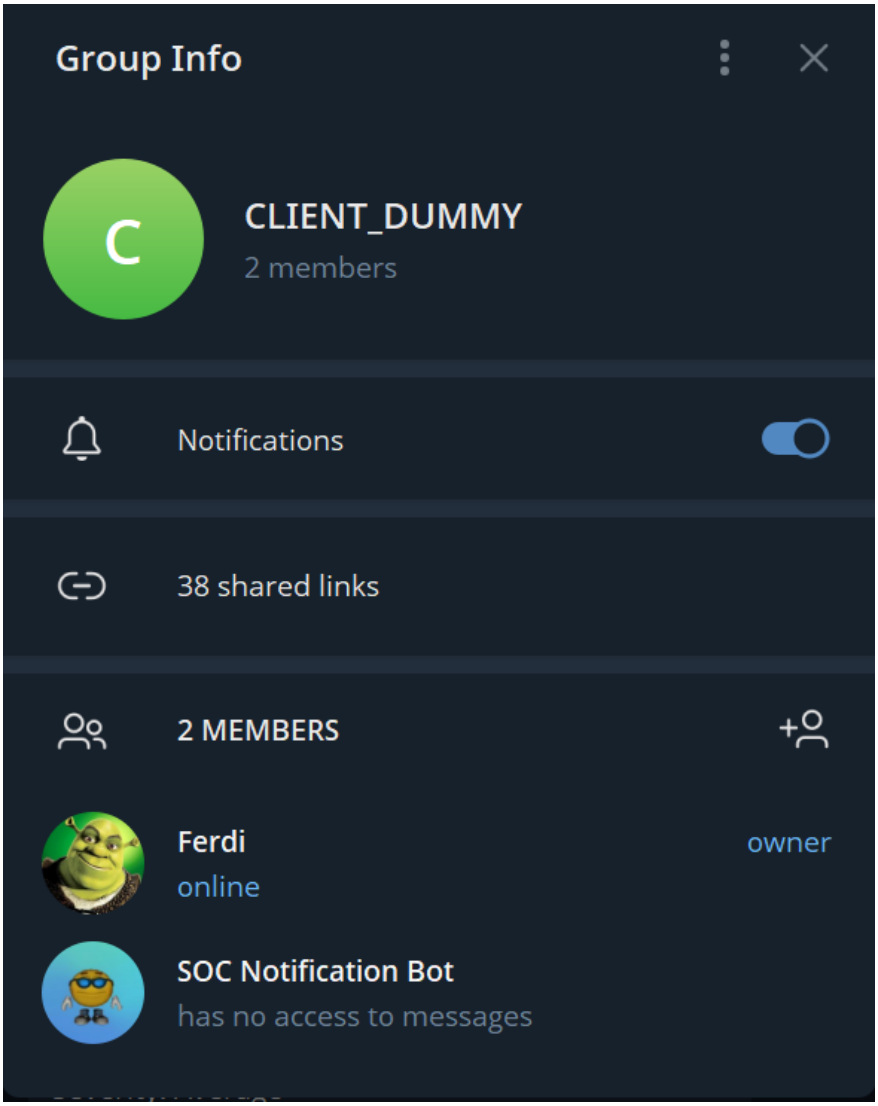
19) Tahap berikutnya adalah membuat bot Telegram yang akan digunakan untuk mengirimkan alert. Bot dibuat dengan bantuan BotFather di Telegram, untuk memulai pembuatan bot kirim pesan /start, setelahnya kirim /newbot, dan akan diminta untuk memberikan nama, untuk bot akan diberi nama SOC Notification Bot dengan username soc_ntf_bot. Setelah bot selesai dibuat, disimpan token bot yang diberikan.



Gambar 4. 95 membuat bot telegram

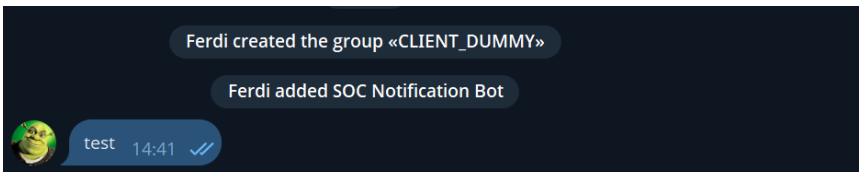
20) Untuk mengujinya, buka link https://t.me//username_bot, sesuaikan dengan username yaitu https://t.me/soc_ntf_bot dan klik Start

21) Setelah mengaktifkan bot yang telah dibuat, buat sebuah group telegram Dimana alert akan dikirimkan dan tambahkan bot sebagai anggota, berikut adalh group yang akan digunakan dengan nama CLIEN_DUMMY.



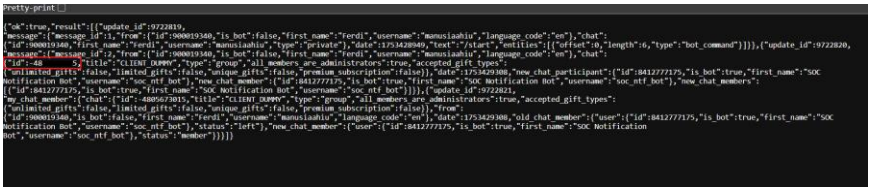
Gambar 4. 96 group telegram info

22) Agar nantinya bot yang telah dibuat dapat mengirimkan pesan ke group, kita perlu mngetahui id dari group yang akan digunakan, untuk itu perlu kirim pesan di group terlebih dahulu.



Gambar 4. 97 pesan testing

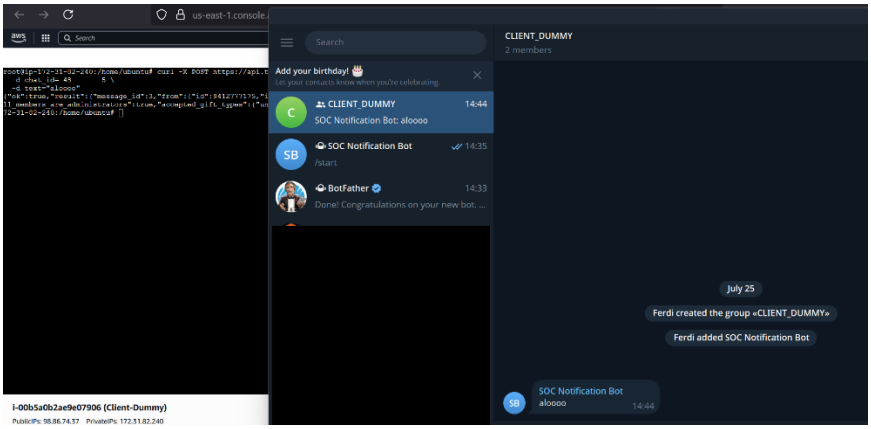
23) Kemudian buka https://api.telegram.org/botTOKEN_BOT/getUpdates pada web browser, halaman web akan menampilkan JSON response Dimana kita bisa mendapatkan Id group. Id dari sebuah group biasanya ditandai dengan awalan minus (-).



Gambar 4. 98 cek group id telegram

24) Setelah mendapatkan id dari group telegram yang digunakan, lakukan Tes Kirim Pesan menggunakan bot ke Grup via cURL dengan menggunakan perintah.

```
curl -X POST https://api.telegram.org/botTOKEN_BOT/sendMessage \ -d chat_id=-(id yang telah didapat) \ -d text="aloooo "
```

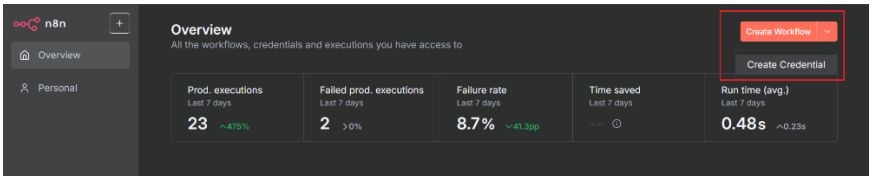


Gambar 4. 99 tes mengirimkan pesan menggunakan bot

Perintah tersebut akan mengirimkan sebuah pesan test “alooo” ke group CLIENT_DUMMY, dan terlihat Bot telah berhasil mengirimkan pesan ke group.

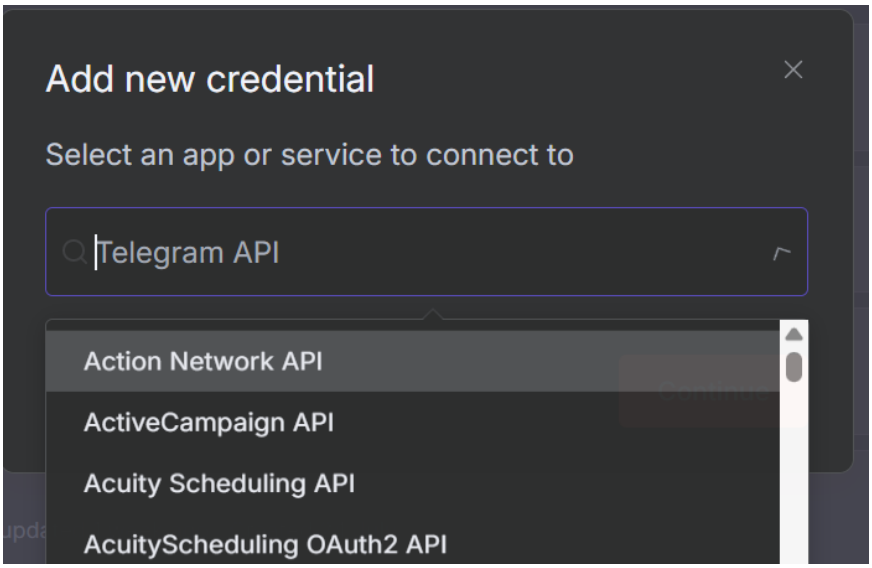
25) Setelah mendapatkan semua informasi dan kredensial yang dibutuhkan Kembali ke halamn UI n8n, disini kita akan membuat credensial yang akan digunakan untuk membuat node telegram

nantinya. Pada pojok kanan atas klik tombol create workflow kemudian akan muncul dropdown pilihan create credential.



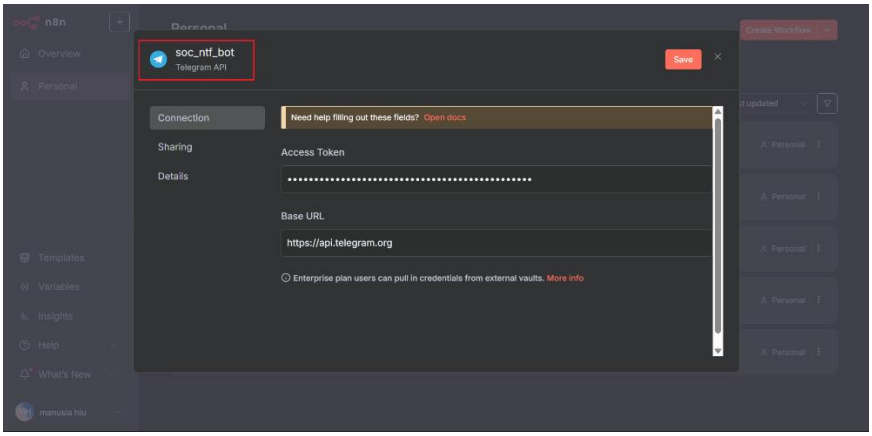
Gambar 4. 100 create credential

26) Search telegram API dan klik continue.



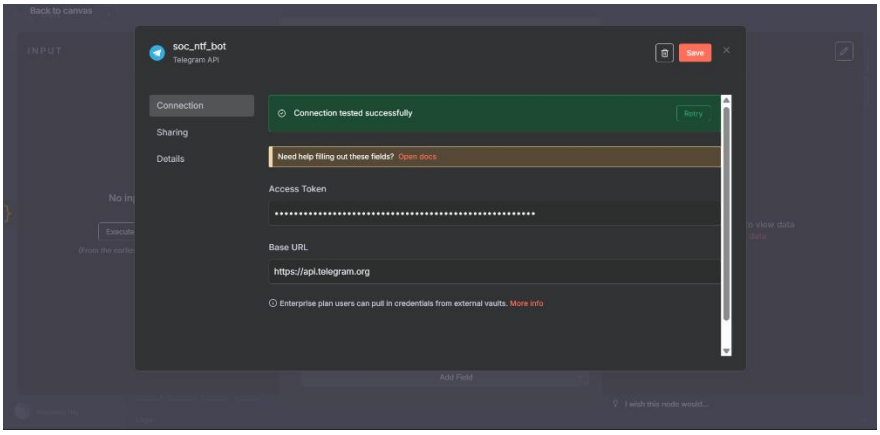
Gambar 4. 101 cari tipe credential

27) Untuk memudahkan Ganti nama dari crential yang dibuat dengan cara klik dua kali pada nama credential, disini kita akan memberi nama credential ini soc_ntf_bot, pada kolom access token isikan token bot yang telah didapat pada Langkah sebelumnya.



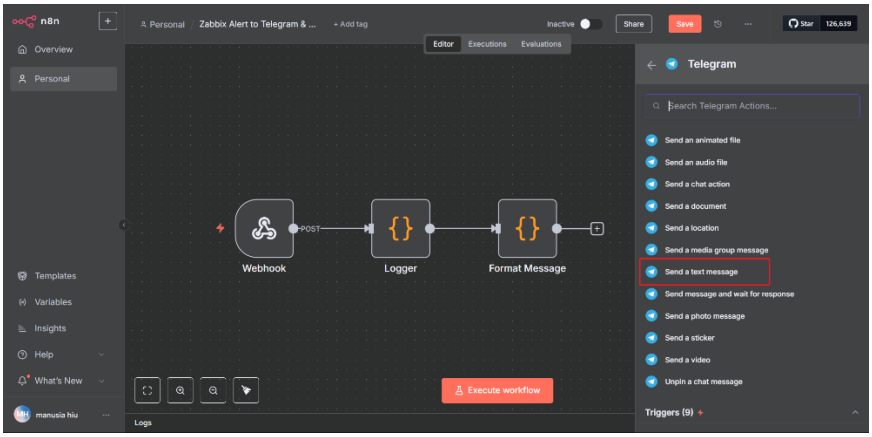
Gambar 4. 102 set telegram credential

28) Setelah mengisi data yang dibutuhkan klik save, n8n akan secara otomatis melakukan test koneksi terhdapa credential yang kita buat dan akan menampilkan informasi bahwa credential telah berhasil terhubung atau gagal.



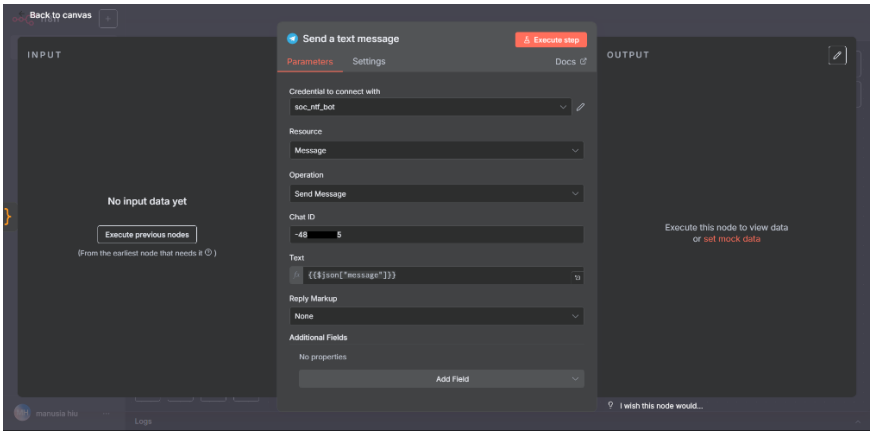
Gambar 4. 103 test credential telegram

29) Langkah selanjutnya Adalah Kembali ke workflow dan tambahkan sebuah nde baru setelah node logger, tambahkan sebuah node telegram dengan memilih operasi send a text message.



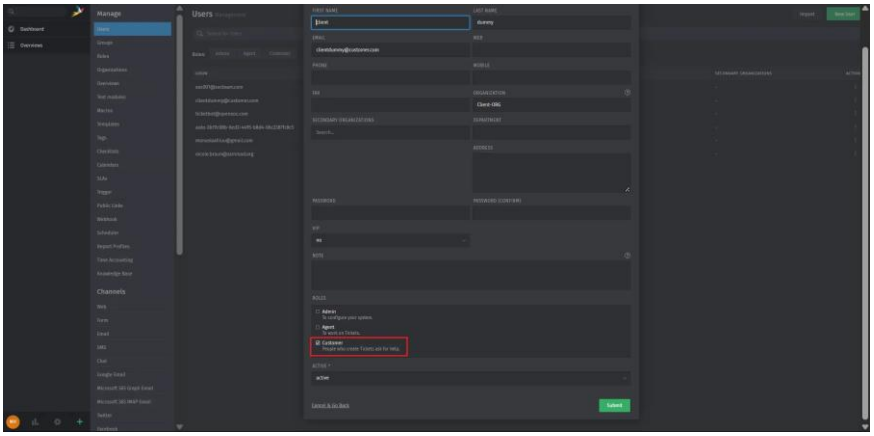
Gambar 4. 104 menambahkan node telegram

30) Pada node ini ini akan dibutuhkan semua kredensial yang telah dibuat dan credential lainnya seperti group id. Kolom Text diisi dengan {{$json["message"]}} yang berarti teks alert dari node sebelumnya akan dikirim langsung ke Telegram.



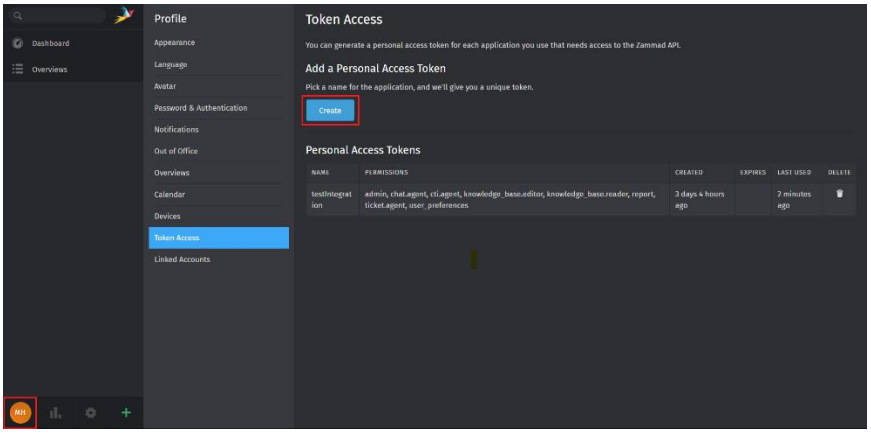
Gambar 4. 105 konfigurasi node telegram

31) Selanjutnya pada zammad perlu dibuat sebuah organisasi dan user yang akan menjadi client/customer, Dimana nantinya ticket yang dibuat akan di assign kepada customer terkait, jika nantinya memiliki banyak client. Untuk sekarang kita akan membuat akun customer dengan nama client dummy dan tergabung ke group Client-Org. pastikan bahwa roles dari client dummy Adalah customer.



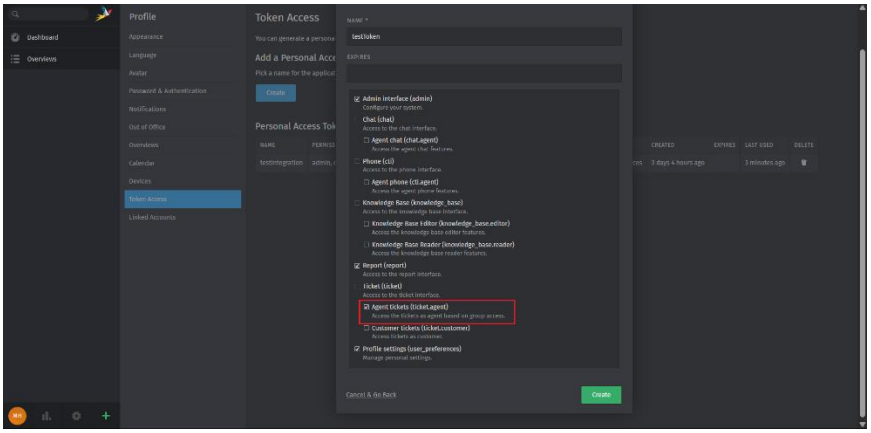
Gambar 4. 106 membuat akun cutommer zammad

32) Kemudian untuk menjalankan proses pembuatan tiket secara otomatis perlu untuk mendapatkan akses token dari akun yang setidaknya memiliki privilege sebagai agent. Untuk test ini kita akan menggunakan akses token milik admin. Klik ikon profil yang terlihat pada halaman home zammad kemudian masuk ke menu Token Access dan klik create.



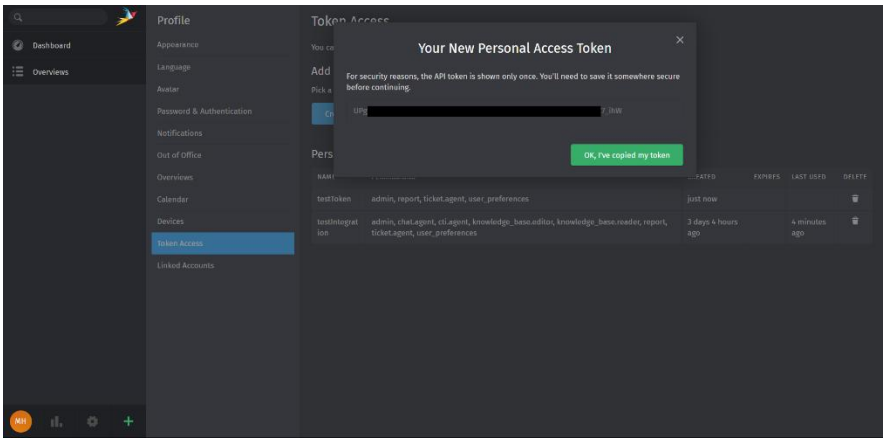
Gambar 4. 107 membuat access token zammad

33) Berikan nama pada token akses dan pastikan untuk memberikan akses pada agent tiket agar nantinya n8n melalui token akses dapat melakukan proses operasi pada tiket.



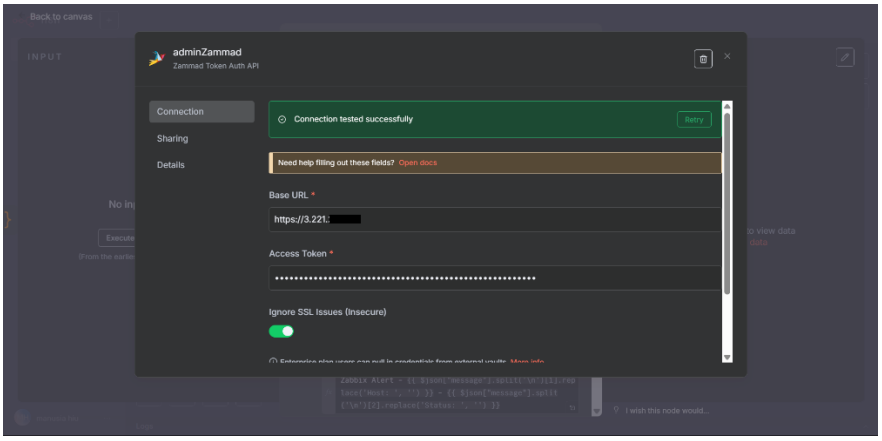
Gambar 4. 108 enable agent ticket

34) Setelah mengklik tombol create akan muncul pemberitahuan untuk segera menyalin token karena token akses hanya akan ditampilkan sekali, tepat setelah proses pembuatan.



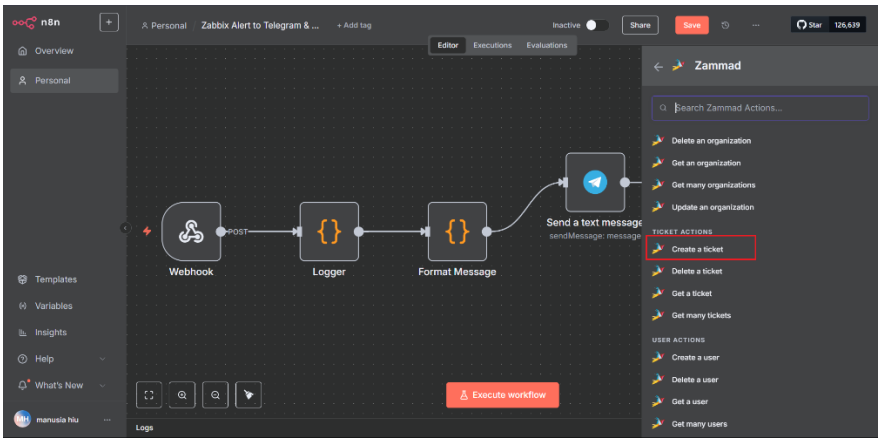
Gambar 4. 109 access token zammad

35) Pada UI n8n buat sebuah credential seperti sebelumnya, pilih Zammad Token Auth API sebagai tipe dari credential, kemudian beri nama adminZammad dan tambahkan url zammad dan token akses yang didapatkan sebelumnya.



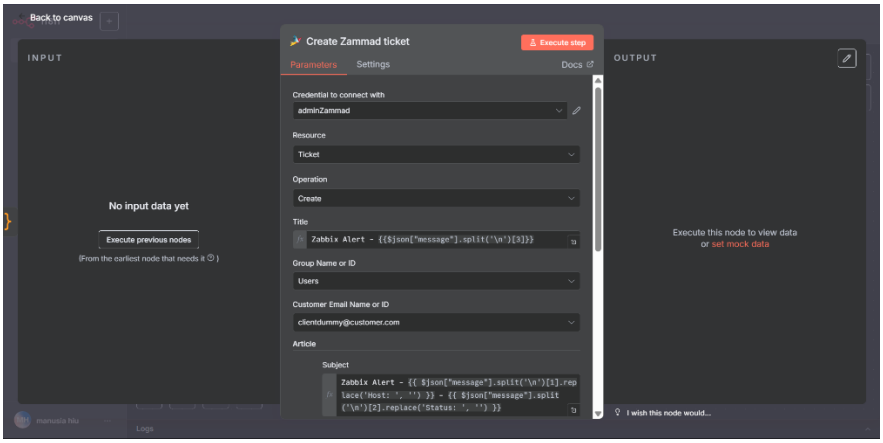
Gambar 4. 110 membuat credential zammad

36) Selanjtunya kemabali ke n8n workflow dan tambahkan sebuah node ‘Create a zammad Ticket’, Dimana nantinya node ini bertugas untuk membuat ticket dari event yang terjadi. Node akan mengambil data dari node format message dan berjalan paralel bersama dengan alert telegram.



Gambar 4. 111 menambahkan node zammad

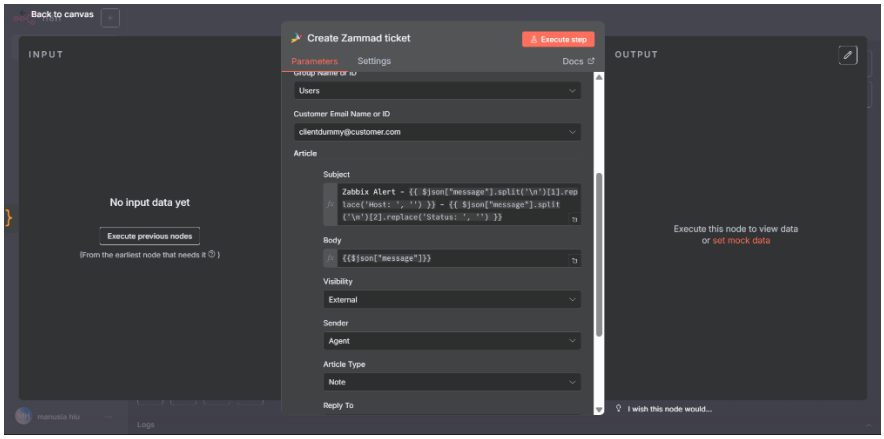
37) Untuk node zammad sendiri terdapat beberapa pengaturan yang harus disesuaikan. Mulai dari memilih credential yang telah dibuat sebelumnya, kemudian pilih resource ticket dan operasi yang akan dilakukan Adalah Create, untuk bagian group pilih Users kemudian bagian customer id name or email pilih user client yang telah dibuat pada proses sebelumnya.



Gambar 4. 112 konfigurasi node zammad

38) Masih pada node Create ticket zammad, untuk bagian body isi dengan {{$json[“message”]}} Dimana nantinya inila yang akan menjadi isi dari ticket yang merupakan pesan incident yang sudah terformat. Kemudian bagian lain yang perlu disesuikan Adalah:

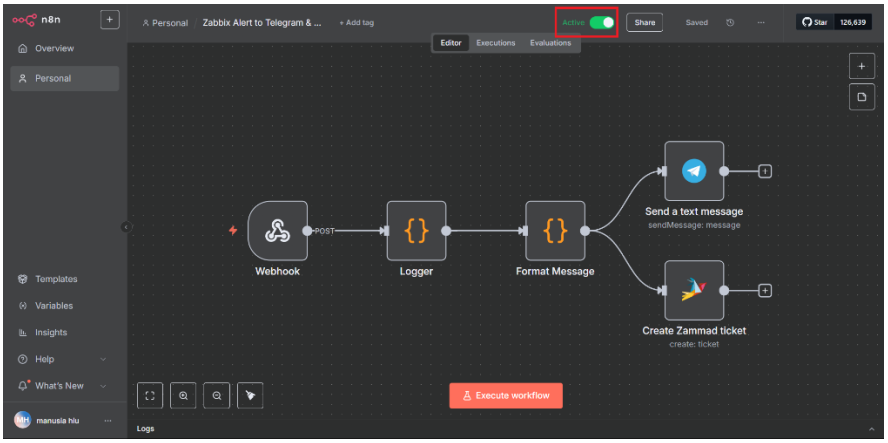
- Visibility: External
- Sender: Agent
- Article Type: Note



Gambar 4. 113 node zammad

39) Stelah seluruh node dikonfigurasi, sembungkan setiap node secara berurutan mulai dari webhook sampai dengan telegram dan zammad, terakhir aktifkan workflow agar nantinya jika terdapat alert workflow akan tertrigger dan berjalan secara otomatis. Berikut

Adalah tampilah final dari workflow Zabbix Alert to Telegram & Zammad.



Gambar 4. 114 aktifkan worflow

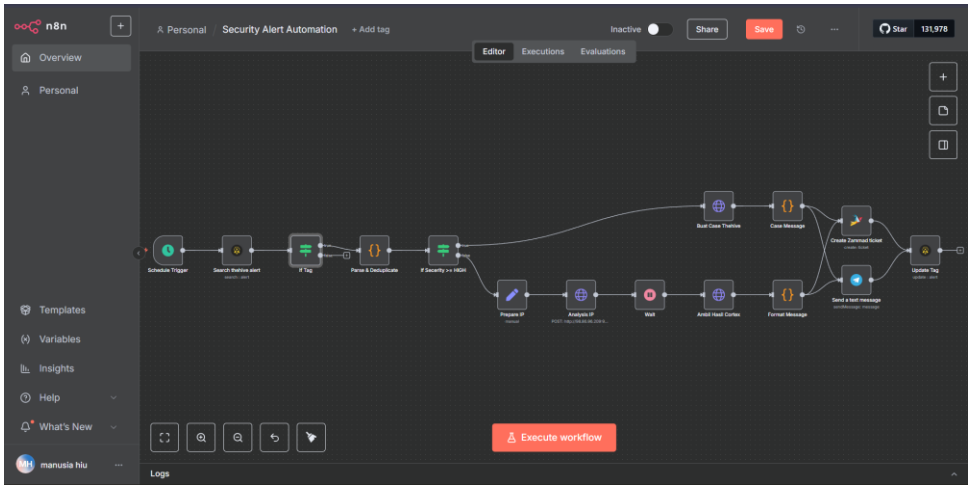
Workflow Automate Utilization Alert sudah berhasil dibuat, workflow ini nantinya akan berjalan Ketika terdapat problem pada Zabbix yang akan mentrigger action untuk mengirimkan alert melalui media type dan user yang telah dibuat ke webhook, alert yang masuk akan melalui node logger terlebih dahulu sebelum kemudian di format menjadi pesan yang rapi. Kemudian pesan yang sudah diformat akan dikirim ke group telegram dan dibuatkan ticket di zammad secara bersamaan.

4.3.5 Security Alert & Ticket Automation

Pada security alert automatin sendiri terdapat dua cabang workflow. Pertama, apabila alert memiliki severity ≥ HIGH, maka secara otomatis n8n workflow akan mengubah alert tersebut menjadi sebuah case pada TheHive. Setelah case berhasil dibuat, sistem juga akan mengirimkan notifikasi informasi ke Telegram lengkap dengan detail case yang tercatat, sekaligus membuatkan ticket baru di Zammad secara otomatis.

Kedua, apabila severity berada di bawah HIGH, maka alert tersebut tidak langsung dibuat menjadi case. Sebagai gantinya, data dari alert akan diekstrak terlebih dahulu untuk diambil artefak penting seperti alamat IP. Artefak ini kemudian dianalisis menggunakan Cortex Analyzer. Hasil

analisis tersebut, bersama dengan ringkasan (summary) dari alert, selanjutnya dikirim ke Telegram agar bisa dipantau, dan pada saat yang sama juga dibuatkan ticket di Zammad untuk penanganan lebih lanjut.

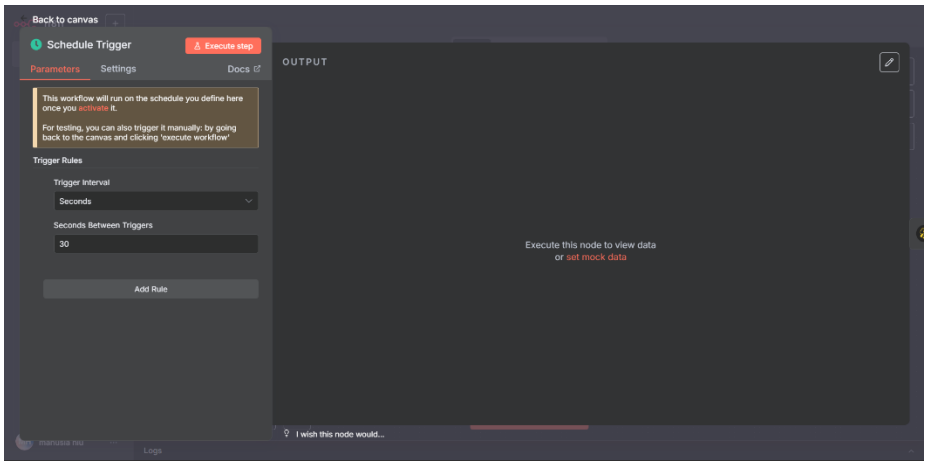


Gambar 4. 115 security automation workflow

Urutan eksekusi dan pengaturan setiap node pada workflow ini Adalah sebagai berikut:

1) Schedule Trigger

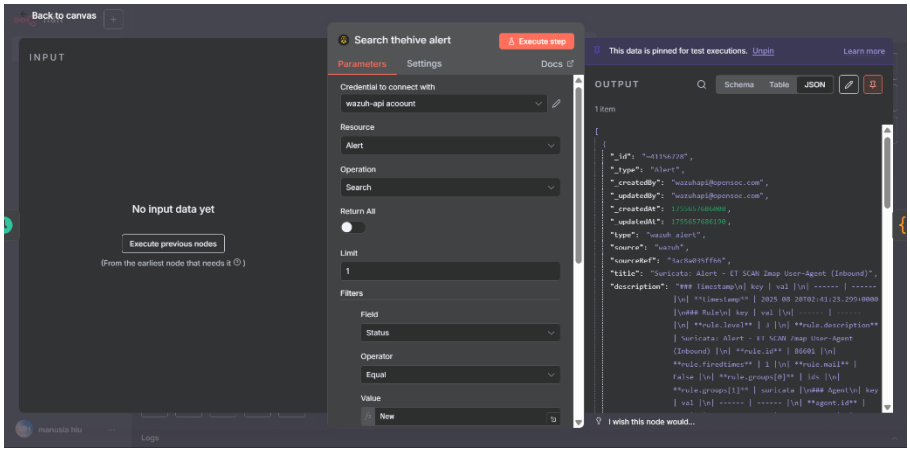
Node ini berfungsi untuk melakukan trigger otomatis berdasarkan jadwal waktu yang sudah ditentukan sebelumnya. Dengan node ini, workflow bisa berjalan secara teratur tanpa harus dijalankan manual.



Gambar 4. 116 node scedhule trigger

2) Search TheHive Alert

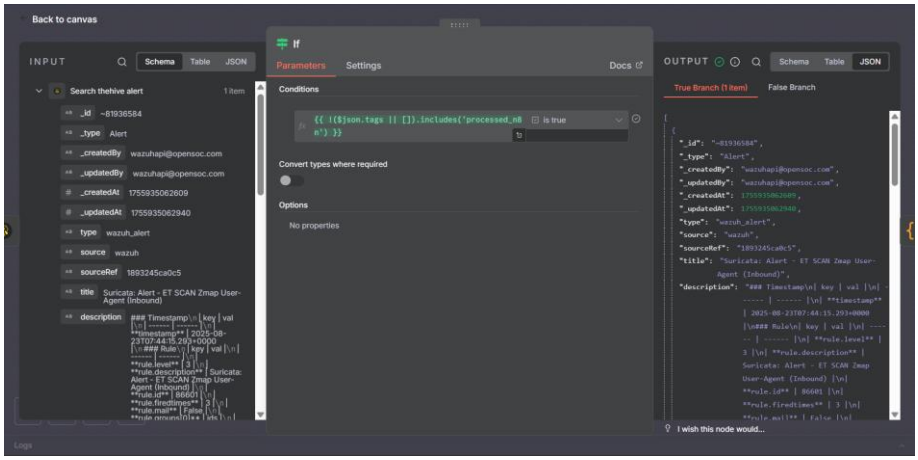
Node ini bertugas mengambil alert terbaru dari TheHive. Fungsinya sebagai pintu masuk data, sehingga workflow selalu bekerja dengan alert paling terbaru yang muncul di sistem. Selain itu, pada node ini juga diterapkan mekanisme polling window, yaitu dengan menambahkan filter `_createdAt > now - 70` detik. Tujuannya untuk memastikan hanya alert yang benar-benar baru (dalam jendela waktu eksekusi workflow) yang diproses, sehingga menghindari duplikasi akibat interval polling.



Gambar 4. 117 node search alert thehive

3) IF – Cek Tag (processed_n8n)

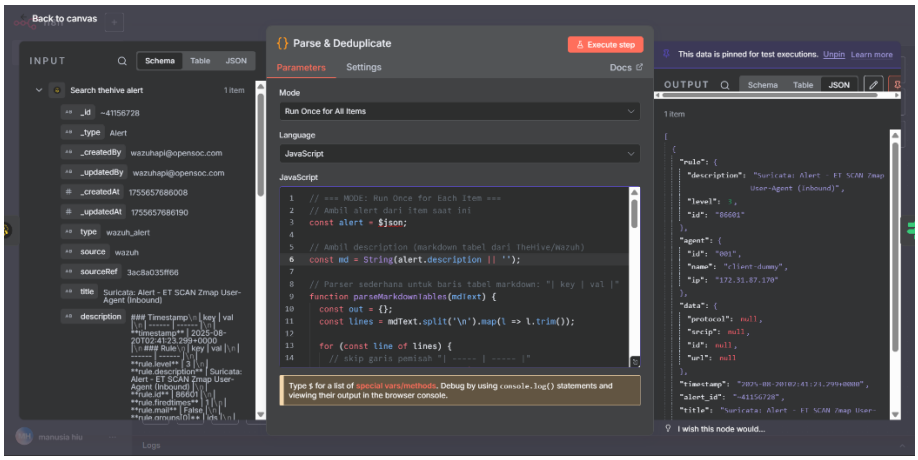
Node ini berfungsi sebagai filter tambahan untuk memastikan alert belum pernah diproses Cara kerjanya yaitu dengan melakukan pengecekan apakah alert memiliki tag processed_n8n jika tidak maka workflow akan melanjutkan pemrosesan, dan jika sudah maka workflow akan menghentikan pemrosesan alert tersebut. Dengan strategi ini, workflow menjadi idempotent (satu alert hanya diproses sekali) walaupun workflow terus berjalan secara periodik.



Gambar 4. 118 node if - cek tag

4) Parse & Deduplicate

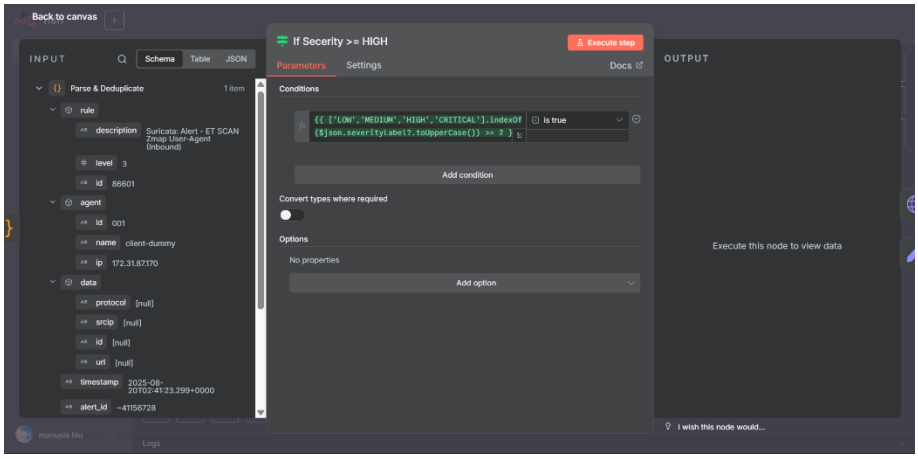
Node ini digunakan untuk melakukan parsing data dari alert TheHive yang sudah diambil oleh node sebelumnya, sekaligus melakukan proses deduplikasi agar tidak ada alert yang sama diproses berulang-ulang.



Gambar 4. 119 node harsing

5) If Severity >= HIGH

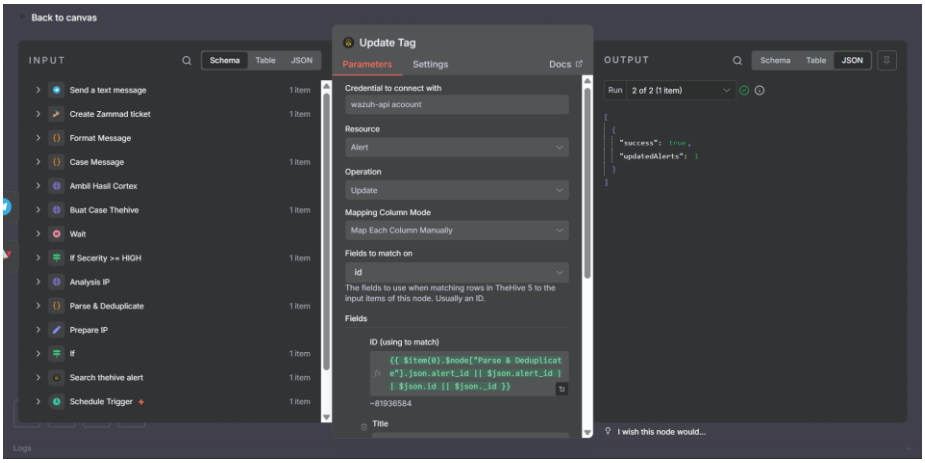
Pada node ini ditentukan tindakan yang akan dilakukan terhadap alert. Jika alert memiliki severity HIGH atau lebih, maka sistem akan langsung membuat case. Namun jika severity berada di bawah HIGH, maka alert tersebut tidak dibuat menjadi case, melainkan akan dilakukan analisis otomatis terlebih dahulu untuk mendapatkan informasi lebih detail.



Gambar 4. 120 node if severity

6) Node Update Tag

Node ini ditempatkan di akhir jalur proses (baik pada branch severity tinggi yang membuat case, maupun branch severity rendah yang melakukan analisis & notifikasi). Fungsinya untuk memberi penanda bahwa alert sudah diproses dengan menambahkan tag khusus processed_n8n ke alert tersebut di TheHive. Dengan adanya tag ini, pada eksekusi workflow berikutnya node IF Cek Tag dapat mengenali alert tersebut dan melewati prosesnya, sehingga tidak terjadi duplikasi.



Gambar 4. 121 node update tag

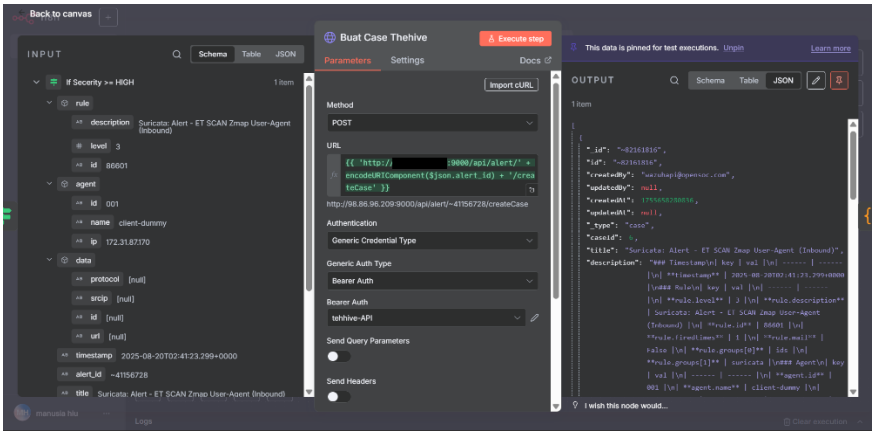
Alur selanjutnya akan dijelaskan berdasarakan branch, yaitu true branch dan falase branch.

A. True branch

Branch ini hanya akan dijalankan bila severity pada alert dikategorikan sebagai HIGH atau lebih.

1) HTTP Request (Buat Case Thehive)

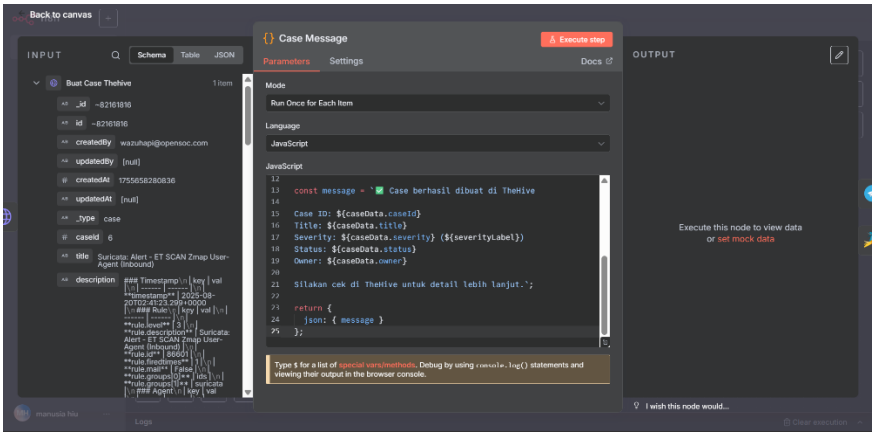
Setelah node If Severity >= HIGH, ditambahkan sebuah node HTTP Request yang bertugas melakukan proses pembuatan case dari alert terkait ke TheHive. Pada node ini diatur endpoint dan API key dari TheHive agar case bisa terbentuk secara otomatis berdasarkan alert yang dipilih.



Gambar 4. 122 node create case

2) Code (Case Message)

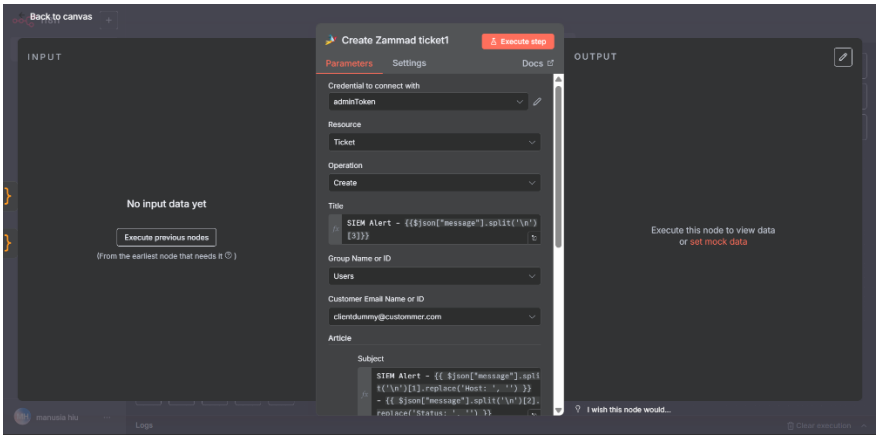
Node ini diberi nama Case Message dan digunakan untuk melakukan formatting dari output node Create Case TheHive. Pesan yang dihasilkan berisi ringkasan singkat (summary) dari detail case sehingga informasi yang dikirimkan nantinya lebih mudah dibaca dan langsung menggambarkan inti dari case yang terbentuk.



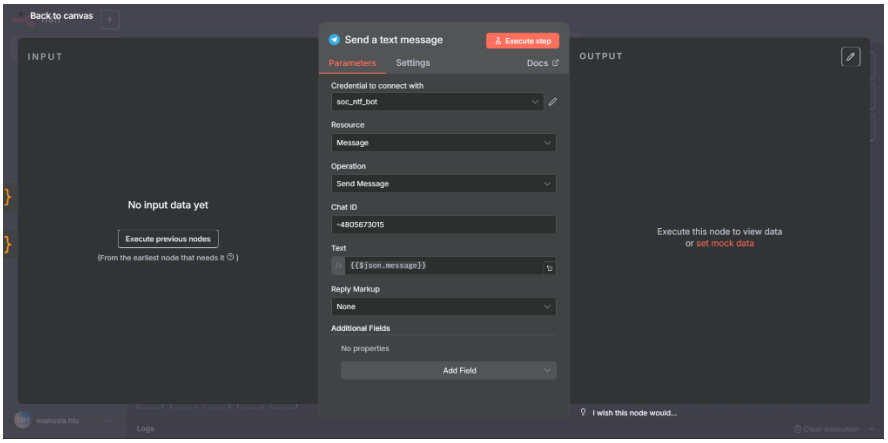
Gambar 4. 123 node case message

3) Telegram & Zammad

Node terakhir adalah Telegram dan Zammad dengan konfigurasi yang sama seperti pada workflow utilization alert. Node Telegram digunakan untuk mengirimkan notifikasi berisi informasi case ke grup, sedangkan node Zammad akan secara otomatis membuat ticket baru berdasarkan case tersebut agar bisa ditindaklanjuti oleh tim.



Gambar 4. 124 node zammad

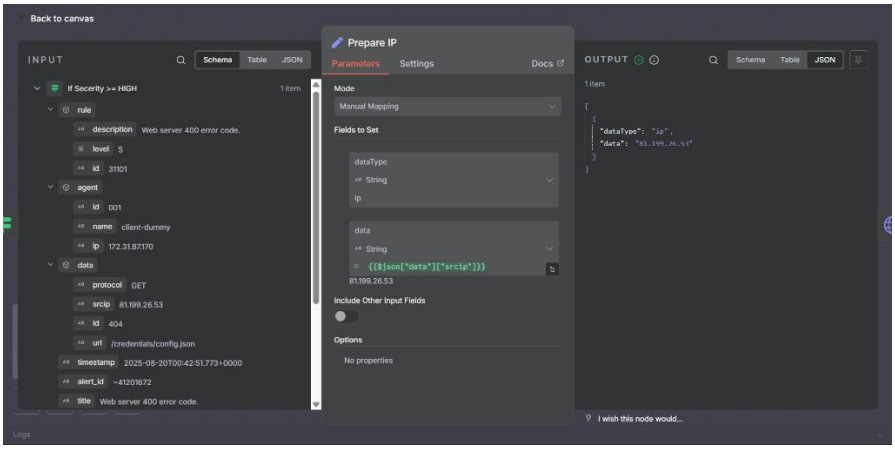


Gambar 4. 125 node telegram

B. False Branch

Branch ini akan dijalankan bila alert memiliki severity yang lebih rendah dari HIGH, untuk pengaturan nodenya yaitu:

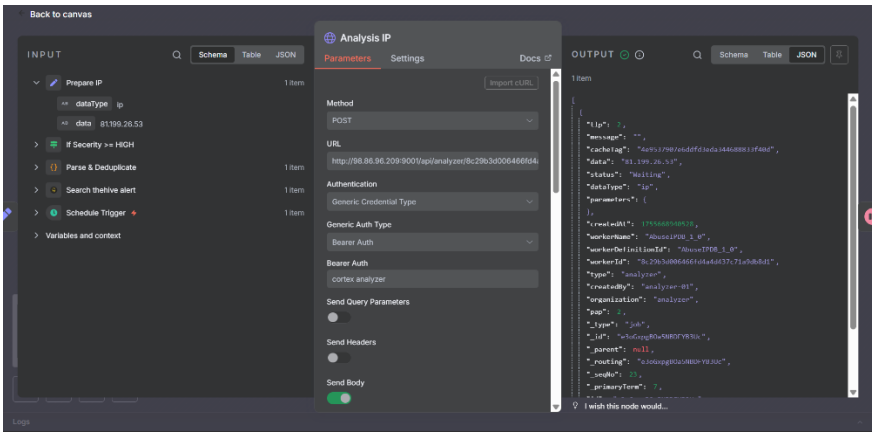
1) Pada branch false akan hetika alert memiliki severity lebih rendah dari HIGH, untuk node 1 sampai 4 sendiri menggunakan node yang sama karena hanya terdapat perbedaan pada node kelima dan seterusnya pada false branch
2) Node yang akan berjalan setelah node if pada false branc Adalah sebuah node set dengan nama Prepare IP, node ini digunakan untuk mengambil artefact ip dari node sebelumnya.



Gambar 4. 126 node prepare ip

3) HTTP Request (Analysis IP)

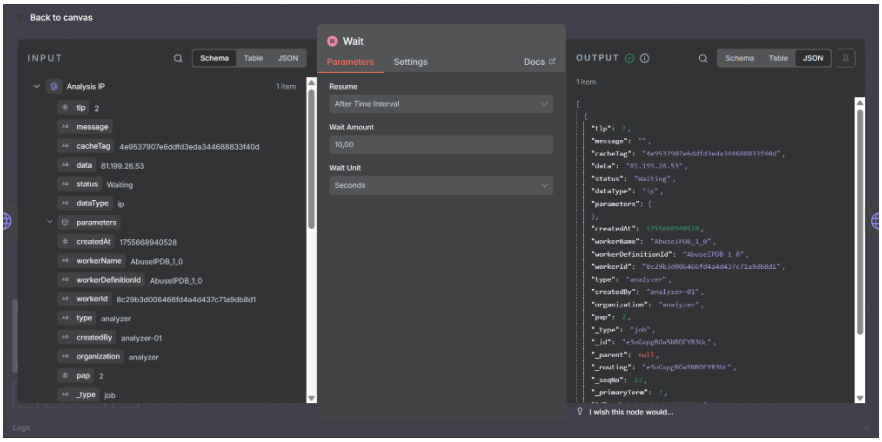
Node ini digunakan untuk mengirimkan artefak berupa IP dari alert ke Cortex. Pada node ini dipilih analyzer yang sesuai untuk melakukan analisis otomatis terhadap IP tersebut, misalnya untuk mengecek reputasi atau indikasi malicious activity.



Gambar 4. 127 node analysis ip

4) Wait

Selanjutnya ditambahkan node Wait untuk memberikan jeda waktu, karena proses analisis pada Cortex membutuhkan waktu tertentu agar hasilnya benar-benar selesai sebelum dilanjutkan ke tahap berikutnya.

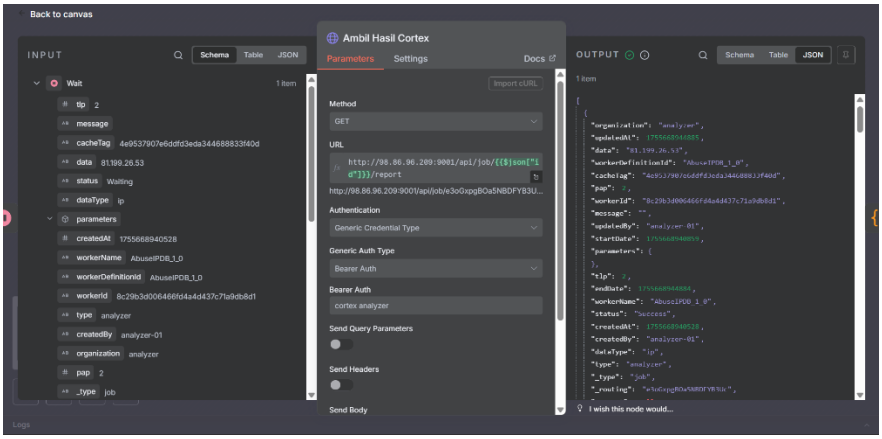


Gambar 4. 128 node wait

5) HTTP Request (Ambil hasil cortex)

Node berikutnya adalah HTTP Request yang berfungsi untuk mengambil hasil analisis yang sudah dijalankan oleh Cortex. Dari

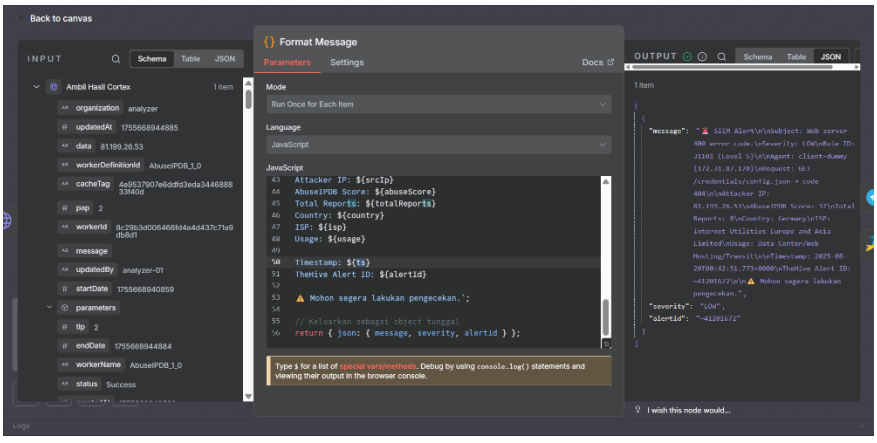
node ini akan diperoleh output berupa detail analisis yang bisa dipakai untuk memperkaya informasi alert.



Gambar 4. 129 node hasil cortex

6) Format Message

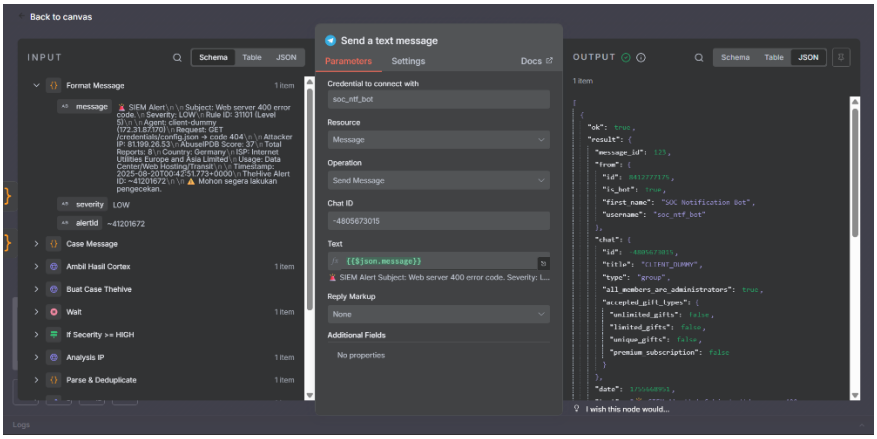
Setelah hasil analisis diperoleh, node ini digunakan untuk memformat pesan yang akan dikirim ke Telegram. Isi pesan berupa ringkasan (summary) dari event alert ditambah dengan hasil analisis Cortex sehingga informasi yang diterima lebih komprehensif.



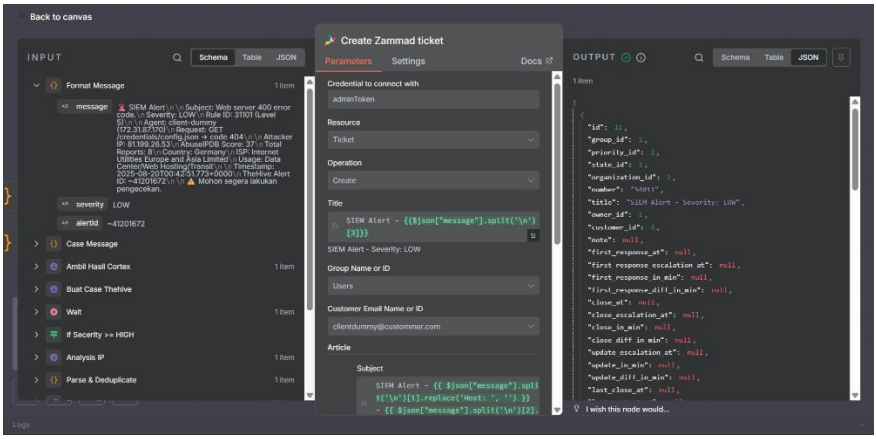
Gambar 4. 130 node format message

7) Telegram & Zammad

Node terakhir adalah Telegram dan Zammad yang digunakan untuk mengirimkan hasil alert lengkap ke Telegram sekaligus membuatkan ticket baru di Zammad. Dengan begitu, alert tetap terdokumentasi dan bisa segera ditindaklanjuti oleh tim.



Gambar 4. 131 node telegram



Gambar 4. 132 node zammad

4.4 Implementasi Scenario Pengujian

4.4.1 Deteksi serangan *"Scanning Attack"*

Pengujian ini bertujuan untuk mensimulasikan serangan pemindaian jaringan (network scanning) yang umum dilakukan penyerang menggunakan tools otomatis seperti ZMap/ZGrab. Aktivitas scanning ini termasuk dalam kategori Reconnaissance Attack, yaitu upaya untuk mengidentifikasi layanan, port, atau konfigurasi sistem target sebelum dilakukan eksploitasi lebih lanjut. Serangan ini juga bertujuan untuk melakukan pengujian pada workflow automation security alert pada branch true yaitu proses pembuatan case otomatis dari alert.

a. Skema pengujian

1) Simulasi serangan dilakukan dari sisi penyerang menuju client-dummy dengan menjalankan perintah:

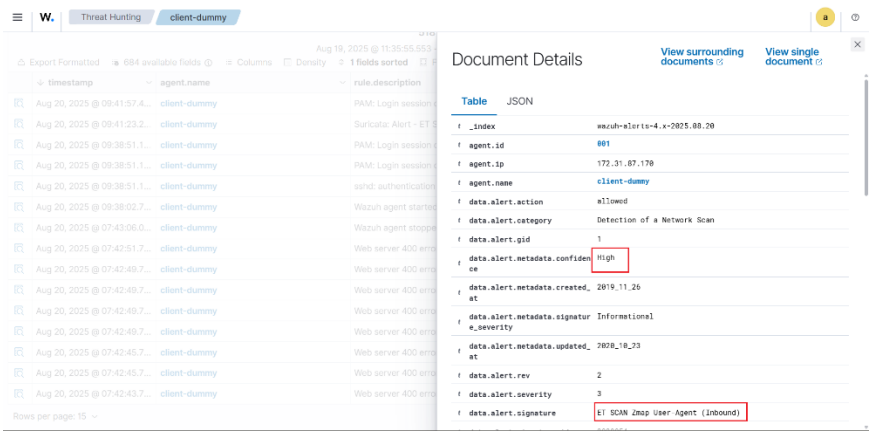
```
curl -A "Mozilla/5.0 zgrab/0.x" -H "Host: <IP Target>"
\
--max-time 3 -sv http://<IP Target>:80/ >/dev/null 2>&1
```

Perintah ini mengirimkan permintaan HTTP (HTTP GET request) dengan User-Agent yang meniru ZGrab, yaitu modul dari ZMap yang umum digunakan untuk pemindaian layanan berbasis aplikasi. Tujuan utamanya adalah memicu signature deteksi Suricata terhadap pola aktivitas scanning.

2) Lalu lintas jaringan yang dihasilkan dari simulasi tersebut diamati oleh Suricata, yang terpasang pada instance client-dummy dan dikonfigurasi untuk memantau trafik di interface eth0.
3) Suricata kemudian menganalisis paket yang masuk dan mencocokkannya dengan rule set yang aktif.
4) Aktivitas mencurigakan terdeteksi dan dicatat dalam bentuk log pada file /var/log/suricata/eve.json.
5) Rule ET SCAN Zmap User-Agent (Inbound) terpicu, menghasilkan sebuah alert. Alert ini selanjutnya diteruskan dan diproses oleh Wazuh, yang berfungsi sebagai SIEM untuk pengelolaan alert dan integrasi dengan komponen SOC lainnya.

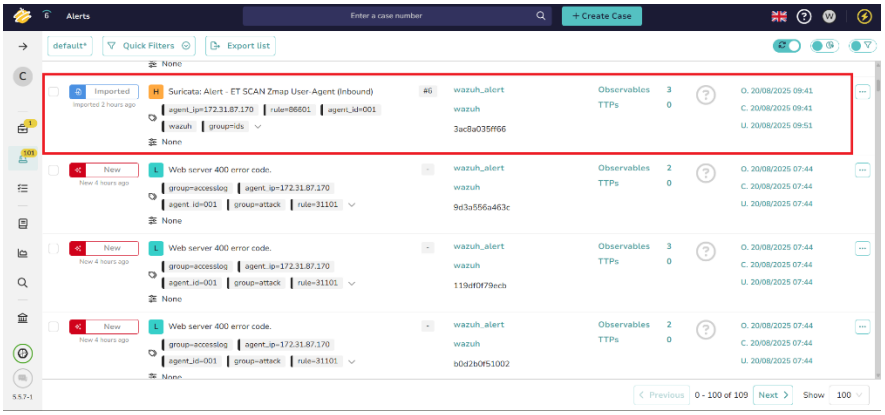
b. Hasil pengujian

1) Wazuh berhasil mendeteksi alert dengan rule ET SCAN Zmap User-Agent (Inbound) dan memberikan klasifikasi severity level tinggi.



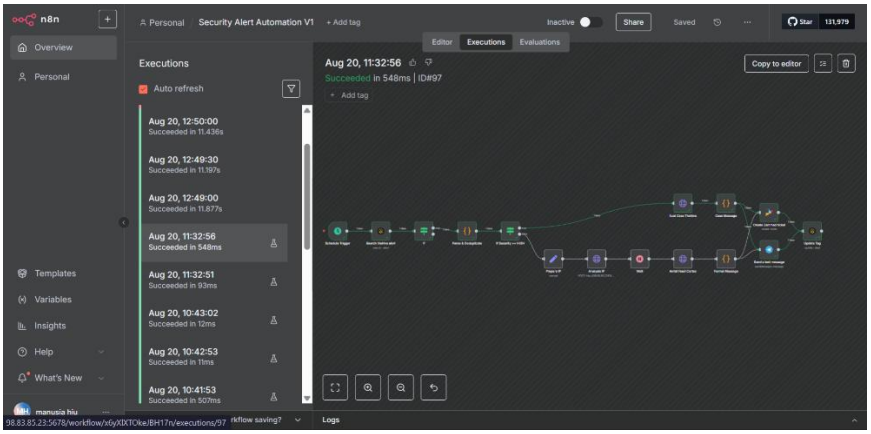
Gambar 4. 133 wazuh alert

2) Alert yang terdeteksi secara otomatis dikirimkan ke TheHive untuk proses manajemen insiden.



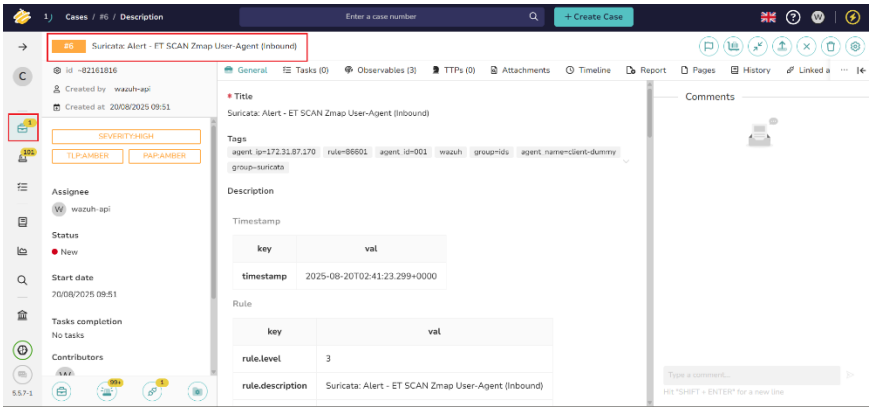
Gambar 4. 134 thehive alert

3) Karena alert memiliki severity tinggi, alur otomasi di n8n memicu workflow pada branch kondisi true, yaitu pembuatan case baru di TheHive.



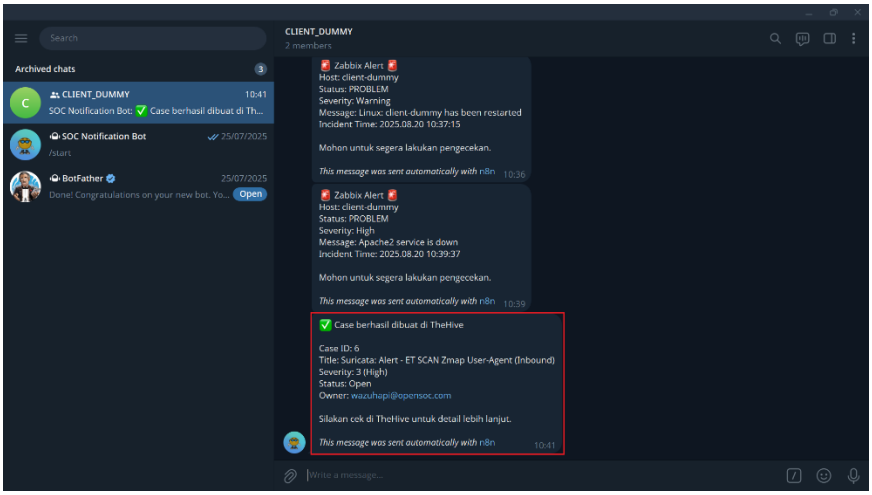
Gambar 4. 135 execution workflow

4) Kasus berhasil dibuat di TheHive sehingga dapat ditindaklanjuti oleh tim keamanan.



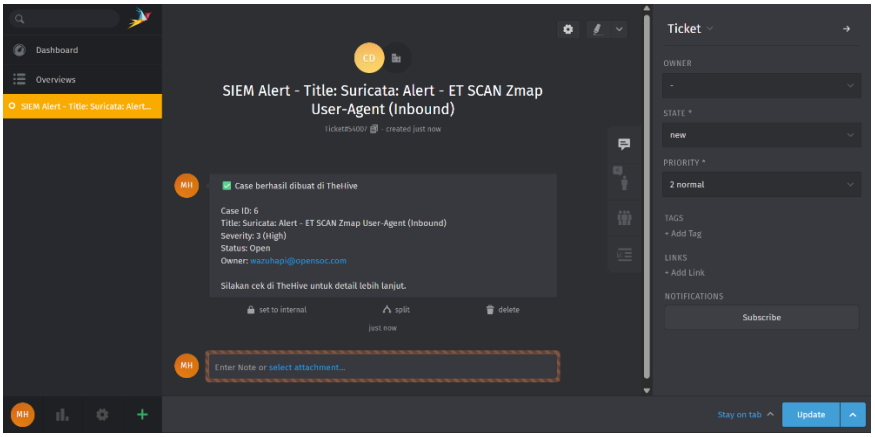
Gambar 4. 136 case thehive

5) Notifikasi alert dikirimkan ke grup Telegram sebagai sarana pemberitahuan cepat terkait case.



Gambar 4. 137 notifikasi telgram

6) Selain itu, sistem secara otomatis membuat tiket insiden pada Zammad sebagai bagian dari incident response & handling untuk nantinya ditindak lanjuti.



Gambar 4. 138 zammad ticket

4.4.2 Deteksi Serangan "*Web Content Discovery / Forced Browsing*"

Pengujian/deteksi ini berfokus pada upaya pencarian konten sensitif pada aplikasi/web server (sering disebut content discovery atau forced browsing). Pola permintaan ke path seperti /credentials/config.json lazim digunakan oleh pemindai otomatis untuk menemukan file kredensial/konfigurasi yang tidak tepat terekspos. Aktivitas semacam ini

dikategorikan sebagai Reconnaissance (tahap pengintaian) dan, bila berhasil, dapat berujung pada kebocoran rahasia aplikasi.

- Klasifikasi serangan: Web Content Discovery / Forced Browsing (Reconnaissance)
- Contoh target path sensitif: /credentials/config.json, /config.json, /backup/, dan sebaginya.
- ATT&CK mapping (relevan): T1595 Active Scanning, T1083 File and Directory Discovery (web-app context).
- OWASP konteks: terkait Broken Access Control / Security Misconfiguration apabila file sensitif bisa diakses.

a) Skema pengujian

Pada client dummy telah diinstal Apache2 yang berfungsi sebagai web server simulasi.

1) Pemicu Trafik (Simulasi / Data Nyata).

Pemindaian dilakukan dengan mengirimkan permintaan HTTP GET ke path sensitif menggunakan perintah:

```
curl -A "Mozilla/5.0" -sv http://<IP Target>/credentials/config.json >/dev/null 2>&1
```

Permintaan ini diharapkan menghasilkan respons 4xx (idealnya 404 atau 403). Tujuannya adalah untuk memicu aturan deteksi pada Wazuh terhadap akses ke path yang dianggap mencurigakan.

2) Pada pengujian ini digunakan data nyata yang berhasil ditangkap oleh Wazuh, yaitu permintaan dari IP publik yang mencoba melakukan serangan serupa dengan mengakses file konfigurasi sensitif.

3) Pengumpulan Log oleh Wazuh.

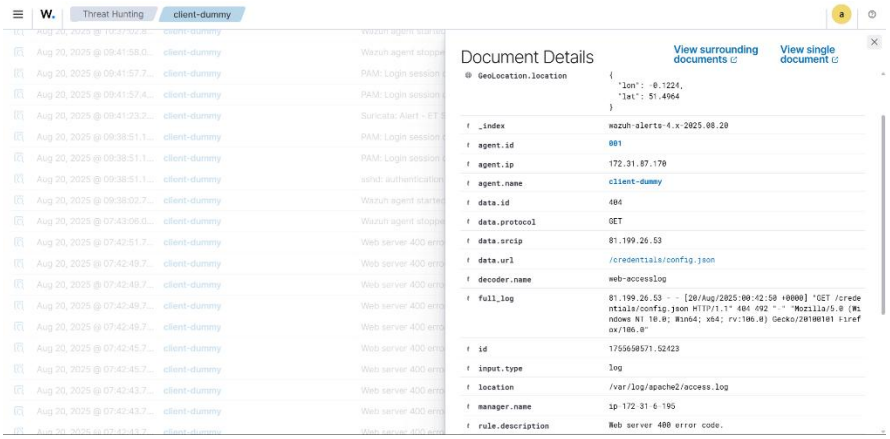
Wazuh Agent melakukan pemantauan terhadap /var/log/apache2/access.log dan memproses entri menggunakan decoder web-accesslog.

4) Pencocokan Aturan (Rule Matching).

Entri akses yang menghasilkan status code 4xx dengan pola path mencurigakan (contoh: /credentials/config.json) memicu Rule ID 31101 dengan deskripsi “Web server 400 error code.” pada level 5 (medium severity).

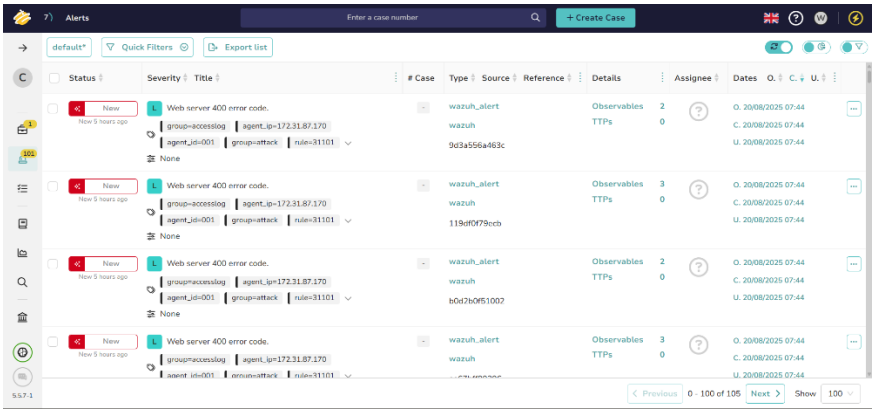
b) Hasil pengujian

1) Wazuh mendeteksi adanya serangan yang berasal dari IP publik 81.199.26.53 yang mencoba mengakses path sensitif /credentials/config.json.

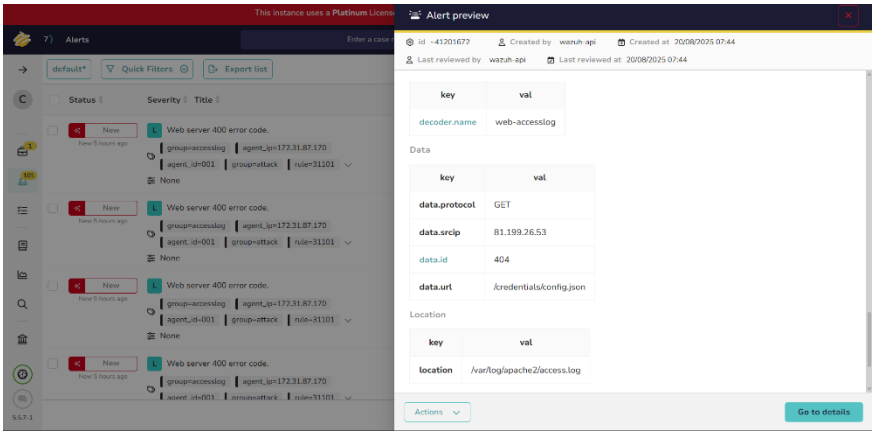


Gambar 4. 139 wazuh alert

2) Alert yang dihasilkan kemudian diteruskan secara otomatis ke TheHive untuk diproses dalam workflow. Pada tampilan TheHive terlihat bahwa alert tersebut hanya memiliki severity Low, ditandai dengan ikon L berwarna biru pada detail alert.

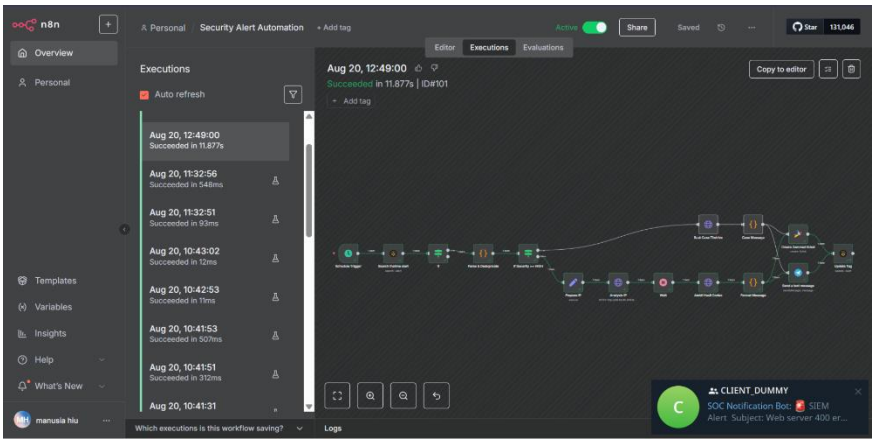


Gambar 4. 140 thehive alert



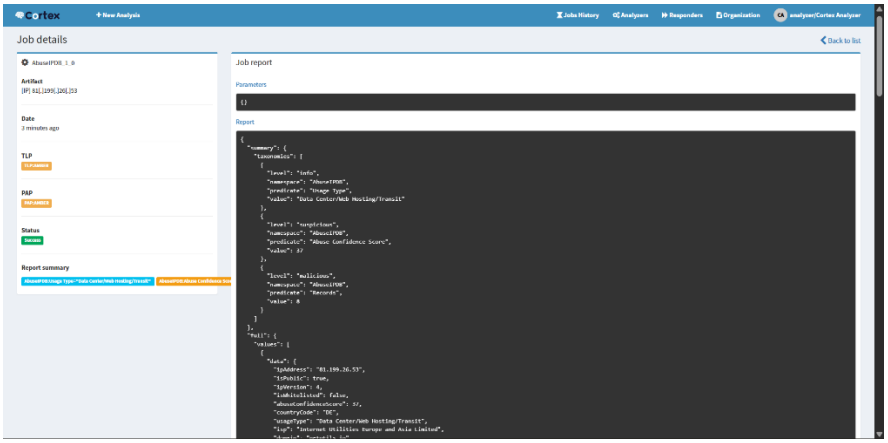
Gambar 4. 141 alert detail

3) Karena severity termasuk kategori low, workflow secara otomatis mengeksekusi jalur false branch alih-alih membuat case baru.



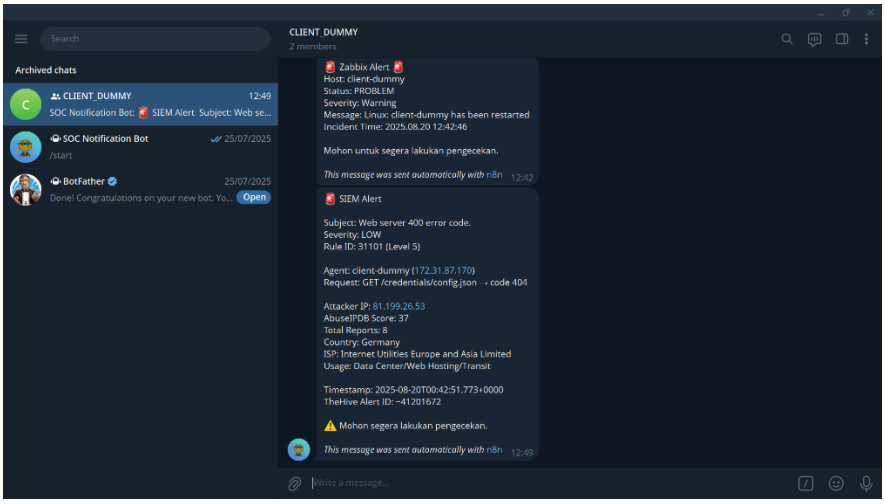
Gambar 4. 142 false branch execution

4) IP address dari alert kemudian diekstrak dan dikirim ke Cortex Analyzer untuk dilakukan analisis reputasi maupun indikasi aktivitas berbahaya. Proses analisis pada Cortex berjalan sukses, dan hasilnya dapat dilihat langsung melalui antarmuka pengguna (UI) Cortex.



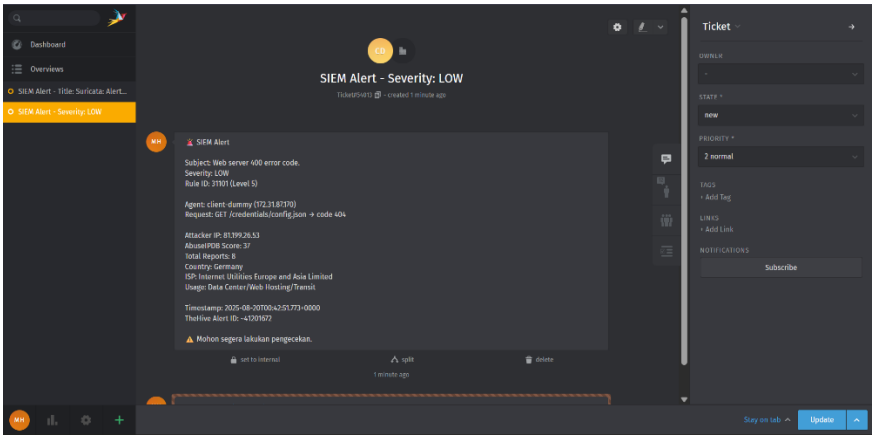
Gambar 4. 143 cortex job

5) Setelah analisis selesai, sistem membentuk pesan ringkasan yang berisi detail event serta hasil analisis Cortex, lalu mengirimkannya ke Telegram.



Gambar 4. 144 tpesan telegram

6) Terakhir, sebuah ticket otomatis berhasil dibuat di Zammad berdasarkan alert ini, sehingga peristiwa tetap terdokumentasi dan bisa ditindaklanjuti.



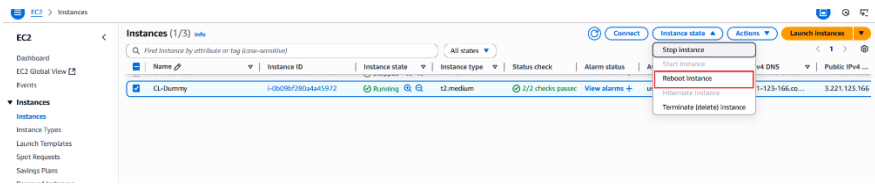
Gambar 4. 145 zammad ticket

4.4.3 Mendeteksi *Host has been restarted* (Restart Client Dummy)

Pengujian ini bertujuan untuk memverifikasi kemampuan Zabbix dalam mendeteksi ketika host (client dummy) tidak dapat dijangkau akibat direstart, serta memastikan alur workflow otomatisasi berjalan end-to-end, mulai dari pemicu action mengirimkan ke webhook n8n, sampai dengan notifikasi Telegram serta pembuatan tiket di Zammad.

a) Skema Pengujian

1) Kondisi awal: host client-dummy dalam keadaan online, Zabbix agent aktif, dan monitoring normal.
2) Reboot client dummy secara manual. Kasus nyata yang dapat menyebabkan host unreachable antara lain: pemadaman listrik, crash kernel, kegagalan patch OS, restart tidak terjadwal, atau masalah hardware.



Gambar 4. 146 reboot instance client

3) Zabbix Server kehilangan koneksi ke agen (item seperti agent.ping atau system.uptime tidak lagi merespons).

4) Trigger Host Unreachable aktif, Action dipicu sesuai dengan ketentuan pada zabbix .
5) alert dikirim ke webhook ke n8n dan worflow dieksekusi hingga menjalan notifikasi Telegram, dan pembuatan tiket Zammad.

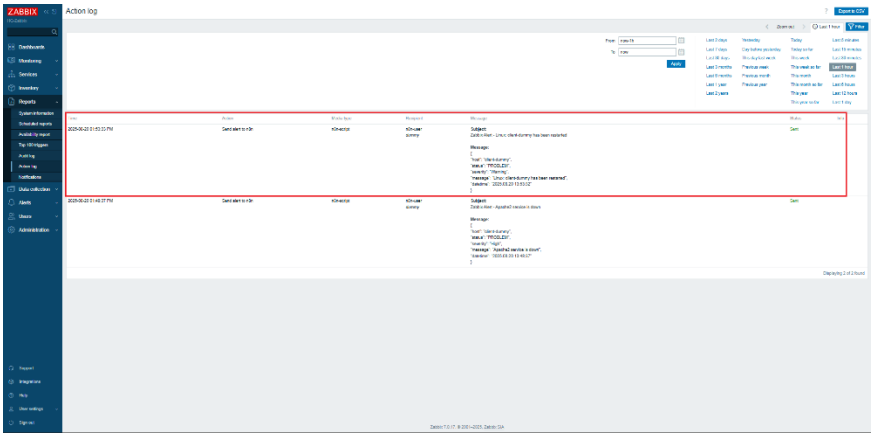
b) Hasil pengujian

1) Problem terbentuk di Zabbix (severity Warning) dengan nama Linux: client-dummy has been restarted yang dapat dimonitoring melalui grafana dahsboard yang telah dibuat



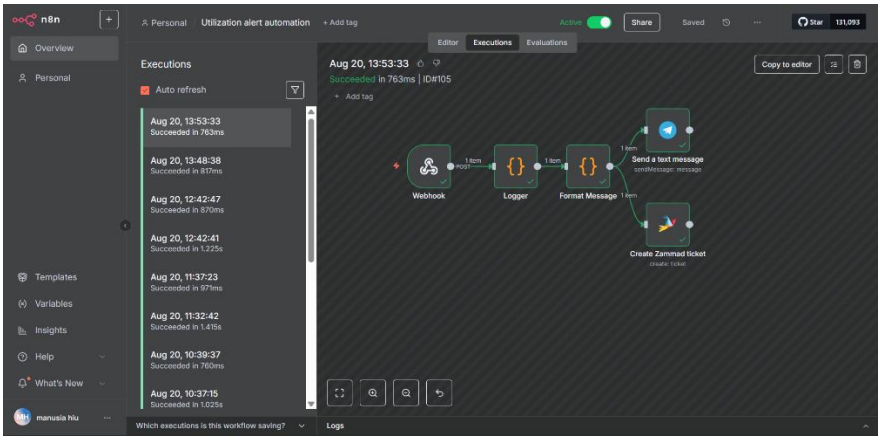
Gambar 4. 147 grafana dashboard

2) Action Log Tercatat operation Send alert to n8n dengan status sent



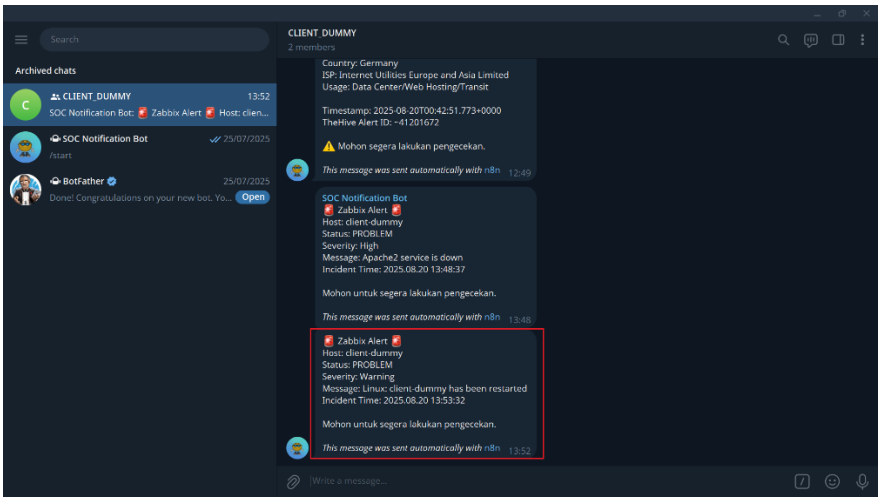
Gambar 4. 148 action log

3) n8n berhasil menjalankan automatin workflow



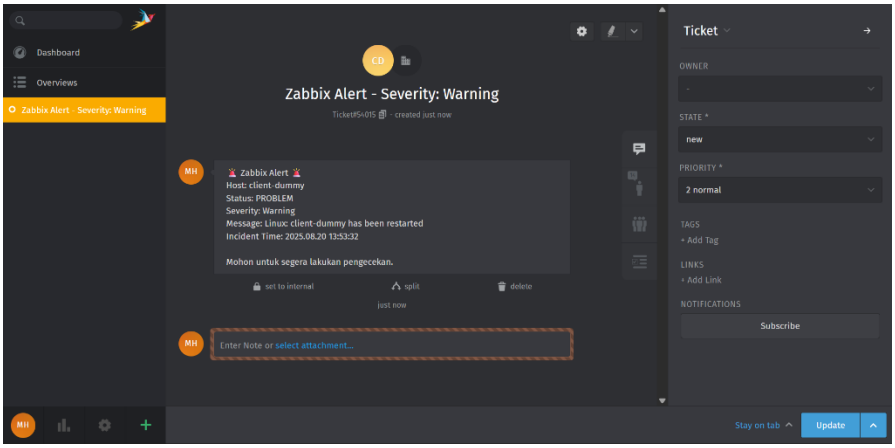
Gambar 4. 149 worflow execution

4) Telegram mengirimkan ke grup insiden berisikan informasi terkait dari alert yang terjadi.



Gambar 4. 150 pesan telegram

5) Zammad membuat ticket insiden



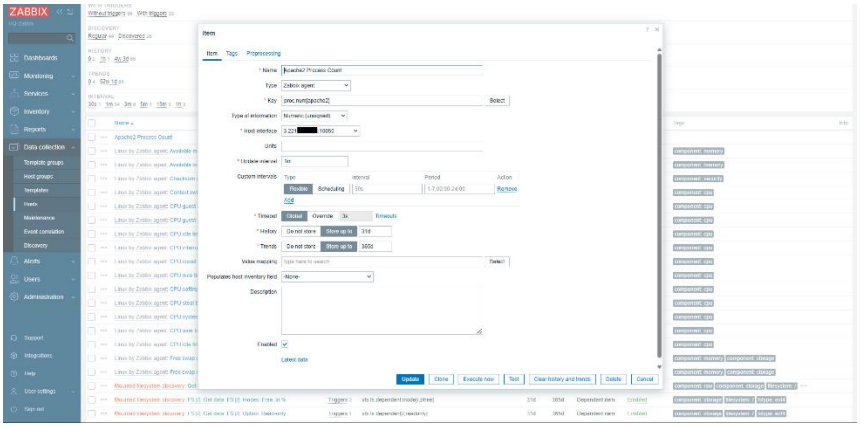
Gambar 4. 151 zammad ticket

4.4.4 Mendeteksi custom service *(Apche2 Service Down)*

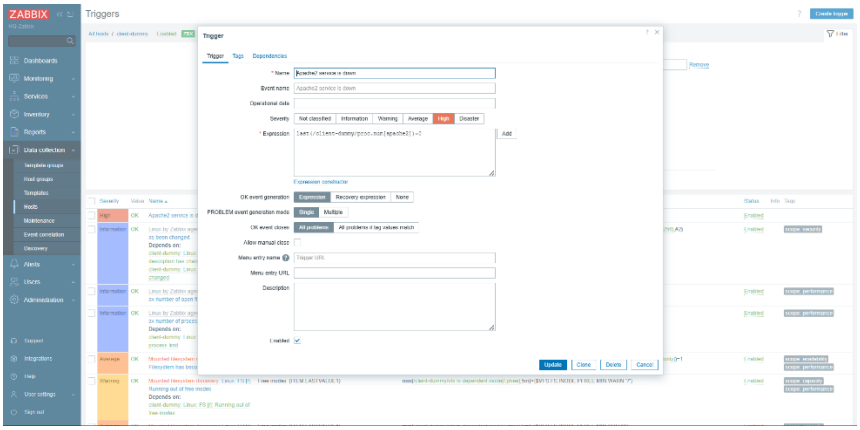
Pengujian ini bertujuan memverifikasi kemampuan Zabbix dalam mendeteksi gangguan layanan (service down) pada Apache2 serta menguji workflow otomatisasi. Dimulai dari pemicu action, pengiriman webhook ke n8n serta notifikasi Telegram dan pembuatan tiket di Zammad.

a) Skema pengujian

1) Pada host client dummy telah dibuat custom item dan trigger yang akan melakukan pengecekan pada service apacher2.

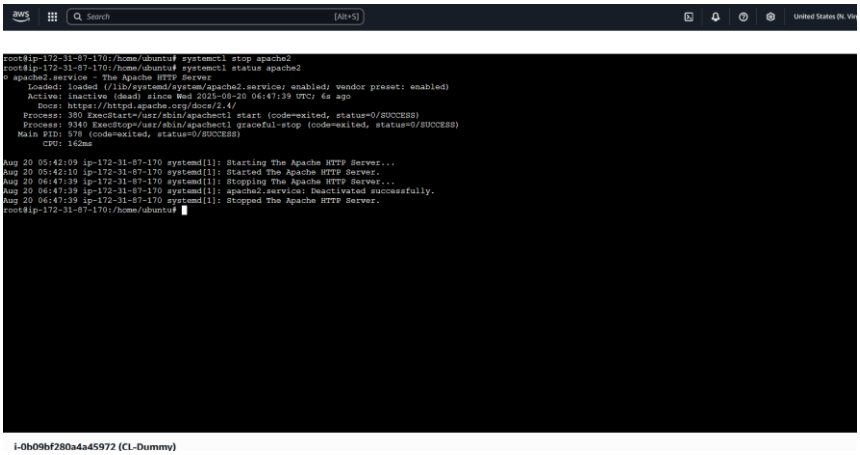


Gambar 4. 152 custo items



Gambar 4. 153 custom trigger

2) Matikan Apache2 secara manual (simulate failure).



Gambar 4. 154 mematikan apche2 service

3) Kasus nyata yang bisa menyebabkan service mati seperti crash aplikasi, kegagalan saat deploy / patching, konflik port (service lain menempati 80), out-of-memory, salah konfigurasi (misconfiguration), atau manual stop yang tidak disadari.
4) Zabbix Agent pada host client-dummy mengirim data item ke Zabbix Server.
5) Trigger Apache2 service is DOWN aktif saat kondisi terpenuhi ≥1 menit (sesuai ekspresi).

6) Action dieksekusi: Script kirim alert ke n8n, kemudian n8n menjalankan workflow untuk mengirim pesan ke Telegram, dan pembuatan tiket.

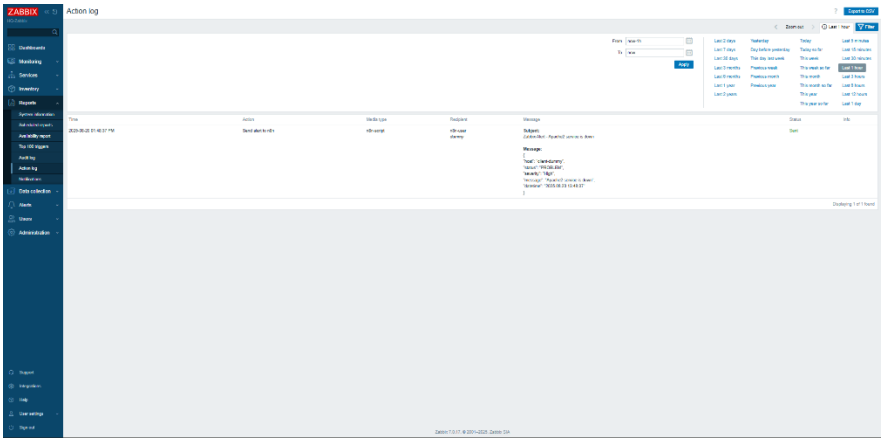
b) Hasil pengujian

1) Problem terbentuk di Zabbix (severity High) dengan nama Apache2 service is DOWN ayng dapat dimonitoring melalui grafana dahsboard yang telah dibuat.



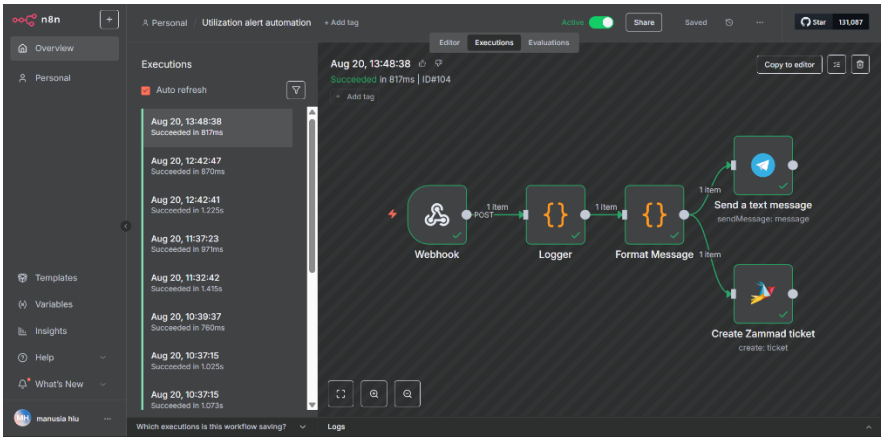
Gambar 4. 155 grafana dashboard

2) Action Log Tercatat operation Send alert to n8n dengan status sent



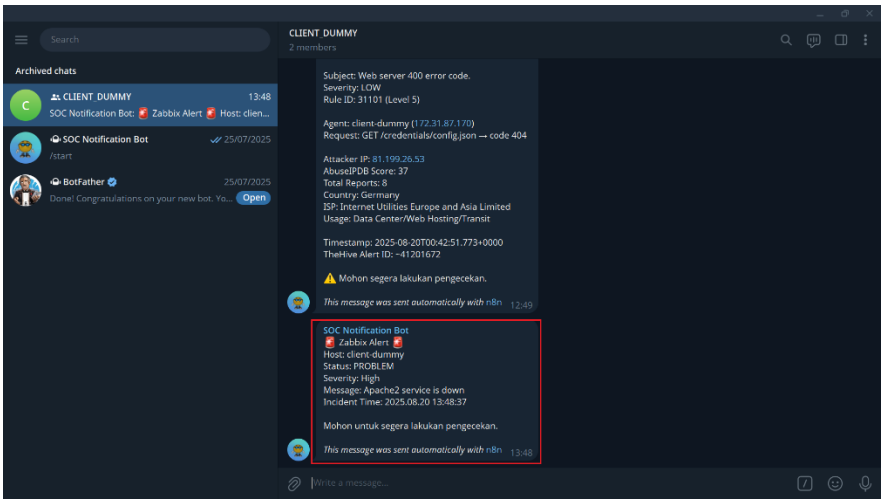
Gambar 4. 156 action log

3) n8n berhasil menjalankan automatin workflow



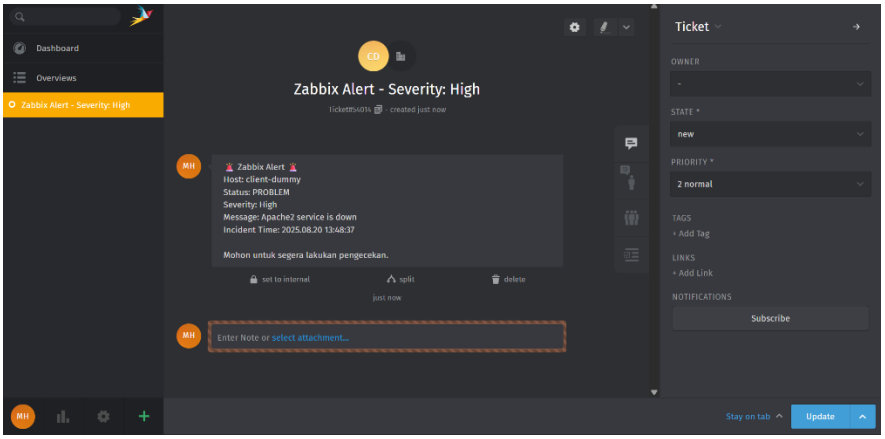
Gambar 4. 157 workflow execution

4) Telegram mengirimkan ke grup insiden berisikan informasi terkait dari alert yang terjadi.



Gambar 4. 158 pesan telegram

5) Zammad membuat ticket insiden



Gambar 4. 159 zammad ticket

4.4.5 Pengujian Latensi & Performa End-to-End SOC

Selain memastikan sistem Security Operations Center (SOC) berbasis open source dapat mendeteksi dan merespons insiden secara fungsional, penting pula untuk mengukur seberapa cepat alur deteksi hingga notifikasi berlangsung. Pengukuran waktu respon (latency) ini dilakukan mulai dari saat serangan dipicu (attack trigger) hingga pesan notifikasi diterima melalui Telegram serta tiket tercatat di Zammad. Hasil pengujian ini memberikan gambaran performa end-to-end dari pipeline SOC yang dibangun, sekaligus menunjukkan efektivitas integrasi antar komponen, telah dilakukan tujuh test meliputi:

a) Scanning Attack

Tabel 4. 1 Scanning Attack

TestID	01
T0 Attack Sent	13:02:41
T1 Siem + Ids Detect	13:02:42
T2 Thehive Aler	13:02:42
T3 n8n start	13:03:30
T4 Telegram Sent	13:03:41
T5 Zammad Ticket	13:03:41
Total	60 detik

b) Invalid checksum

Tabel 4. 2 Invalid Checksum

TestID	02
T0 Attack Sent	13:10:29
T1 Siem + Ids Detect	13:10:33
T2 Thehive Aler	13:10:33
T3 n8n start	13:11:27
T4 Telegram Sent	13:11:30
T5 Zammad Ticket	13:11:30
Total	61 detik

c) Web Content Discovery Attack

Tabel 4. 3 Web Content Discovery Attack

TestID	03
T0 Attack Sent	13:15:18
T1 Siem + Ids Detect	13:15:19
T2 Thehive Aler	13:15:19
T3 n8n start	13:15:27
T4 Telegram Sent	13:15:39
T5 Zammad Ticket	13:15:39
Total	21 detik

d) XSS Injection

Tabel 4. 4 XSS Injection

TestID	04
T0 Attack Sent	13:18:15
T1 Siem + Ids Detect	13:18:16
T2 Thehive Aler	13:18:16
T3 n8n start	13:18:27
T4 Telegram Sent	13:18:38

T5 Zammad Ticket	13:18:38
Total	23 detik

e) Sql injection attempt

Tabel 4. 5 Sql injection attempt

TestID	05
T0 Attack Sent	13:21:13
T1 Siem + Ids Detect	13:21:13
T2 Thehive Aler	13:21:13
T3 n8n start	13:21:27
T4 Telegram Sent	13:21:38
T5 Zammad Ticket	13:21:38
Total	25 detik

f) Apache2 Service is down

Tabel 4. 6 Apache2 Service is down

TestID	06
T0 Attack Sent	13:26:12
T1 Zabbix Detect	13:26:37
T2 Zabbix Sent alert	13:26:37
T3 n8n start	13:26:37
T4 Telegram Sent	13:26:38
T5 Zammad Ticket	13:26:38
Total	26 detik

g) Client has been restarted

Tabel 4. 7 Client has been restarted

TestID	07
T0 Attack Sent	13:28:59
T1 Zabbix Detect	13:30:06

T2 Zabbix Sent alert	13:30:06
T3 n8n start	13:30:06
T4 Telegram Sent	13:30:07
T5 Zammad Ticket	13:30:07
Total	68 detik

4.4.6 Estimasi AWS Penggunaan AWS ec2

Berikut merupakan estimasi biaya penggunaan infrastruktur AWS EC2 untuk setiap instance yang digunakan dalam perancangan SOC, dengan rincian biaya harian, bulanan, dan tahunan:

Tabel 4. 8 Aws Estimate

No	Instance	Tipe EC2	Disk	Biaya Harian (USD)	Biaya Bulanan (USD)	Biaya Tahunan (USD)
1	HQ-Wazuh-Suricata	t3a.large	60GB	1.99	59.70	716.40
2	HQ-Zabbix-Grafana	t2.medium	60GB	1.29	38.67	464.04
3	HQ-TheHive-Cortex	t2.large	80GB	2.47	74.14	889.68
4	HQ-Zammad-n8n	t2.large	60GB	2.42	72.54	870.48
Total				8.17	245.05	2,940.60

BAB V
PENUTUP

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian, dapat disimpulkan bahwa integrasi berbagai komponen open source seperti Wazuh, Suricata, Zabbix, Grafana, TheHive, Cortex, Zammad, dan n8n berhasil membentuk sebuah Security Operations Center (SOC) yang ekonomis, fleksibel, dan fungsional. Sistem ini mampu mendeteksi security alert berupa aktivitas pemindaian jaringan dan forced browsing, serta utilization alert seperti layanan Apache2 yang berhenti maupun kondisi host yang direstart. Setiap alert yang terdeteksi dapat diproses secara otomatis melalui n8n untuk kemudian dikirimkan ke TheHive sebagai kasus, diteruskan ke Telegram sebagai notifikasi cepat terkait alert, dan dicatat di Zammad sebagai tiket insiden. Hasil pengujian membuktikan bahwa SOC berbasis open source ini mampu meningkatkan efektivitas deteksi, mempercepat respons terhadap insiden, serta mendukung orkestrasi alur kerja keamanan yang terintegrasi. Dengan demikian, SOC yang dirancang tidak hanya memberikan solusi yang terjangkau bagi organisasi dengan keterbatasan sumber daya, tetapi juga membuktikan bahwa pendekatan berbasis open source dapat menjadi alternatif andal untuk meningkatkan kapabilitas pertahanan siber modern.

5.2 Saran

Pada Tugas Akhir ini berhasil dirancang dan diimplementasikan SOC berbasis *open source* yang mampu mendeteksi serta merespons insiden secara otomatis. Meskipun demikian, sistem yang dibangun masih memiliki ruang untuk pengembangan agar dapat berfungsi lebih optimal, adaptif, dan sesuai dengan kebutuhan lingkungan produksi. Beberapa saran yang dapat dilakukan antara lain:

1. Fine-tuning rule pada Wazuh dan Suricata untuk mengurangi false positive serta menyesuaikan dengan konteks spesifik organisasi.

2. Pengembangan workflow otomasi di n8n agar mencakup proses eskalasi yang lebih kompleks, misalnya integrasi ke threat intelligence feeds eksternal atau incident prioritization otomatis.
3. Integrasi dengan kontrol akses jaringan (seperti firewall atau security group) sehingga sistem mampu melakukan automated blocking terhadap IP atau host berbahaya berdasarkan IOC (Indicators of Compromise).
4. Penerapan mekanisme redundansi dan high availability pada komponen utama SOC agar sistem tetap andal saat menghadapi kegagalan server atau lonjakan trafik.
5. Ekspansi kapabilitas visualisasi melalui dashboard yang lebih kaya di Grafana untuk mendukung analisis tren insiden jangka panjang.

DAFTAR PUSTAKA

[1] IBM, “cost-of-a-data-breach-report-2025-executive-summary,” 2025, doi: 10.1787/ffa8d501-id.

[2] CROWDSTRIKE, “CrowdStrike Global Threat Report 2025,” 2025.

[3] FORTINET, “GLOBAL THREAT LANDSCAPE REPORT 2025 A Report by FortiGuard Labs,” 2025, [Online]. Available: https://www.fortinet.com/content/dam/fortinet/assets/threat-reports/threat-landscape-report-2025.pdf

[4] IBM, “What is Cybersecurity?,” IBM Think. Accessed: Aug. 19, 2025. [Online]. Available: https://www.ibm.com/think/topics/cybersecurity

[5] ENISA, *How to set up CSIRT and SOC -- Good Practice Guide*, no. December. 2020. doi: 10.2824/056764.

[6] Australian Signals Directorate, “Implementing SIEM and SOAR platforms: Practitioner guidance | Cyber.gov.au”, [Online]. Available: https://www.cyber.gov.au/resources-business-and-government/maintaining-devices-and-systems/system-hardening-and-administration/system-monitoring/implementing-siem-and-soar-platforms/implementing-siem-and-soar-platforms-practitioner-guidance

[7] RedHat, “What is an intrusion detection and prevention system (IDPS),” Security. Accessed: Aug. 19, 2025. [Online]. Available: https://www.redhat.com/en/topics/security/what-is-an-IDPS

[8] Wazuh Inc., “Getting started with Wazuh,” Wazuh.Com. Accessed: Aug. 09, 2025. [Online]. Available: https://documentation.wazuh.com/current/getting-started/index.html

[9] OISF, “What is Suricata,” Https://Docs.Suricata.Io/En/Latest/What-Is-Suricata.Html. Accessed: Aug. 09, 2025. [Online]. Available: https://docs.suricata.io/en/latest/what-is-suricata.html

[10] Zabbix SIA, “Overview of Zabbix,” zabbix.com. Accessed: Aug. 19, 2025. [Online]. Available: https://www.zabbix.com/documentation/1.8/en/manual/about/overview_of_zabbix

[11] Grafana Labs, “Grafana fundamentals | Grafana Labs,” Grafana Labs. [Online]. Available: https://grafana.com/tutorials/grafana-fundamentals/?pg=tutorials&plcmt=results

[12] StrangeBee, “TheHive - Overview,” StrangeBee Docs. Accessed: Aug. 19, 2025. [Online]. Available: https://www.strangebee.com/thehive

[13] StrangeBee, “Cortex - Documentation,” StrangeBee Docs. Accessed: Aug. 19, 2025. [Online]. Available: https://docs.strangebee.com/cortex/

[14] n8n, “n8n Documentation,” n8n Docs. Accessed: Aug. 19, 2025. [Online]. Available: https://docs.n8n.io/

[15] TelegramBot, “Telegram Bot API,” Telegram Documantation. Accessed: Aug. 19, 2025. [Online]. Available: https://core.telegram.org/bots/api

[16] Zammad GmbH, “Zammad: Open-source helpdesk and ticketing system,” Zammad Website. Accessed: Aug. 19, 2025. [Online]. Available: https://zammad.com/en/company/open-source

[17] Cybersecurity and Infrastructure Security Agency (CISA), “Cybersecurity Incident & Vulnerability Response Playbooks Operational Procedures for Planning and Conducting Cybersecurity Incident and Vulnerability Response Activities in FCEB Information Systems,” no. November, p. 43, 2021, [Online]. Available: https://www.cisa.gov/tlp/.

[18] K. K. Watson, “Cybersecurity Automation and Threat Intelligence Sharing Best Practices UNDERSTANDING SERVICE MODELS FOR CYBER THREAT INTELLIGENCE (CTI) Intelligence, Enrichment, and Brokering as a Service,” pp. 1–3, 2021.

[19] Amazon Web Services Inc., “Amazon EC2 Concepts - User Guide,” AWS Documentation. Accessed: Aug. 19, 2025. [Online]. Available: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html